VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT

CAPSTONE PROJECT

SEMESTER 252 ACADEMIC YEAR 2025-2026

DOMAIN GENERALIZATION FOR ADVERSARIAL ROBUSTNESS

MAJOR: COMPUTER SCIENCE

COUNCIL:

SUPERVISOR(s): Nguyễn An Khương

Văn Minh Hào

REVIEWER:

——o0o——

STUDENT 1: Nguyễn Anh Kiệt - 2252403

STUDENT 2: Lê Văn Đức Anh - 2252027

STUDENT 3: Nguyễn Tiến Thành - 2053437

HO CHI MINH CITY, May 2026

Instructor’s Signature

_____ Date: _____

NGUYỄN AN KHƯƠNG (Instructor)

_____

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Dr. Nguyen An Khuong at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology and Mr. Van Minh Hao at University of Arkansas.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during our research.

Ho Chi Minh City, May 2026

Authors

Acknowledgement

We would like to express our appreciation to Dr. Nguyen An Khuong for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from Mr. Van Minh Hao for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on adversarial attack so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to our family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in our potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

This thesis addresses the critical vulnerability of deep learning models to adversarial perturbations, particularly the limitation where standard defense mechanisms fail to generalize across different threat models. Our research proposes a novel perspective that treats distinct families of adversarial attacks as separate ”domains,” aiming to identify and quantify the ”cross-attack robustness gap” inherent in current training paradigms.

We establish a comprehensive experimental framework to evaluate the transferability of robustness across diverse adversarial environments. By subjecting models trained on specific perturbation norms to a wide spectrum of unseen attack algorithms, the study systematically measures the performance degradation that occurs when shifting between these domains.

Ultimately, this quantitative analysis motivates the proposed method evaluated in this report. By characterizing the domain shifts between attack families, the report integrates Domain Generalization principles into adversarial training and tests whether group-aware multi-attack training can reduce the observed robustness gaps under the stated experimental protocol.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definitions
AT	Adversarial Training
CI	Confidence Interval
CW	Carlini–Wagner
DDN	Decoupled Direction and Norm
DG	Domain Generalization
DNN	Deep Neural Network
DRO	Distributionally Robust Optimization
ERM	Empirical Risk Minimization
FGSM	Fast Gradient Sign Method
FGSM-RS	Fast Gradient Sign Method with Random Start
MI-FGSM	Momentum Iterative Fast Gradient Sign Method
PGD	Projected Gradient Descent
pp	Percentage points
SGD	Stochastic Gradient Descent
TPGD	TRADES-style Projected Gradient Descent
W&B	Weights & Biases

Chapter 1

Introduction

1.1 Motivation

Deep neural networks can achieve high standard accuracy while remaining vulnerable to small, deliberately constructed input perturbations. These adversarial examples are usually constrained by a norm budget, but they can still change the prediction of a trained classifier with high reliability [1, 2]. Adversarial training addresses this failure mode by including adversarially perturbed examples during optimization and is one of the most widely used empirical defenses under specified threat models [3].

The practical limitation is that adversarial training is shaped by the attack used to generate the training examples. A model trained mainly against a projected-gradient ℓ_∞ attack can become strong for that geometry while remaining less reliable under ℓ_2 attacks, boundary-oriented attacks, or attacks with different optimization dynamics. Likewise, a model trained against an ℓ_2 source attack can develop a different pattern of strengths and weaknesses. This report studies that behavior as a cross-attack robustness problem: the question is not whether a model survives one attack, but how its robustness is distributed across seen and held-out attacks.

Multi-attack adversarial training is the main strong baseline for this setting. In this report, Multi-AT means uniform multi-attack adversarial training on two source attacks, PGD-ℓ_∞ and DDN-ℓ_2. This is a stronger practical baseline than training on a single source attack because the model is exposed to both an ℓ_∞ and an ℓ_2 training signal. However, Multi-AT still averages over the generated examples with a fixed rule. If hard examples are defined by class, representation geometry, margin, or optimization trajectory rather than only by attack identity, then fixed source-attack labels are too coarse to describe where training is failing.

This motivates an attacks-as-domains view. Each attack generator induces a domain-like distribution of adversarial examples through its perturbation set, loss objective, and optimization procedure. Group distributionally robust optimization (Group DRO) suggests that training can emphasize groups with higher loss rather than optimizing only an average loss [4]. The key question is how those groups should be defined. Attack identity is one possible grouping, but examples within the same attack can have different difficulty, and examples from different attacks can share similar failure modes.

AttackDRO++ addresses this group-definition problem with latent clusters, gradient fingerprints, and a uniform anchor. Latent clusters replace fixed attack identities with groups discovered from adversarial examples. Gradient fingerprints add optimization-aware information, so grouping is not based only on representation similarity. The uniform anchor keeps the adaptive cluster-level objective tied to the Multi-AT loss, making AttackDRO++ a refinement of Multi-AT rather than a replacement for multi-attack training.

The intended claim is deliberately bounded. The goal is not to establish unrestricted robustness or a new universal defense. Instead, the report asks whether AttackDRO++ can provide a modest but consistent average improvement and lower observed seed-to-seed variation over Multi-AT under the evaluated CIFAR-10/ResNet-18 setting, while keeping hardest-case and AutoAttack behavior separate from the main aggregate ranking.

1.2 Problem Statement

This report studies robust image classification under a controlled multi-attack training protocol. A classifier is trained using adversarial examples from a fixed source attack set. For the multi-attack methods, the shared source attacks are PGD-ℓ_∞ and DDN-ℓ_2. The trained classifier is then evaluated on both seen-source attacks and held-out attacks that differ in norm, objective, and attack dynamics.

The central problem is how to use the source attacks without overfitting the training objective to coarse attack identities. Single-source adversarial training can specialize to one perturbation family. Multi-AT reduces this specialization by training on both source families, and therefore serves as the main practical baseline. Yet fixed attack identities may still hide important variation inside each attack family. Some examples can be difficult because of their class, feature representation, local margin, or induced gradient signal, not simply because they were generated by PGD or DDN.

AttackDRO++ is motivated by this latent difficulty structure. It discovers cluster groups from adversarial examples, enriches those groups with gradient fingerprints, and uses an

anchored Cluster-DRO objective to emphasize harder groups without discarding the uniform multi-attack signal. The main empirical comparison focuses on PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ under the same CIFAR-10/ResNet-18 training and evaluation protocol. AttackDRO and Cluster-DRO are used as design steps in the methodology, not as the primary Chapter 6 comparison set.

Evaluation follows the same distinction between source and held-out attacks. Mean(8) averages robust accuracy over the eight non-AutoAttack evaluation attacks. Worst(8) reports the weakest attack in that same suite, and held-out summaries separate attacks that were not used as training sources. AutoAttack-ℓ_∞ is reported separately as a standardized stress test and is not part of Mean(8) or the main aggregate ranking [5]. All claims are therefore tied to the defined attack suite, paired seed protocol, and model family used in the experiments.

1.3 Research Questions and Objectives

The report is organized around four research questions that can be answered by the evidence in Chapter 6 and summarized in Chapter 7.

RQ1: To what extent does single-source adversarial training produce attack-dependent robustness across attacks with different norms, objectives, or optimization behavior?

RQ2: Does Multi-AT improve robustness balance across seen and held-out attacks compared with single-source adversarial training baselines?

RQ3: Does AttackDRO++ provide a modest but consistent average improvement over Multi-AT, while leaving hardest-case and AutoAttack behavior broadly unchanged under the evaluated protocol?

RQ4: How sensitive is AttackDRO++ to practical configuration choices such as cluster count, anchor strength, and cluster refresh schedule?

These questions lead to four objectives. First, the report defines a controlled multi-attack training and evaluation pipeline with fixed source attacks, held-out attacks, paired seed comparisons, and separate AutoAttack reporting. Second, it develops AttackDRO++ as a latent-group refinement of Multi-AT using clusters, gradient fingerprints, and a uniform anchor. Third, it compares PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ using aggregate, per-attack, class-wise, graybox, and hardest-case diagnostics. Fourth, it interprets ablations and supplementary architecture checks as configuration and robustness evidence, not as broad claims beyond the evaluated setting.

1.4 Contributions

This report makes four contributions within the bounded CIFAR-10/ResNet-18 setting defined in Chapter 5.

1. It frames the task as cross-attack adversarial robustness and uses an attacks-as-domains view to separate source attacks from held-out attacks. The evaluation protocol reports per-attack accuracy, Mean(8), Worst(8), held-out summaries, and a separate AutoAttack-ℓ_∞ stress test.
2. It defines AttackDRO++ as the final proposed method: uniform-anchored Cluster-DRO with gradient fingerprints. The method combines latent cluster groups, optimization-aware gradient fingerprints, and a uniform anchor that keeps the adaptive objective connected to the Multi-AT baseline.
3. It provides a controlled empirical comparison in which the main methods are PGD-AT, DDN-AT, Multi-AT, and AttackDRO++. Under the evaluated CIFAR-10/ResNet-18 setting, the evidence supports a modest but consistent average improvement and lower observed seed-to-seed variation versus Multi-AT, while hardest-case and AutoAttack results remain broadly similar.
4. It adds diagnostic evaluation beyond the main aggregate score, including bounded multi-attack whitebox evaluation, graybox transfer, class-wise analysis, configuration ablations, and a supplementary WRN-28-10 check. These analyses are used to calibrate the claims rather than to argue for broad architecture or attack-suite generalization.

1.5 Report Organization

Chapter 2 introduces the background needed for the report: adversarial examples, attack algorithms, adversarial training, distributionally robust optimization, domain generalization, clustering, and statistical tools for paired multi-seed evaluation. Appendix A provides compact pseudocode for the attack and training procedures.

Chapter 3 reviews multi-attack adversarial training, Group DRO, domain generalization, and cluster discovery. It uses these areas to identify the gap addressed by latent group discovery with an anchored multi-attack objective.

Chapter 4 defines the method progression from Multi-AT to AttackDRO, Cluster-DRO, and the final AttackDRO++ training framework. Chapter 5 specifies the dataset, architecture, source attacks, evaluation suite, metrics, hyperparameters, statistical protocol, and graybox transfer design.

Chapter 6 presents the empirical analysis: aggregate whitebox robustness, per-attack and class-wise diagnostics, graybox transfer, ablation studies, and supplementary checks. Chapter 7 answers the research questions, states the limitations of the evidence, and outlines future work needed for broader datasets, architectures, and attack settings.

Chapter 2

Background

2.1 Notation and Learning Setting

This chapter fixes the technical vocabulary used by the later methodology and evaluation chapters. The goal is not to survey every defense or attack, but to define the optimization setting in which adversarial examples, multi-attack training, group reweighting, clustering, and multi-seed evaluation are used. The notation is kept deliberately close to the notation used in Chapters 4 and 5.

We consider supervised classification with input space $\mathcal{X} \subseteq \mathbb{R}^d$ and label space $\mathcal{Y} = \{1, \dots, C\}$. A training set is denoted by

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n},$$

where each pair is sampled from an underlying data distribution P_{XY}. A classifier with parameters θ maps an input to logits

$$f_\theta : \mathcal{X} \to \mathbb{R}^C,$$

and predicts $\hat{y} = \arg\max_c f_\theta(\mathbf{x})_c$. The softmax transformation converts logits into class probabilities,

$$p_\theta(y = c \mid \mathbf{x}) = \frac{\exp(f_\theta(\mathbf{x})_c)}{\sum_{j=1}^{C} \exp(f_\theta(\mathbf{x})_j)}.$$

2.1.1 Deep Neural Networks and Supervised Classification

Deep neural networks are used here as parameterized classifiers rather than as objects of architectural study. Their relevance to this report is that the same differentiable structure used for gradient-based learning also exposes gradients with respect to the input. These input gradients are the basic computational object behind many first-order adversarial attacks, including FGSM and PGD [2, 3].

For a labeled example $(\mathbf{x}, y)$, the standard classification loss is cross-entropy,

$$\ell(f_\theta(\mathbf{x}), y) = -\log p_\theta(y \mid \mathbf{x}).$$

Empirical risk minimization (ERM) fits the model by minimizing average training loss:

$$\min_\theta \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(\mathbf{x}_i), y_i).$$

ERM is the clean-data reference point for the rest of the report. It optimizes performance on sampled training examples, but it does not require the prediction to remain stable inside a local neighborhood of each input. Adversarial robustness adds precisely this local stability requirement.

2.1.2 Loss Functions and Gradient-Based Optimization

In gradient-based training, the model parameters are updated using derivatives of the loss with respect to θ. In gradient-based attacks, the adversary instead uses derivatives with respect to $\mathbf{x}$. A small perturbation δ can therefore be selected to increase the loss while remaining visually or semantically close to the original input:

$$\arg\max_c f_\theta(\mathbf{x})_c \neq \arg\max_c f_\theta(\mathbf{x} + \delta)_c, \quad \|\delta\|_p \leq \varepsilon.$$

This mismatch between small input displacement and large prediction change was established by early adversarial-example work and remains the core failure mode studied in adversarial training [1, 2].

The later chapters use this notation to distinguish attack algorithms from training methods. For example, PGD-ℓ_∞ is an attack used to generate perturbed examples, whereas PGD-AT is a training method that repeatedly trains on PGD-generated examples. Likewise, DDN-ℓ_2 is an attack, while DDN-AT is a single-source adversarial training baseline. This distinction matters because Chapter 4 compares training strategies while holding the source attack set fixed.

2.2 Adversarial Examples and Threat Models

An adversarial example is an input deliberately perturbed to induce an incorrect model prediction while preserving the ground-truth semantic label. Given a clean example (x, y), an adversarial example has the form

$$x^{\text{adv}} = x + \delta.$$

In an untargeted attack, the adversary searches for a perturbation that makes the classifier predict any class other than y. In a targeted attack, the adversary instead tries to force prediction of a chosen target class $y_{\text{target}} \neq y$. This report primarily uses untargeted robustness evaluation, with TPGD treated according to the TRADES-style implementation described in Chapter 5.

2.2.1 Norm-Bounded Perturbation Sets

Most image-classification robustness evaluations constrain δ by an ℓ_p budget. The feasible perturbation set is

$$\mathcal{S}_p(\varepsilon) = \{\delta \in \mathbb{R}^d : \|\delta\|_p \leq \varepsilon\},$$

with p usually chosen as 2 or ∞. The perturbed input is also clipped to the valid input range, so the effective search region is

$$x^{\text{adv}} \in (x + \mathcal{S}_p(\varepsilon)) \cap [0, 1]^d.$$

The label-preservation assumption states that the semantic label should not change inside this small perturbation set. Under that assumption, a misclassification at x^{adv} is treated as a robustness failure rather than a change in the underlying task.

For a fixed model and loss, adversarial example generation is often written as an inner maximization:

$$\delta^* \in \arg\max_{\delta \in \mathcal{S}_p(\varepsilon)} \ell(f_\theta(x + \delta), y).$$

Different attacks approximate this maximization in different ways. Some follow the input gradient, some optimize a margin-based objective, and others estimate a nearby decision boundary. These algorithmic differences are important because robustness against one perturbation family or loss surrogate does not necessarily transfer to another [5–7].

For attack condition e, let $\mathcal{A}_e$ denote the corresponding adversarial example generator.

The generated input can be written as

$$x_e^{\text{adv}} = \mathcal{A}_e(x, y, f_\theta),$$

and the associated attack-domain risk is

$$R_e(\theta) = \mathbb{E}_{(x,y)\sim P_{XY}} \left[\ell(f_\theta(x_e^{\text{adv}}), y) \right].$$

This notation supports the later multi-attack setting: a method can reduce risk for one source attack while leaving another attack-domain risk high. The training methods in this report therefore compare not only average adversarial risk, but also how training emphasis is allocated across attack-induced conditions and latent groups.

2.2.2 Whitebox, Graybox, and Blackbox Access

The perturbation set specifies what changes are allowed in input space, while the threat model specifies what information the adversary can use. In the whitebox setting, the attacker has access to the model architecture, parameters, loss, and gradients. First-order attacks such as FGSM and PGD are whitebox attacks because they use $\nabla_x \ell(f_\theta(x), y)$ directly. Whitebox evaluation is the standard setting for testing gradient-based robustness and is also the natural setting for adversarial training, where the current model generates its own adversarial examples [3, 8].

In the graybox setting, the adversary has partial information. A common case is transfer evaluation: adversarial examples are generated on a surrogate model f_{θ_s} and evaluated on a target model f_{θ_t}. This tests whether perturbations built from one loss landscape remain harmful on another. Graybox transfer is relevant to the attacks-as-domains view because it changes the mechanism that generates the adversarial distribution without necessarily changing the data distribution itself.

In the blackbox setting, the adversary does not access model parameters or gradients and instead relies on queries. Score-based attacks use probabilities or logits, while decision-based attacks use only predicted labels. Blackbox attacks are useful for detecting gradient masking, where a defense appears strong because gradients are uninformative rather than because the classifier has learned genuinely robust decision boundaries [8, 9]. The main experiments in this report focus on the specified whitebox evaluation suite and separate stress checks, while the blackbox setting remains part of the broader threat-model vocabulary.

Because the later protocols combine direct whitebox attacks, surrogate transfer, and stan-

dardized stress checks, Table 2.1 fixes the access assumptions before the individual attack algorithms are introduced.

Table 2.1: Threat-model access assumptions used in this report.

Threat model	Parameters	Gradients	Scores/logits	Used?	Role in this report
Whitebox	Yes	Yes	Yes	Yes	Main setting for training attacks and direct robustness evaluation.
Graybox	Surrogate only	Surrogate only	Protocol-dependent	Yes	Transfer setting where adversarial examples are generated on a surrogate and tested on a target model.
Blackbox	No	No	Query only	Limited	Appears through score-based components of standardized evaluation, especially AutoAttack.

These definitions prepare the move from individual attacks to training objectives. Once attacks are viewed as generators of distinct adversarial risks, multi-attack and group-aware training can be framed as choices about which risks are optimized, how they are weighted, and how hidden failure modes are discovered.

2.3 Adversarial Attack Algorithms

The attacks used in this report are best understood as different approximations to the same inner maximization problem. They vary in perturbation norm, attacker access, optimization objective, update rule, and role in training or evaluation. Some attacks, such as PGD-based methods, directly maximize a loss within a fixed norm ball, while others, such as DeepFool, DDN, and CW, search for low-norm or boundary-oriented adversarial examples. These differences matter because a model that is robust to one attack procedure may still fail under another, which is the core motivation for the attacks-as-domains.

Chapter 5 gives the concrete configuration values, including perturbation budgets, step counts, and which attacks are used for training or evaluation. This section instead defines the attacks conceptually so that later comparisons can refer to them without reintroducing their mechanics. Appendix A provides compact pseudocode for the attack and training procedures used throughout the report.

2.3.1 Fast Gradient Sign Method with Random Start (FGSM-RS)

Fast Gradient Sign Method with Random Start (FGSM-RS) is motivated by the need for a cheap first-order evaluation attack. It operates in the whitebox ℓ_∞ threat model: the adversary can access the model gradient and must keep the perturbation inside an ℓ_∞ ball around the clean input. The base FGSM attack uses a single gradient-sign step to increase the cross-entropy loss [2].

The objective is the usual untargeted inner maximization of $\ell(f_\theta(x+\delta), y)$ over an ℓ_∞ budget. FGSM-RS first samples a random initial perturbation $\delta_0 \in \mathcal{S}_\infty(\varepsilon)$ and then takes one sign-gradient step:

$$x^{\text{adv}} = \Pi_{(x+\mathcal{S}_\infty(\varepsilon))\cap[0,1]^d}\left(x + \delta_0 + \alpha\,\text{sign}\left(\nabla_x \ell(f_\theta(x+\delta_0), y)\right)\right).$$

The random start reduces dependence on the clean point's immediate gradient direction and makes the one-step attack less brittle than deterministic FGSM [10].

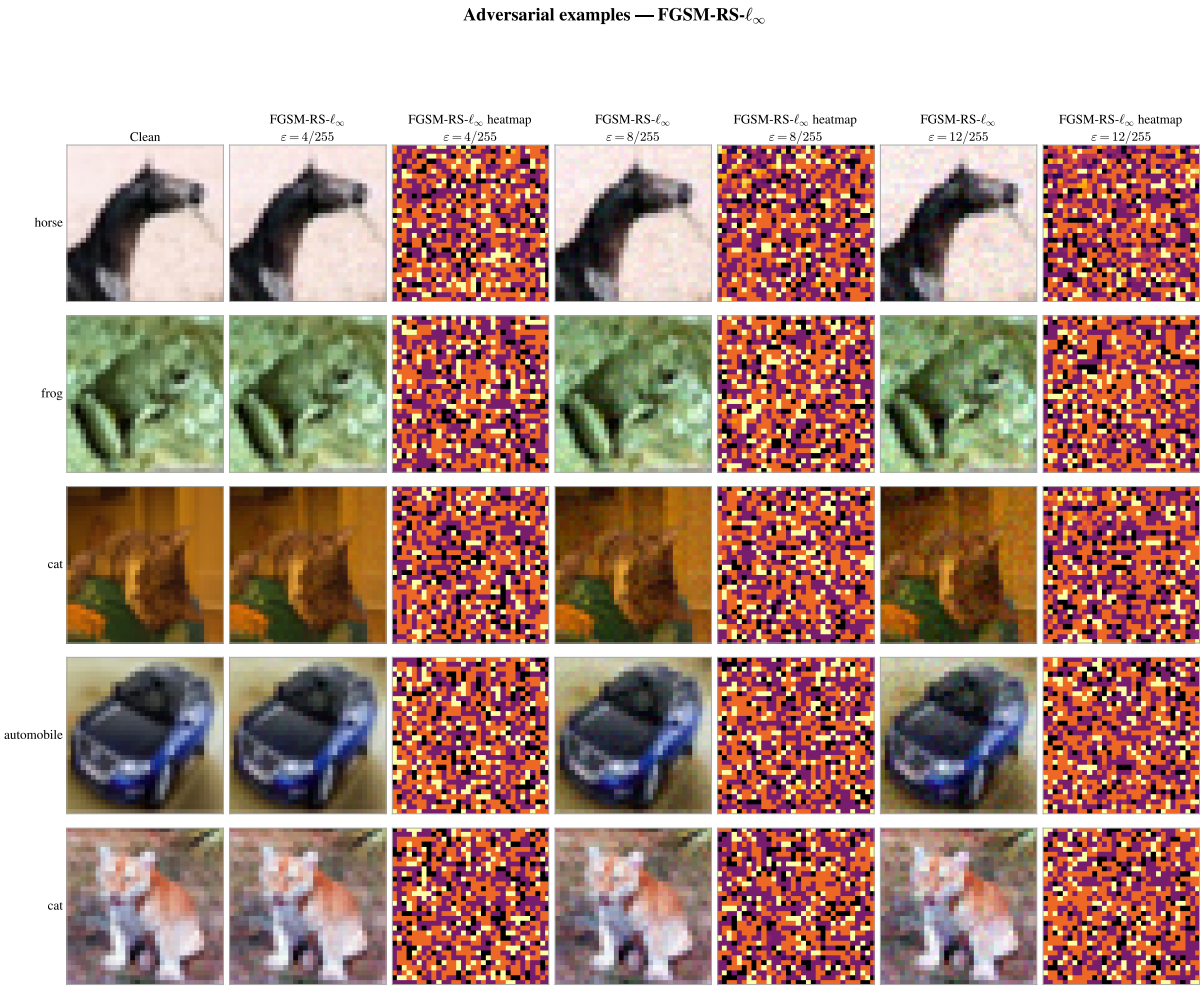


Figure 2.1: FGSM-RS-ℓ_∞ adversarial examples on CIFAR-10 under three perturbation budgets. Each row shows the clean image, adversarial image, and normalized perturbation heatmap.

FGSM-RS is fast and useful for exposing obvious failures, but it is weaker than multi-step attacks and should not be treated as a complete robustness evaluation. In this thesis it is used only as a held-out ℓ_∞ evaluation attack. It is not a source training attack and does not define a training method.

2.3.2 Projected Gradient Descent (PGD)

Projected Gradient Descent (PGD) is the standard iterative first-order attack used to approximate the adversarial inner maximization [3]. It is a whitebox attack because each step uses the current input gradient. In this report, PGD-ℓ_∞ is both a source training attack and an evaluation attack, while PGD-ℓ_2 is used only for evaluation.

For an untargeted ℓ_∞ attack, PGD repeatedly ascends the loss and projects the result back to the feasible perturbation set:

$$x_{t+1}^{\text{adv}} = \Pi_{(x+\mathcal{S}_\infty(\varepsilon))\cap[0,1]^d}\left(x_t^{\text{adv}} + \alpha\,\text{sign}\left(\nabla_x \ell(f_\theta(x_t^{\text{adv}}), y)\right)\right).$$

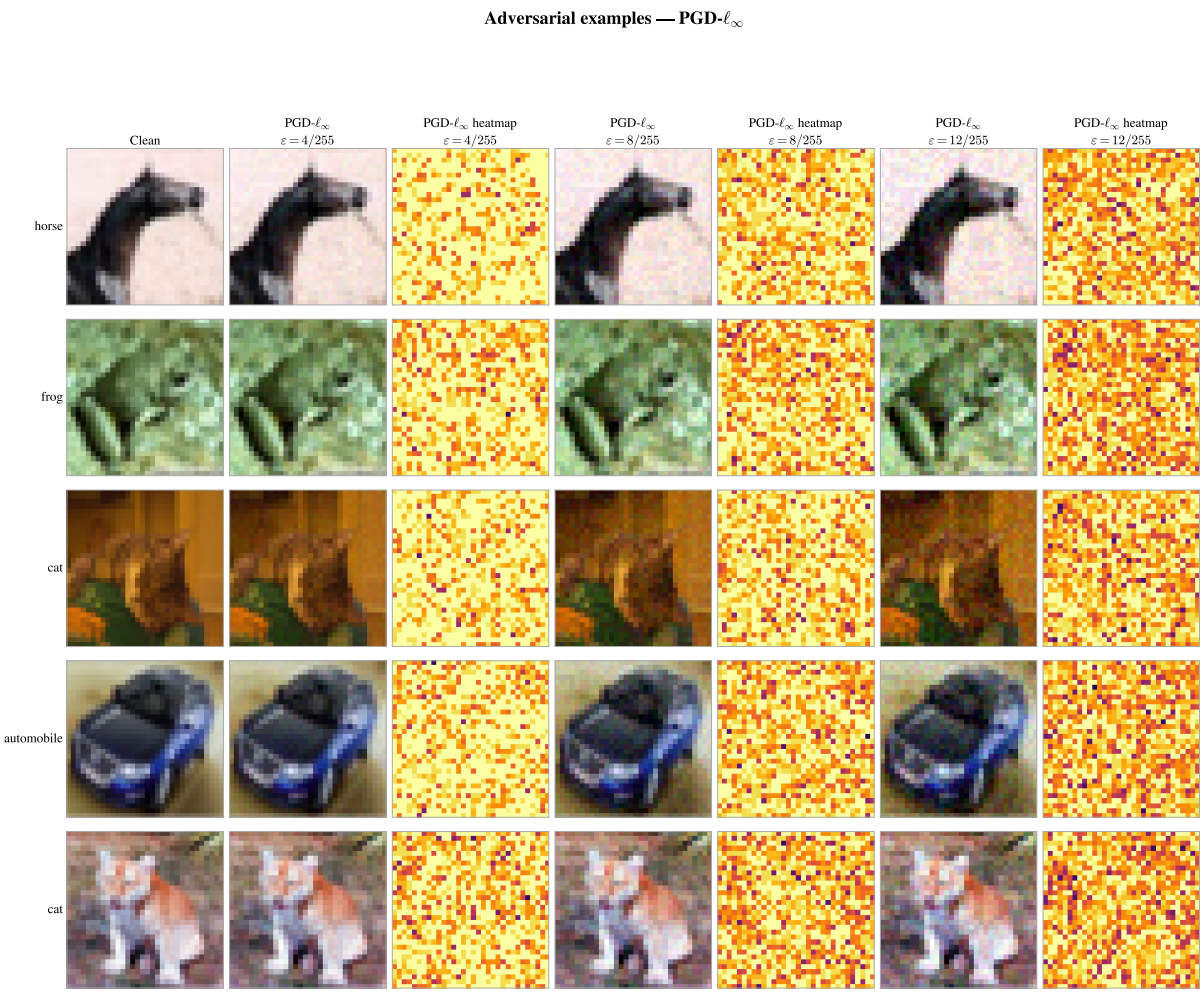


Figure 2.2: PGD-ℓ_∞ adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

For an ℓ_2 variant, the update uses a normalized gradient direction and projection onto the ℓ_2 ball. The algorithmic intuition is simple: repeated local ascent gives the attack more opportunities to find a high-loss point than a single FGSM step.

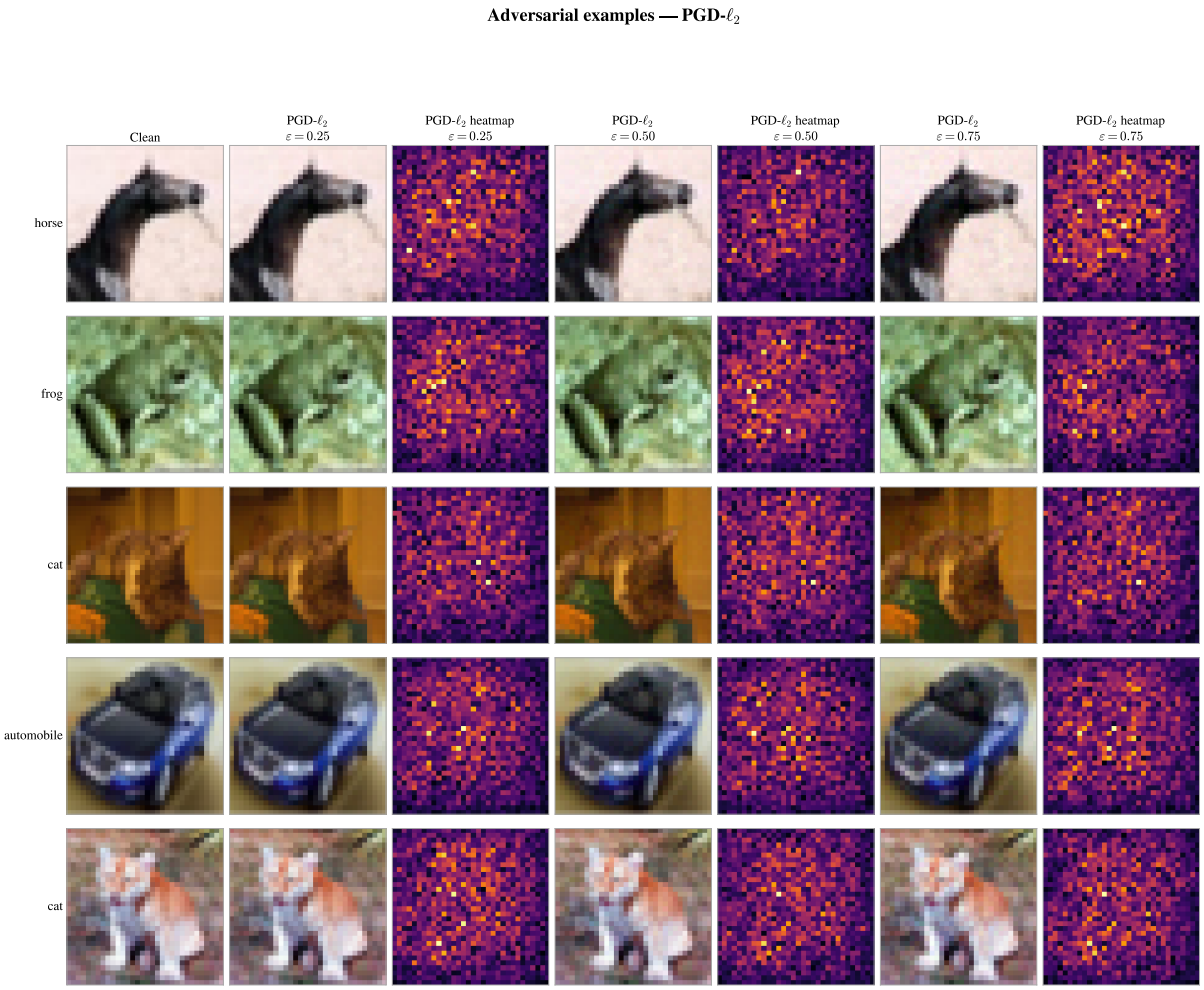


Figure 2.3: PGD-ℓ_2 adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

PGD is strong, broadly used, and closely tied to adversarial training, but it still depends on the chosen norm, loss, step size, and iteration budget. PGD-AT is the training method that uses PGD-generated examples; PGD-ℓ_∞ and PGD-ℓ_2 are the attacks themselves. This distinction is central to the thesis because Multi-AT, AttackDRO, Cluster-DRO, and AttackDRO++ share PGD-ℓ_∞ as a source attack while differing in how generated examples are grouped and weighted.

2.3.3 Carlini–Wagner Attack (CW)

The Carlini–Wagner attack is motivated by the need to test robustness under an optimization objective that differs from standard cross-entropy gradient ascent [11]. It is usually formulated as a whitebox attack and is especially associated with the ℓ_2 threat model.

The adversary searches for a small perturbation that changes the classification decision while controlling perturbation size.

A common CW-style objective combines an ℓ_2 penalty with a margin-based attack loss:

$$\min_{\delta} \|\delta\|_2^2 + c\, g(x+\delta), \quad x+\delta \in [0,1]^d,$$

where $g(\cdot)$ is chosen so that successful adversarial examples have low objective value. This form makes the attack different from direct cross-entropy maximization: the perturbation norm and the classification margin are optimized together.

CW-ℓ_2 is a useful held-out evaluation attack because it probes a different loss geometry from PGD and FGSM-style attacks. Its limitation is computational cost, and its strength depends on optimization settings. In this thesis it is used as an evaluation attack only, not as a source attack for training.

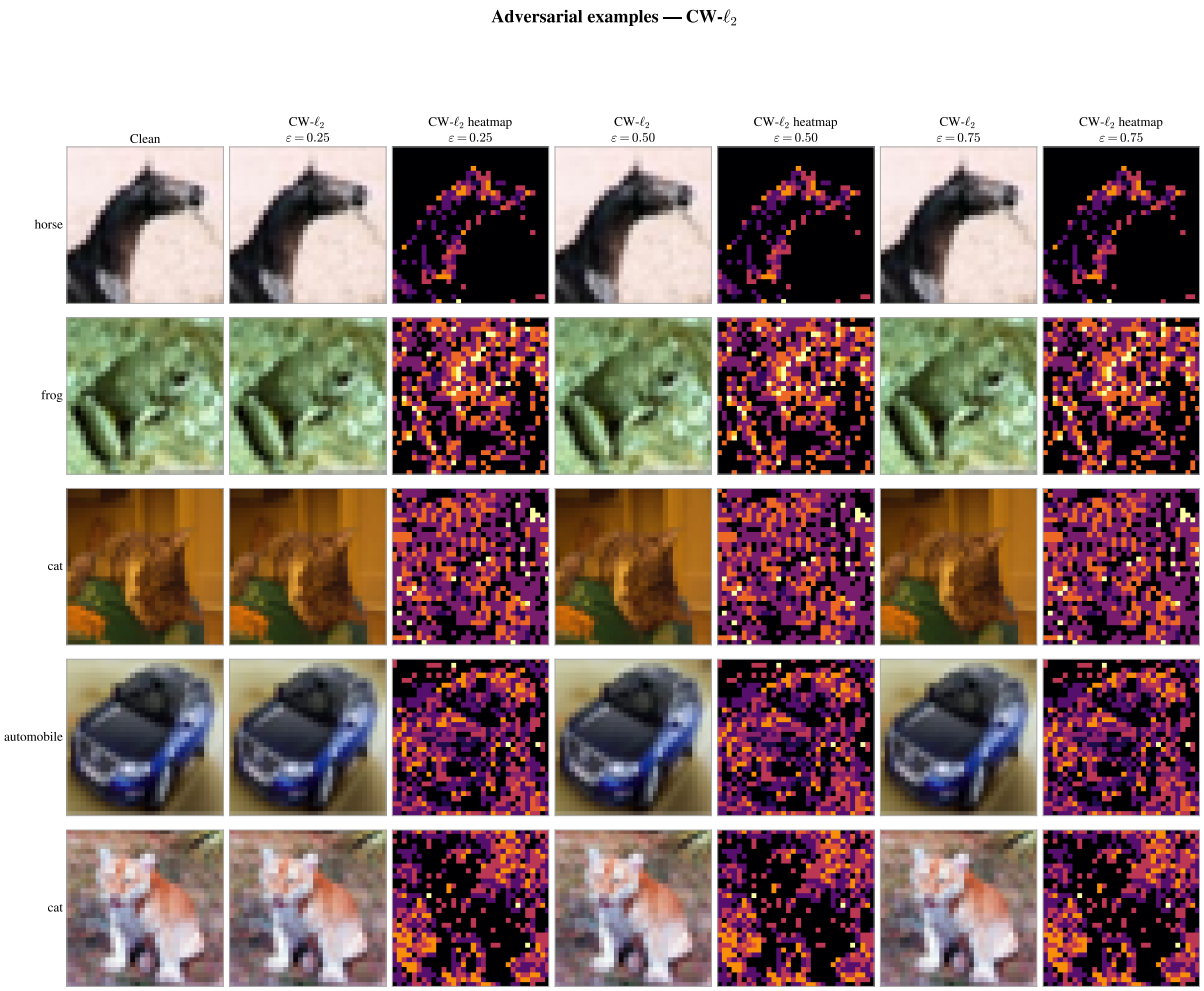


Figure 2.4: CW-ℓ_2 adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

2.3.4 DeepFool

DeepFool is motivated by a geometric view of adversarial examples: if the classifier's decision boundary is nearby, a small movement toward that boundary can change the predicted class [12]. It is a whitebox boundary-oriented attack because it uses local gradient information to approximate the classifier near the current point. The report uses DeepFool-ℓ_2 as a held-out ℓ_2 evaluation attack.

For a locally linearized binary classifier, the smallest boundary-crossing perturbation has the form

$$r_t = -\frac{f(x_t)}{\|\nabla_x f(x_t)\|_2^2} \nabla_x f(x_t).$$

In multiclass neural networks, DeepFool generalizes this idea by estimating the closest class boundary under a local linear approximation and iteratively updating the input until the predicted class changes or the iteration budget is reached.

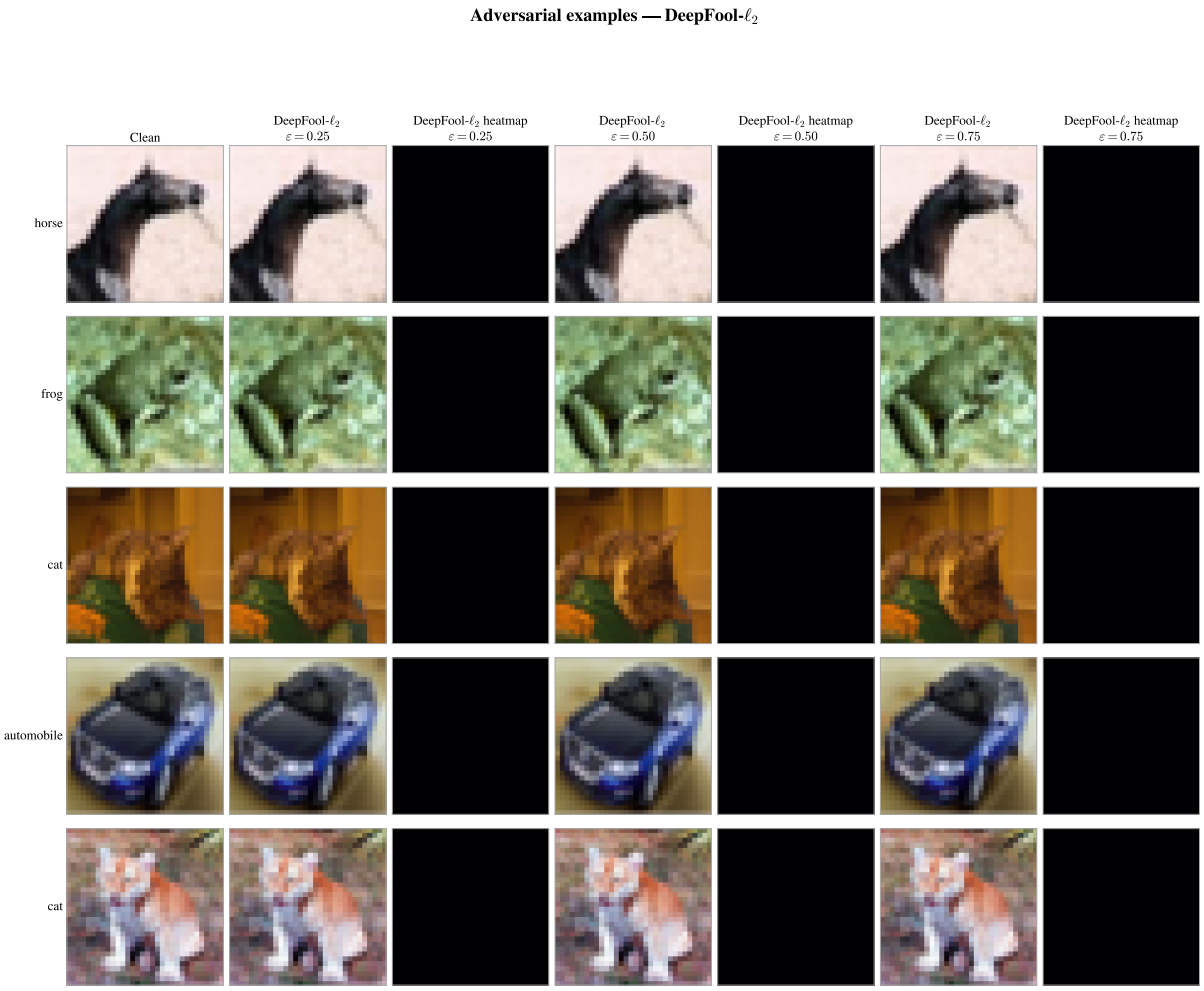


Figure 2.5: DeepFool-ℓ_2 adversarial examples on CIFAR-10 under three perturbation budgets, with normalized boundary-oriented perturbation heatmaps.

DeepFool is valuable because it emphasizes boundary proximity rather than directly maximizing cross-entropy. Its limitation is that local linearization is only an approximation for nonlinear networks, so it is still an attack heuristic rather than a certificate. In this thesis it complements PGD-ℓ_2, DDN-ℓ_2, and CW-ℓ_2 in the evaluation suite.

2.3.5 Decoupled Direction and Norm (DDN)

Decoupled Direction and Norm (DDN) is motivated by the observation that the direction of an adversarial perturbation and the size of that perturbation can be adapted separately [13]. It is a whitebox ℓ_2 attack: the adversary uses gradients to update direction while controlling the perturbation norm. At a high level, DDN updates a normalized direction using the loss gradient and then adjusts the current radius depending on attack success:

$$\delta_{t+1} = \rho_{t+1} \frac{\delta_t + \alpha \,\text{Normalize}\left(\nabla_x \ell(f_\theta(x+\delta_t), y)\right)}{\left\|\delta_t + \alpha \,\text{Normalize}\left(\nabla_x \ell(f_\theta(x+\delta_t), y)\right)\right\|_2}.$$

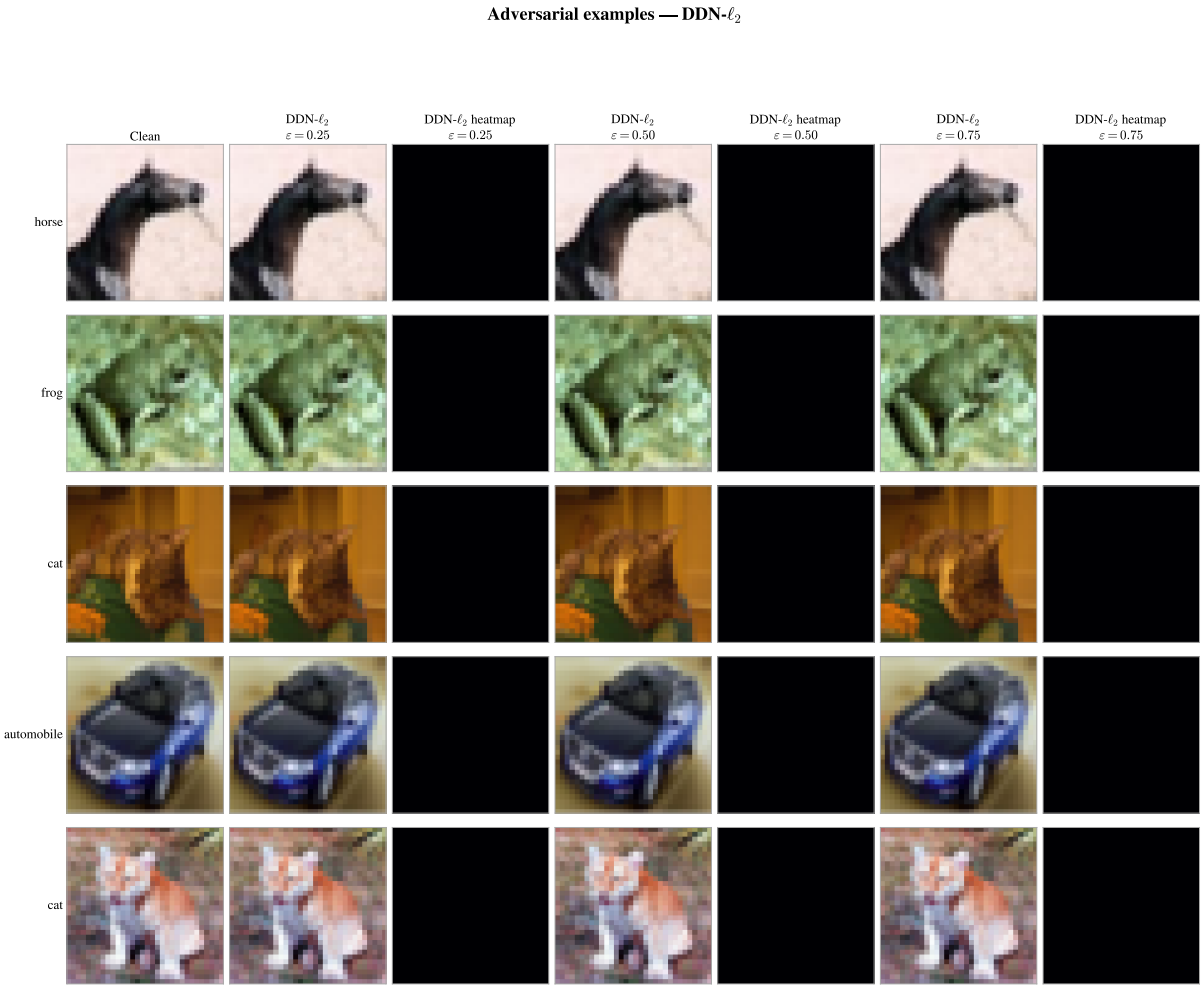


Figure 2.6: DDN-ℓ_2 adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

If the current perturbation is already adversarial, the radius can be reduced; otherwise, it can be increased. This decoupling lets the attack search for small successful ℓ_2 perturbations without fixing the radius throughout the trajectory. DDN is efficient and well suited to ℓ_2 adversarial training, but its behavior still depends on update and radius-adjustment choices.

2.3.6 Momentum Iterative Fast Gradient Sign Method (MI-FGSM)

Momentum Iterative Fast Gradient Sign Method (MI-FGSM) is motivated by the instability that can occur when iterative sign-gradient attacks change direction sharply across steps. It is a whitebox ℓ_∞ attack that augments iterative FGSM with a momentum term [14]. The objective remains untargeted loss maximization under an ℓ_∞ constraint.

The central update accumulates a normalized gradient direction before applying a sign step:

$$g_{t+1} = \mu g_t + \frac{\nabla_x \ell(f_\theta(x_t^{\text{adv}}), y)}{\left\|\nabla_x \ell(f_\theta(x_t^{\text{adv}}), y)\right\|_1}, \qquad x_{t+1}^{\text{adv}} = \Pi_{\mathcal{S}_\infty(\varepsilon)}\left(x_t^{\text{adv}} + \alpha \,\text{sign}(g_{t+1})\right).$$

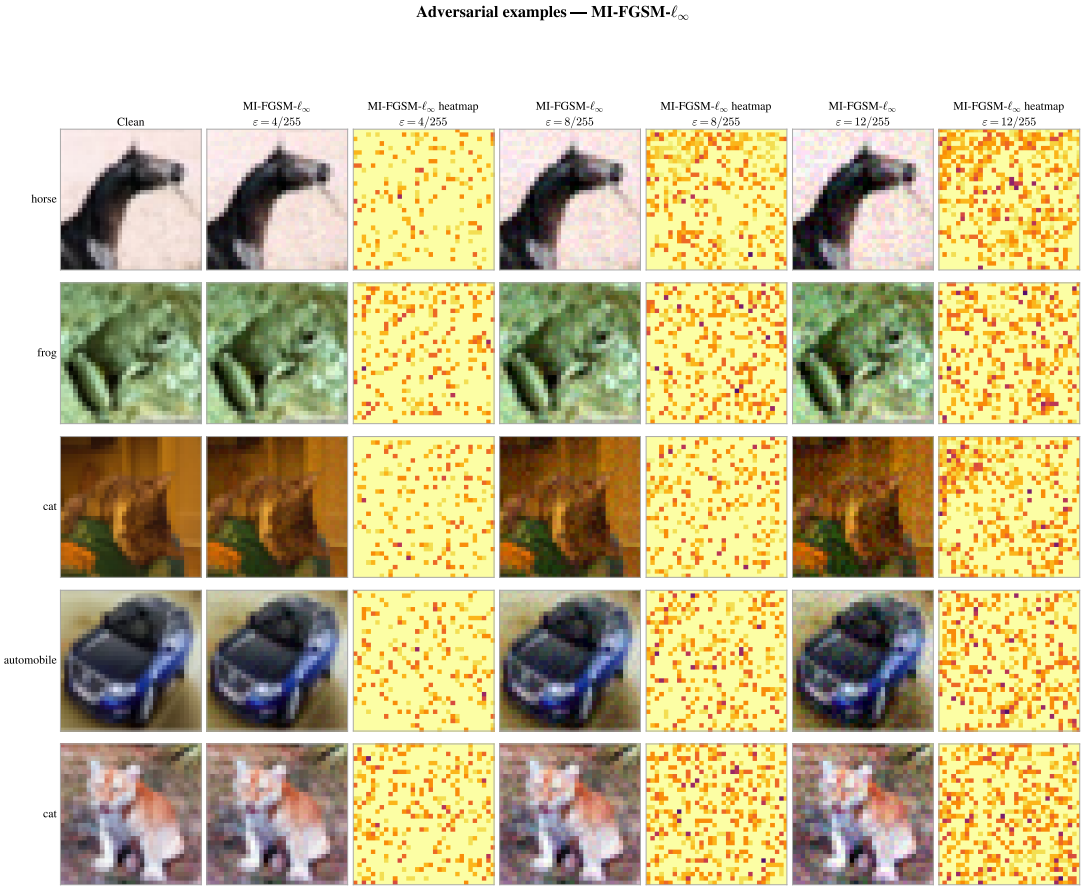


Figure 2.7: MI-FGSM-ℓ_∞ adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

The momentum term smooths the update direction and can improve transferability across models.

MI-FGSM is useful because it is close to PGD while still changing the attack dynamics. It can reveal whether robustness depends too narrowly on a particular first-order update rule. In this thesis it is a held-out ℓ_∞ evaluation attack and is not used for source adversarial training.

2.3.7 TRADES-style Projected Gradient Descent (TPGD)

The TPGD name is overloaded across software libraries and papers, so its meaning must be fixed for this report. Here, TPGD refers to the TRADES-style PGD attack used by the implemented evaluation code, not to target-label PGD. It is a whitebox ℓ_∞ evaluation attack whose objective is based on the divergence between clean and perturbed predictions [15].

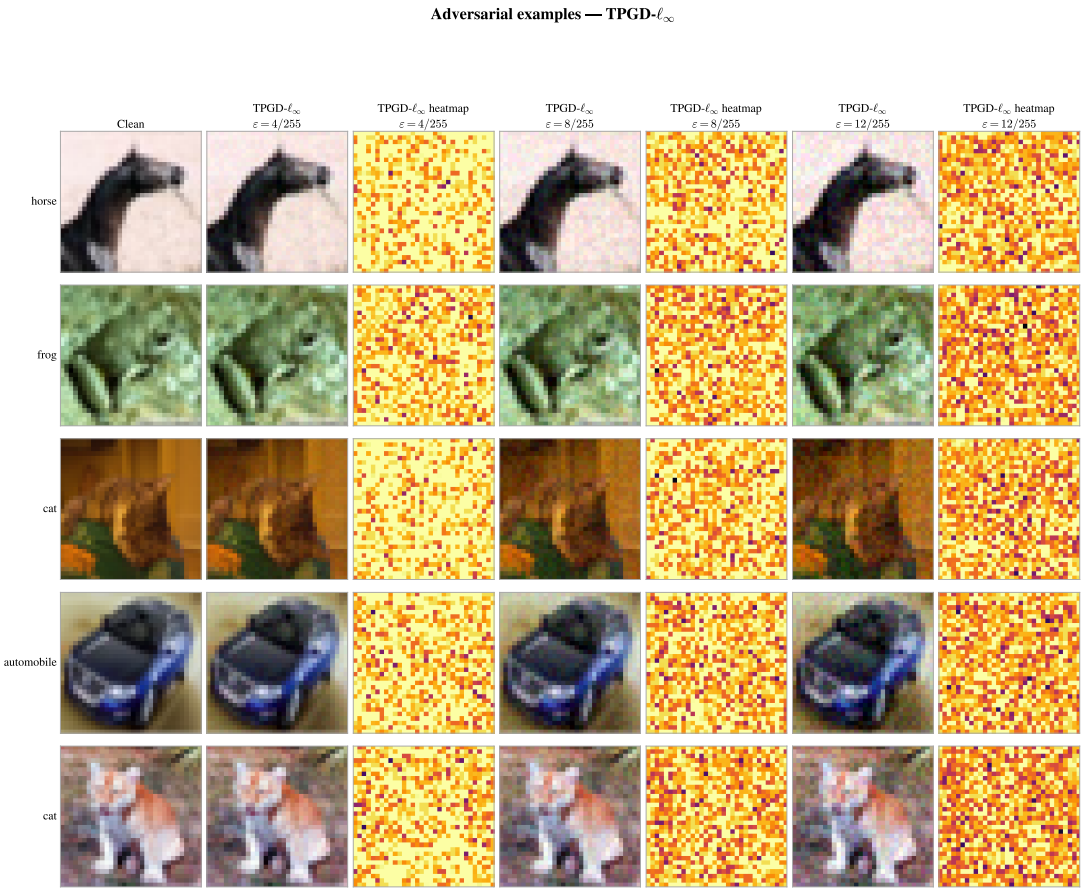


Figure 2.8: TPGD-ℓ_∞ adversarial examples on CIFAR-10 under three perturbation budgets, with normalized perturbation heatmaps.

The attack maximizes a KL-divergence objective rather than ordinary cross-entropy:

$$\max_{\delta \in \mathcal{S}_\infty(\varepsilon)} \mathrm{KL}\left(p_\theta(\cdot \mid x) \,\|\, p_\theta(\cdot \mid x+\delta)\right).$$

Projected gradient ascent then proceeds as in PGD, but with the KL objective replacing the classification loss. The intuition is to find perturbations that make the predictive distribution move away from the clean prediction, even when the ground-truth label is not directly used in the inner objective.

TPGD is useful because it tests a loss objective that differs from cross-entropy PGD. Its limitation is that it is still an ℓ_∞ first-order attack and therefore does not replace broader evaluation across norms and attack families. In this thesis it is a held-out evaluation attack included in Mean(8), not a training source attack.

2.3.8 AutoAttack

AutoAttack is motivated by the need for a standardized robustness evaluation that reduces manual attack tuning and combines complementary failure modes [5]. It is used here as a whitebox ℓ_∞ stress evaluation. Unlike the individual attacks above, AutoAttack is an evaluation suite rather than a single update rule.

At the wrapper level, AutoAttack evaluates a model using a fixed collection of attacks and reports robustness under the strongest successful component:

$$\mathrm{Acc}_{\mathrm{AA}}(f_\theta) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\forall A \in \mathcal{A}_{\mathrm{AA}},\ f_\theta(A(x_i, y_i, f_\theta)) = y_i\right].$$

This expression captures the ensemble logic without deriving the internals of APGD, FAB, or Square Attack.

AutoAttack is stronger and more standardized than a single configured attack, but it is computationally more expensive. This report therefore evaluates AutoAttack-ℓ_∞ separately from Mean(8). AutoAttack-512 is used only as a sanity or stress check, while the primary ranking metric remains the eight-attack evaluation suite defined in Chapter 5.

To keep the attack descriptions focused on the optimization objectives, the grouped qualitative comparison of the full attack suite is provided in Appendix C.1. The individual attack sections include representative three-budget visualizations, while the appendix places the ℓ_∞ and ℓ_2 attacks side by side.

The attack suite separates source attacks, held-out evaluation attacks, and the separate

AutoAttack stress test; Table 2.2 records those roles without repeating the exact budgets and step counts deferred to Chapter 5.

Table 2.2: Attack algorithms used in the report. Exact budgets and step counts are specified in Chapter 5.

Attack	Norm	Iterative	Training source	Evaluation	Role
FGSM-RS	ℓ_∞	No	No	Yes	Fast random-start one-step check for local gradient sensitivity.
PGD-ℓ_∞	ℓ_∞	Yes	Yes	Yes	Main ℓ_∞ source attack and standard whitebox evaluation attack.
TPGD	ℓ_∞	Yes	No	Yes	TRADES-style/KL-PGD held-out attack with a different loss geometry.
MI-FGSM	ℓ_∞	Yes	No	Yes	Momentum-based held-out attack, also useful for transfer behavior.
PGD-ℓ_2	ℓ_2	Yes	No	Yes	Held-out projected-gradient attack under the ℓ_2 threat model.
DDN-ℓ_2	ℓ_2	Yes	Yes	Yes	Main ℓ_2 source attack and evaluation attack.
DeepFool-ℓ_2	ℓ_2	Yes	No	Yes	Boundary-oriented held-out ℓ_2 attack.
CW-ℓ_2	ℓ_2	Yes	No	Yes	Margin-based optimization attack for held-out ℓ_2 evaluation.
AutoAttack-ℓ_∞	ℓ_∞	Yes	No	Separate	Standardized stress test reported outside Mean(8).

2.4 Adversarial Training as Min-Max Optimization

Adversarial training replaces ERM with a min-max objective in which an inner adversary generates difficult perturbed examples and an outer optimizer updates the model to classify them correctly [3]. A standard formulation is

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \max_{\delta_i \in \mathcal{S}_p(\varepsilon)} \ell(f_\theta(x_i + \delta_i), y_i).$$

The inner maximization approximates the worst local perturbation for the current model, while the outer minimization trains the model against those perturbations.

Single-attack adversarial training. Single-attack adversarial training chooses one source attack for the inner maximization. PGD-AT uses PGD-generated adversarial examples, while DDN-AT uses DDN-generated adversarial examples. These baselines are important because they isolate the effect of a strong source threat model, but they can also specialize the learned robustness to the attack family, norm, loss, and optimization path used during training.

Multi-attack adversarial training. Multi-AT extends the source set by training on a uniform mixture of source attacks. In this report, Multi-AT uses PGD-ℓ_∞ and DDN-ℓ_2 as source attacks, matching the source attacks used by AttackDRO, Cluster-DRO, and AttackDRO++. A compact objective is

$$\min_\theta \frac{1}{n}\sum_{i=1}^{n}\frac{1}{|\mathcal{E}|}\sum_{e\in\mathcal{E}}\ell(f_\theta(\mathcal{A}_e(x_i,y_i,f_\theta)),y_i),$$

where $\mathcal{E}$ is the source-attack set. This objective broadens attack coverage relative to a single-source baseline, but uniform averaging does not decide whether weak attack conditions, hard classes, or latent subgroups should receive additional emphasis.

Trade-off intuition. Adversarial training has become one of the most widely used empirical defenses, but it introduces practical trade-offs. Training against stronger inner attacks increases computational cost, and improving adversarial accuracy can reduce clean accuracy or shift errors across groups [15–17]. Multi-attack training adds another trade-off: expanding the source attack set can improve coverage, but it may also dilute training pressure if all sources are weighted uniformly despite unequal difficulty.

The later methodology builds on this background by asking how source attacks and discovered groups should be weighted during training. Chapter 3 reviews prior work that motivates this question, and Chapter 4 defines the specific group-aware methods evaluated in this report.

2.5 Distributionally Robust Optimization

Distributionally robust optimization (DRO) replaces average-case optimization with objectives that are sensitive to unfavorable distributions or groups. In its classical form, DRO minimizes the worst expected loss over an ambiguity set $\mathcal{U}$ around a reference distribution:

$$\min_\theta \sup_{Q\in\mathcal{U}} \mathbb{E}_{(x,y)\sim Q}\left[\ell(f_\theta(x),y)\right].$$

The ambiguity set can be defined in many ways, but the shared principle is that the model should not be optimized only for the empirical average when plausible shifts may expose higher-risk subpopulations [18, 19].

This progression from average-risk minimization to group-aware robust optimization provides the conceptual path from PGD-AT and DDN-AT to AttackDRO and Cluster-DRO; Table 2.3 summarizes the roles of these objective families in the report.

Table 2.3: Comparison of objective families used to motivate the training methods in this report.

Framework	Objective idea	Group assumption	Role in this report
ERM	Minimize average loss on clean training examples.	No explicit adversarial groups.	Reference learning objective and the clean-data baseline concept.
Adversarial training	Minimize loss on adversarial examples generated under a threat model.	Usually tied to one source attack unless extended.	Foundation for PGD-AT, DDN-AT, and Multi-AT.
DRO	Optimize for performance under a family of possible distributions.	Groups may be implicit through an uncertainty set.	Conceptual bridge from average-risk training to robustness under distribution shift.
Group DRO	Emphasize groups with higher loss through group-specific risks and weights.	Groups are predefined or discovered.	Foundation for AttackDRO over attack identities and Cluster-DRO over latent groups.

2.5.1 From ERM to Worst-Group Risk

Group DRO specializes the robust objective to a finite collection of groups. Suppose each example belongs to a group $g \in \mathcal{G}$, and let $R_g(\theta)$ denote the expected loss on group g. The worst-group objective is

$$\min_{\theta} \max_{g \in \mathcal{G}} R_g(\theta).$$

Equivalently, the maximum can be written as an optimization over the probability simplex:

$$\min_{\theta} \max_{q \in \Delta^{|\mathcal{G}|}} \sum_{g \in \mathcal{G}} q_g R_g(\theta),$$

where q_g is the weight assigned to group g. This form makes clear how Group DRO differs from ERM: ERM weights examples according to their empirical frequency, while Group DRO can emphasize a small group if it has high loss [4].

2.5.2 Group DRO Formulation

In practice, Group DRO alternates between estimating group losses and updating group weights. A common exponentiated-gradient update has the form

$$q_g^{(t+1)} = \frac{q_g^{(t)} \exp(\eta_q \hat{R}_g^{(t)})}{\sum_{h \in \mathcal{G}} q_h^{(t)} \exp(\eta_q \hat{R}_h^{(t)})},$$

where $\hat{R}_g^{(t)}$ is the current estimated loss for group g and η_q is a group-weight learning rate [4, 20]. Groups with higher loss receive larger future weights, which increases their influence on the parameter update.

Regularization in overparameterized models. In overparameterized neural networks, simply optimizing the worst empirical group loss can overfit small or noisy groups. Prior Group DRO work therefore emphasizes the role of regularization, validation-based model selection, and careful group construction when training high-capacity models [4]. This point is especially relevant for adversarial training because the inner attack already increases optimization difficulty and computational cost.

DRO in adversarial training. The group variable in adversarial training can be defined in multiple ways. AttackDRO uses fixed source attack identities as groups, so the group weights adapt over PGD-ℓ_∞ and DDN-ℓ_2 source conditions. Cluster-DRO instead uses discovered latent clusters as groups. Both choices inherit the same central constraint from Group DRO: the objective can only emphasize groups that have been meaningfully defined. If groups are too coarse, the optimizer may miss hard subpopulations inside them; if groups are too noisy, weight updates can chase unstable loss estimates.

Later chapters introduce q-flooring and uniform anchoring as method-specific stabilization ideas rather than as standard Group DRO theory. The key background point is simpler: Group DRO supplies an adaptive weighting mechanism, but its usefulness depends

on the granularity and reliability of the group definitions supplied to it.

2.6 Domain Generalization and Clustering

Domain generalization studies learning under multiple observed training environments with the goal of improving performance on unseen environments. In a standard formulation, each environment $e \in \mathcal{E}$ has its own data distribution P_e, and the learner must avoid relying only on features that are predictive in the observed environments but unstable under distribution shift [21, 22]. This report does not aim to solve general domain generalization. It uses the domain-generalization vocabulary more narrowly to describe how different adversarial attacks can induce distinct training and evaluation conditions.

2.6.1 Domain Generalization: Learning Beyond Known Distributions

The attacks-as-domains view treats each attack generator $\mathcal{A}_e$ as producing a distribution of adversarial examples from the same clean data. PGD-ℓ_∞, DDN-ℓ_2, MI-FGSM, DeepFool-ℓ_2, and CW-ℓ_2 do not correspond to natural domains such as cameras or hospitals, but they do create algorithm-induced shifts in geometry, loss objective, and optimization trajectory. This view helps explain why robustness to one attack may not fully transfer to another [6, 7].

Adversarial robustness differs from standard domain generalization in an important way. Evaluation is threat-model-aware: each attack is constructed with a perturbation set, loss, and access assumption, and the examples are generated against the model rather than passively sampled from an external distribution. Consequently, domain-generalization concepts are useful as an organizing language, but adversarial evaluation still requires explicit attack definitions and robust evaluation protocols [5, 22].

2.6.2 K-Means Clustering for Group Discovery

Group DRO requires group labels, but many hard subpopulations are not annotated. Hidden-group methods address this problem by inferring groups from representations, losses, or environment structure, then using those inferred groups to guide robust optimization [23–25]. In the adversarial setting, hidden groups can arise from class-specific failure modes, attack-specific geometry, or interactions between the two.

K-means clustering is a simple group-discovery tool that partitions feature vectors into

K clusters by minimizing within-cluster squared distance:

$$\min_{\{c_i\},\{\mu_k\}} \sum_{i=1}^{n} \left\| z_i - \mu_{c_i} \right\|_2^2,$$

where z_i is the feature vector for sample i, c_i is its cluster assignment, and μ_k is the centroid of cluster k. In robust training pipelines, the feature vector may come from model representations, attack metadata, losses, gradients, or combinations of these signals. The resulting clusters can then be treated as candidate groups for Group DRO.

Representation-only clustering is useful but incomplete for the setting studied here. Two samples can be close in representation space while producing different optimization behavior under adversarial perturbation, and two attacks can have similar representation effects while generating different gradients or loss trajectories. Prior work on input gradients and representation behavior suggests that these optimization-level signals can contain information not captured by activations alone [26, 27].

This motivates the later use of augmented clustering and gradient fingerprints. Chapter 4 defines how Cluster-DRO and AttackDRO++ construct groups for the report's training pipeline. The background role of this section is only to establish why discovered groups are plausible and why representation-only discovery may need optimization-aware features in a multi-attack adversarial setting.

2.7 Statistical Tools for Multi-Seed Evaluation

Robustness comparisons are noisy because training depends on initialization, data order, attack sampling, and optimizer dynamics. This report therefore uses paired random-seed panels, with the largest aggregate whitebox comparisons reported over 20 random-seed runs and smaller panels used for class-wise, graybox, ablation, and supplementary checks. The purpose of the statistical protocol is to compare methods under matched randomness wherever possible, not to claim that any single seed panel eliminates all uncertainty.

2.7.1 Paired Tests and Bootstrap Confidence Intervals

For two methods A and B evaluated under the same seed s, let $m_{A,s}$ and $m_{B,s}$ denote a metric such as robust accuracy or Mean(8). The paired difference is

$$d_s = m_{A,s} - m_{B,s}.$$

Paired analysis focuses on the distribution of d_s rather than on two unrelated sets of scores. This is appropriate when each method is evaluated on the same seed panel because common random variation can be removed from the comparison.

The paired t-test evaluates whether the mean paired difference is distinguishable from zero under an approximate normality assumption for the differences [28]. If $\bar{d}$ is the sample mean of the paired differences, s_d is their sample standard deviation, and S is the number of seeds, the statistic is

$$t = \frac{\bar{d}}{s_d/\sqrt{S}}.$$

The Wilcoxon signed-rank test is a nonparametric alternative that ranks the magnitudes of nonzero paired differences and tests whether their signs are symmetrically distributed around zero [29]. It is useful when the paired differences are few or visibly non-Gaussian, although it also has limited power for small seed panels.

Confidence intervals describe uncertainty in an estimated effect size or mean difference. A paired confidence interval can be formed analytically from the paired t distribution, or by bootstrap resampling of the paired differences. Bootstrap intervals repeatedly resample the set of seed-level differences with replacement and recompute the statistic of interest. In this report, confidence intervals are interpreted as uncertainty summaries around the observed seed panel, not as guarantees about all possible training runs.

2.7.2 Effect Size and Multiple-Comparison Correction

A p-value measures how unusual the observed statistic would be under a specified null hypothesis, but it does not measure the practical size of the effect. Effect sizes therefore accompany significance tests. For paired comparisons, a common standardized effect size is

$$d_z = \frac{\bar{d}}{s_d},$$

which scales the mean paired difference by the standard deviation of the paired differences [30]. Reporting both the raw paired difference and a standardized effect size helps separate practical magnitude from sampling variability.

When many attacks and metrics are compared, isolated small p-values can occur by chance. Multiple-comparison correction adjusts the interpretation of significance tests to account for the number of related hypotheses being examined. The correction should be read together with the direction of the paired difference, confidence intervals, and effect sizes, because statistical significance alone can overstate a small or unstable improvement.

Repeated random-seed evaluation. Random seeds are treated as paired experimental units throughout the evaluation protocol. For each attack metric, the comparison is made seed by seed, then summarized across the seed panel. This design supports statements about consistency across repeated training runs while avoiding exact seed identifiers in the thesis text.

The practical interpretation is deliberately conservative. A method comparison is most persuasive when the paired differences have the same direction across most seeds, the confidence interval is reasonably separated from zero, the effect size is meaningful, and the result remains coherent after accounting for multiple comparisons [31]. No single statistic is treated as sufficient on its own. This chapter established the technical background used by the rest of the report. It introduced the supervised classification notation, adversarial perturbation sets, threat-model assumptions, and attack algorithms needed to interpret the training and evaluation protocols. It then framed adversarial training as min-max optimization, with PGD-AT, DDN-AT, and Multi-AT serving as baselines that differ in how source attacks are used.

The chapter also defined the DRO and Group DRO machinery that later supports Attack-DRO and Cluster-DRO, emphasizing that adaptive weighting depends on meaningful group definitions. The domain-generalization and clustering discussion explained why attacks can be treated as domain-like shifts without equating adversarial robustness with general domain generalization. Finally, the statistical section fixed the paired multi-seed vocabulary used for Chapter 6. Chapter 3 next reviews the prior work that motivates the remaining gap, and Chapter 4 defines AttackDRO++ as the final proposed method.

Chapter 3

Related Work

Chapter 1 framed this work around the cross-attack robustness gap: adversarial robustness can depend strongly on the attack used during training, and a single robustness score can hide uneven behavior across threat models. This chapter reviews the related work through that lens. The goal is not to list every adversarial training variant, but to identify which parts of the literature motivate the method developed in Chapter 4 and which assumptions remain insufficient for the practical multi-attack setting studied in this report.

The chapter proceeds in five steps. Section 3.1 reviews how adversarial training moved from single-threat training toward multi-attack and multi-perturbation objectives. Section 3.2 discusses Group DRO as a mechanism for emphasizing weak groups, while also highlighting its dependence on useful group definitions. Section 3.3 connects the attacks-as-domains view to domain generalization, but distinguishes standard domain generalization from threat-model-aware adversarial evaluation. Section 3.4 turns to hidden-group and cluster-discovery methods, which motivate latent grouping but also expose the limits of representation-only clusters. Section 3.5 synthesizes these threads into the gap addressed by AttackDRO++.

3.1 Multi-Attack Adversarial Training

Adversarial training has become one of the most widely used empirical defenses because it turns robustness into an optimization problem over adversarially constructed examples. The early line from fast gradient methods to PGD-style min-max training made this dependence explicit: a defense is trained and evaluated relative to a specified perturbation set, attack objective, and optimization procedure [2, 3]. Recent surveys emphasize that adversarial-training outcomes depend not only on the attack used to generate adversar-

ial examples, but also on data construction, model design, and training-configuration choices [17]. This broader view matters for a multi-attack setting because the training attack is only one part of a larger robustness pipeline.

The threat-model dependence of adversarial training also created a parallel literature on robust evaluation. Defenses that appear effective under weak or misconfigured attacks can fail under stronger optimization, and gradient masking can make measured robustness misleading [8, 32]. AutoAttack and related benchmarking work respond to this problem by emphasizing standardized, diverse, and difficult attack suites rather than relying on a single attack implementation [5, 33]. For this report, the lesson is that a training method should be judged under attacks that differ from the source attacks used during optimization, because apparent robustness under one attack family does not necessarily transfer to another.

Multi-attack adversarial training is a natural response to this limitation. Instead of optimizing against one perturbation family, it exposes the model to several attack sources or to a union of perturbation models during training. Prior work on multiple perturbations shows that robustness against one ℓ_p geometry or spatial transformation can trade off against robustness to another, while work on the union of perturbation models directly studies how to train against heterogeneous threat sets [6, 7]. Later methods and benchmarks further show that different perturbation types can form distinct distributions, so multi-attack robustness is not simply a matter of adding more samples to a standard adversarial training loop [33, 34].

This literature establishes the role of Multi-AT in the present report. Multi-AT is the uniform multi-attack adversarial training baseline: it trains on a balanced mixture of PGD-ℓ_∞ and DDN-ℓ_2 source attacks while using the same architecture and training budget as the group-aware variants. It is stronger and more relevant than a single-attack baseline when the evaluation suite spans multiple attack families. However, its weighting rule is still fixed. Each source attack receives uniform treatment regardless of whether the current model is failing more severely on one attack, one class, or one latent subgroup of adversarial examples.

Broader attack coverage therefore does not by itself decide how training emphasis should be allocated. A method can train on multiple attacks and still average away the examples that define the weakest robust behavior. The next section turns to Group DRO because it offers a principled way to focus on weak groups, but it also introduces a new question: what should count as a group in a multi-attack adversarial training pipeline?

3.2 Group DRO Applied to Robustness

Group distributionally robust optimization addresses a different weakness from multi-attack coverage. Its starting point is that average-risk minimization can produce a model that performs well on the majority distribution while leaving minority or atypical groups with high loss. Hashimoto et al. develop this view in repeated loss minimization without demographic labels, and Sagawa et al. show that neural-network Group DRO can strengthen worst-group behavior when the robust objective is paired with appropriate regularization and model selection [4, 18]. The central idea is not to optimize only the mean loss, but to adapt training pressure toward the groups that currently determine the weakest performance.

This objective is attractive for adversarial robustness because attack-induced examples are rarely uniform in difficulty. Some attacks may remain harder than others at a given point in training, and some classes or subregions of the data distribution may be consistently more vulnerable. If those failure modes are known in advance, Group DRO provides a direct mechanism for increasing their training weight. In the terminology of this report, AttackDRO is exactly this coarse fixed-group baseline: it applies Group DRO over the fixed source attack identities used during training.

The same literature also makes clear why this construction is incomplete. Group DRO depends on group labels that meaningfully align with the failure structure. When group annotations are unavailable or too coarse, worst-group optimization can emphasize the wrong partition. Hidden-group methods respond to this problem by estimating groups from model representations, losses, or inferred environments. GEORGE clusters representation space to find latent subclasses, Just Train Twice uses high-loss examples from a first-stage model to locate minority groups, and environment inference methods infer partitions for invariant learning when environments are not provided [23–25].

The implication for robustness is that fixed source attack identities are useful but coarse. Two adversarial examples generated by different attacks may share the same failure mode, while two examples generated by the same attack may differ sharply in margin, class, representation, or gradient behavior. AttackDRO can therefore test whether attack-level reweighting helps, but it cannot determine whether the hardest robust groups actually coincide with the attack labels. This motivates moving from fixed attack identities to group discovery, while retaining the Group DRO principle of adaptive weak-group emphasis.

3.3 Domain Generalization and Adversarial Robustness

Domain generalization provides the conceptual language for treating attacks as domain-like shifts. In standard domain generalization, a learner observes several source environments and is evaluated on environments that may differ from the training distribution. Methods such as invariant risk minimization and risk extrapolation formalize this goal by encouraging representations or risks that remain stable across source environments [21, 35]. DomainBed adds an important caution: domain generalization claims are sensitive to implementation details, model selection, and baseline strength, and carefully tuned ERM can be hard to beat [22]. That caution is directly relevant to this report, where Multi-AT must remain a strong baseline rather than a weak point of comparison.

The attacks-as-domains analogy is useful because each attack induces a different training condition. PGD-ℓ_∞, DDN-ℓ_2, and held-out attacks differ in perturbation geometry, optimization path, and loss landscape. From this perspective, cross-attack robustness resembles performance under structured distribution shift: the model is trained under some attack-induced domains and evaluated under both seen and unseen attack conditions. Prior work linking adversarial robustness and generalization supports this separation between standard accuracy, adversarial robustness, and robustness under distribution shift [36–38].

At the same time, adversarial robustness is not identical to standard domain generalization. A held-out domain in ordinary DG is usually a naturally occurring distribution shift, while an adversarial example is generated by an attacker under an explicit threat model. Evaluation is therefore threat-model-aware: the norm, budget, attack steps, loss objective, and attacker access all shape the measured robust accuracy. Reliable adversarial evaluation must also guard against weak attacks and misleading optimization artifacts, which is why strong evaluation suites remain central [5, 32, 33].

Thus, domain generalization motivates the report's framing but does not supply the full method. Treating attacks as domains explains why averaging over source attacks can obscure heterogeneous failure modes, but the grouping mechanism still needs to be attack-aware and tied to adversarial training dynamics. The next section therefore turns to cluster discovery, which offers a way to infer latent groups inside the training pipeline rather than assuming that fixed attack identities are the only meaningful domains.

3.4 Cluster Discovery in Robust Training Pipelines

Cluster discovery becomes relevant when visible group labels are missing or too coarse to describe the actual failure modes of a model. Hidden stratification work shows that coarse class labels can contain meaningful subgroups with very different error rates, and that learned representations can help expose those subgroups. GEORGE is a representative example: it clusters representation space to estimate latent subclasses and then applies a robust objective over the discovered groups [23]. Environment inference methods make a related point from the domain-generalization side by inferring environments that can be used by invariant objectives when the environment labels are not known [24].

These methods are important for multi-attack robustness because they weaken the assumption that useful groups must be supplied by the dataset designer. In an adversarial training pipeline, the obvious supplied labels are class labels and source attack identities, but neither is assured to match the robust failure structure. A representation-based cluster may identify subgroups that cut across attacks, classes, or both. This motivates Cluster-DRO as an intermediate baseline in this report: Group DRO over discovered clusters rather than over fixed source attack identities.

Representation geometry, however, is only one signal. Other work shows that training dynamics and gradients can reveal example difficulty that is not captured by semantic similarity alone. Just Train Twice uses high-loss examples from an initial model to identify groups that average-risk training overlooks [25]. Data Diet and related gradient-based scoring work show that early-training gradient or error signals can identify examples that are disproportionately influential for learning [27]. Charpiat et al. similarly define input similarity through the effect of parameter updates, which makes sample relations explicitly optimization-sensitive [26].

For adversarial robustness, this distinction matters. A cluster built only from penultimate representations may capture semantic or visual similarity while missing the optimization-level difficulty created by adversarial generation. Two adversarial examples can be close in representation space but differ in margin, gradient direction, or attack sensitivity; conversely, examples that look semantically different may create similar training pressure. The cluster literature therefore motivates latent group discovery, but it also motivates augmenting representation-based clusters with optimization-aware features. Chapter 4 uses this opening to introduce gradient fingerprints and augmented clustering features for AttackDRO++.

3.5 The Remaining Gap and Motivation for This Work

The related work leaves a layered gap rather than a single missing citation. Single-attack adversarial training gives a clear min-max formulation, but it can specialize to the training threat model. Multi-attack adversarial training broadens attack coverage, but common formulations still rely on fixed source attacks, fixed perturbation definitions, or uniform mixtures [3, 6, 7]. Group DRO adds adaptive weak-group emphasis, but its behavior depends on whether the chosen groups reflect the relevant failures [4]. Domain generalization and hidden-group discovery suggest that useful groups may be latent, but adversarial robustness still requires threat-model-aware evaluation and grouping signals that reflect optimization difficulty [22–24].

This combination motivates the method progression used in the rest of the report. Multi-AT serves as the uniform multi-attack adversarial training baseline. AttackDRO applies Group DRO over fixed source attack identities, testing whether adaptive attack-level weighting changes robustness relative to uniform mixing. Cluster-DRO then moves the grouping unit from source attack identities to discovered clusters, addressing the possibility that robust failure modes cut across attack labels. AttackDRO++ denotes only the final proposed method: uniform-anchored Cluster-DRO with gradient fingerprints, where the anchor keeps the objective tied to the Multi-AT signal and the gradient fingerprints make the clusters sensitive to optimization-level difficulty.

The prior strands motivate this combination but do not by themselves provide it. Multi-attack training explains why several attacks should be present during optimization; Group DRO explains why the weakest groups should receive additional emphasis; domain generalization explains why attacks can be viewed as domain-like shifts; and cluster discovery explains why the grouping variable need not be supplied in advance. What remains is a practical multi-attack adversarial training method that combines these ideas without treating fixed attack identities as the final grouping granularity.

The novelty claim is therefore deliberately conservative. This report addresses a practical multi-attack adversarial training setting, not robustness against arbitrary attacks or a general solution to domain generalization. This conservative framing is consistent with recent survey evidence that adversarial training remains shaped by practical challenges such as catastrophic overfitting, fairness, robustness-accuracy trade-offs, and computational cost [17]. Chapter 4 introduces the AttackDRO++ method that addresses the gap identified here, and Chapter 5 defines the protocol used to evaluate it without previewing the numerical results.

Chapter 4

Proposed Methodology

The diagnostic motivation is that adversarial robustness does not transfer uniformly across attack families. A model trained against one threat model can remain vulnerable to attacks with different perturbation geometries, optimization procedures, or loss objectives. This motivates the proposed method: instead of treating all adversarial examples as samples from one homogeneous distribution, this chapter models attack-induced examples as domains and asks how training can emphasize weak domains or latent hard groups without losing the stability of the uniform multi-attack baseline.

The chapter develops this progression step by step. Section 4.1 first formulates multi-attack adversarial training under the attacks-as-domains perspective. Section 4.2 then defines Multi-AT, the uniform multi-attack baseline used alongside the single-attack PGD-AT and DDN-AT baselines. Section 4.3 introduces AttackDRO, which applies group distributionally robust optimization (Group DRO) over source attack identities, but this still assumes that each attack forms one homogeneous group.

The remaining sections address that limitation. Section 4.4 introduces a cluster-level DRO component that replaces fixed attack identities with discovered latent clusters. Section 4.5 augments those clusters with gradient fingerprints so that grouping can reflect optimization-level difficulty, not only representation similarity. Section 4.6 adds a uniform anchor that stabilizes the adaptive cluster-level correction. In this report, AttackDRO++ denotes the final complete method: latent cluster discovery, augmented clustering features, gradient fingerprints, and a uniform-anchored Cluster-DRO objective.

4.1 Multi-Attack Training as a Domain Problem

4.1.1 Setup and Notation

Let

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n} \tag{4.1}$$

denote the supervised training dataset, where $\mathbf{x}_i \in \mathcal{X}$ is an input image and $y_i \in \{1, \dots, C\}$ is its class label. Let f_θ be a classifier parameterized by θ, and let $\ell(f_\theta(\mathbf{x}), y)$ denote the cross-entropy loss.

We denote the logits produced by the model as

$$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^C,$$

and the penultimate-layer representation as

$$\mathbf{h} = \phi_\theta(\mathbf{x}) \in \mathbb{R}^{d_h}.$$

The representation $\phi_\theta(\cdot)$ is later used for cluster discovery.

We define a set of source attack domains

$$\mathcal{A}_{\text{src}} = \{A_1, A_2, \dots, A_M\}, \tag{4.2}$$

where each attack operator A_m is characterized by a threat model, perturbation budget, optimization procedure, number of inner steps, and loss objective. Given a clean sample $(\mathbf{x}_i, y_i)$, the adversarial example produced by source attack A_m is

$$\mathbf{x}_i^{(m)} = A_m(f_\theta, \mathbf{x}_i, y_i). \tag{4.3}$$

In the main experiments, we use two source attacks: a PGD-ℓ_∞ attack and a DDN-ℓ_2 attack. This choice exposes the model to two common adversarial norm families during training. Additional attacks are reserved as heldout evaluation attacks and are never used for optimization.

4.1.2 Multi-Attack Risk and Its Limitations

The empirical adversarial risk under source attack A_m is

$$R_m(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\Big(f_\theta\Big(\mathbf{x}_i^{(m)}\Big), y_i\Big). \tag{4.4}$$

A standard multi-attack objective minimizes the average risk across source attacks:

$$\min_\theta \frac{1}{M}\sum_{m=1}^{M} R_m(\theta). \tag{4.5}$$

This objective improves attack diversity compared with single-attack adversarial training. However, it treats all source attacks and all adversarial examples with equal importance. This creates two limitations.

First, if one source attack consistently induces higher loss than another, equal weighting may underemphasize the harder attack. Second, adversarial examples within the same attack are not homogeneous. Two samples generated by the same algorithm can differ substantially in classification margin, representation, and optimization direction. These within-attack differences are not captured by Equation (4.5).

The central goal of this chapter is therefore to construct adaptive objectives that can identify and upweight hard groups while preserving the stability of the uniform multi-attack training signal.

4.2 Uniform Multi-Attack ERM Baseline

Uniform Multi-Attack empirical risk minimization (ERM) serves as the main baseline for this work. It trains on multiple source attacks within the same minibatch but assigns equal weight to all generated adversarial examples. This makes it a strong and simple reference point for evaluating whether adaptive reweighting provides additional benefit.

Given a minibatch

$$\mathcal{B} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N},$$

we partition it into M disjoint subsets

$$\mathcal{B}_1, \dots, \mathcal{B}_M, \qquad \mathcal{B}_m \cap \mathcal{B}_{m'} = \emptyset \ \text{ for } m \neq m', \qquad \bigcup_{m=1}^{M} \mathcal{B}_m = \mathcal{B}.$$

Each subset $\mathcal{B}_m$ is assigned to one source attack A_m. For each $(\mathbf{x}_i, y_i) \in \mathcal{B}_m$, we generate

$$\mathbf{x}_i^{\text{adv}} = A_m(f_\theta, \mathbf{x}_i, y_i).$$

The adversarial examples from all source attacks are then concatenated into one mixed adversarial batch. The uniform multi-attack loss is

$$\mathcal{L}_{\text{ERM}} = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i)\,. \tag{4.6}$$

In our main setting, the source attacks are PGD-ℓ_∞ and DDN-ℓ_2, so the model is exposed to both ℓ_∞ and ℓ_2 perturbation geometries during training. However, the objective in Equation (4.6) treats all adversarial examples equally. It does not explicitly identify which attack, class, or latent subgroup is harder. This limitation motivates the adaptive reweighting methods introduced in the following sections.

4.3 AttackDRO: Group DRO Over Attack Identities

AttackDRO extends Uniform Multi-Attack ERM by applying Group DRO over attack identities. Each source attack A_m is treated as one group. The method maintains an attack-group weight vector

$$\mathbf{q} = (q_1, \ldots, q_M) \in \Delta^M, \tag{4.7}$$

where Δ^M is the M-dimensional probability simplex.

For each attack group m, the group loss is

$$\mathcal{L}_m = \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} \ell(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i)\,. \tag{4.8}$$

The AttackDRO objective is then

$$\mathcal{L}_{\text{AttackDRO}} = \sum_{m=1}^{M} q_m \mathcal{L}_m. \tag{4.9}$$

The group weights are updated using an exponentiated-gradient (EG) rule. Let s_m be the difficulty score of attack group m. The update is

$$q_m \leftarrow q_m \exp(\eta_q s_m), \qquad m = 1, \ldots, M, \tag{4.10}$$

followed by normalization and floor projection:

$$q_m \geq q_{\min}, \qquad \sum_{m=1}^{M} q_m = 1. \tag{4.11}$$

The floor constraint prevents any attack group from being completely ignored.

In our formulation, the difficulty score is based on the classification margin. For a sample i with logits $\mathbf{z}_i$, define the margin as

$$r_i = z_{i,y_i} - \max_{c \neq y_i} z_{i,c}. \tag{4.12}$$

A large positive margin indicates a confident correct prediction, while a small or negative margin indicates a hard or misclassified sample. We convert this margin into a per-sample difficulty score using

$$s_i = \text{softplus}(-r_i/\tau) = \log(1 + \exp(-r_i/\tau)), \tag{4.13}$$

where $\tau > 0$ is a temperature parameter. The group-level difficulty score is the average difficulty within group m:

$$s_m = \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} s_i. \tag{4.14}$$

AttackDRO can adaptively emphasize whichever source attack is currently harder. However, it relies on a coarse grouping assumption: all examples generated by the same attack are treated as one homogeneous group. In practice, two PGD-ℓ_∞ examples may have very different margins, representations, and gradient directions. Conversely, examples generated by different attacks may share similar failure modes. Thus, the EG update can only adapt at the attack level and may miss finer-grained difficulty structure within and across attacks. The cluster-based extension in the next section addresses this limitation.

4.4 Cluster-Level DRO: Discovering Latent Groups

This section introduces the cluster-level DRO component used by AttackDRO++. Instead of assigning one group to each source attack identity, the component discovers latent groups from adversarial examples. The central hypothesis is that attack identity is too coarse as a grouping criterion. Two adversarial examples generated by different attacks may share the same failure mode, while two examples generated by the same attack may have very different margins, representations, and optimization directions. Cluster-based grouping allows the DRO objective to adapt to hard subgroups that may cut across

attack boundaries. The complete AttackDRO++ method is defined after the augmented clustering features and uniform anchor are introduced in Sections 4.5 and 4.6.

For each adversarial example $\mathbf{x}_i^{\text{adv}}$, the model extracts a representation vector

$$\mathbf{h}_i = \phi_\theta(\mathbf{x}_i^{\text{adv}}), \tag{4.15}$$

which is augmented with the scalar label feature and gradient fingerprint to form the clustering feature ψ_i defined in Section 4.5. A clustering model then assigns the example to a latent group

$$c_i \in \{1, \dots, K\}. \tag{4.16}$$

Let $\mathcal{B}_k$ denote the subset of the current minibatch whose adversarial examples are assigned to cluster k:

$$\mathcal{B}_k = \{i \in \{1, \dots, N\} : c_i = k\}. \tag{4.17}$$

The cluster loss is

$$\mathcal{L}_k = \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} \ell(f_\theta(\mathbf{x}_i^{\text{adv}}), y_i)\,. \tag{4.18}$$

The cluster-DRO component maintains a cluster-level weight vector

$$\mathbf{q} = (q_1, \dots, q_K) \in \Delta^K. \tag{4.19}$$

Because not every cluster appears in every minibatch, the cluster weights are renormalized over the clusters present in the current minibatch. Let

$$\mathcal{C}_{\mathcal{B}} = \{k \in \{1, \dots, K\} : |\mathcal{B}_k| > 0\} \tag{4.20}$$

denote the set of present clusters. For each $k \in \mathcal{C}_{\mathcal{B}}$, we define

$$\tilde{q}_k = \frac{q_k}{\sum_{j \in \mathcal{C}_{\mathcal{B}}} q_j}. \tag{4.21}$$

The cluster-DRO loss is then

$$\mathcal{L}_{\text{ClusterDRO}} = \sum_{k \in \mathcal{C}_{\mathcal{B}}} \tilde{q}_k \mathcal{L}_k. \tag{4.22}$$

This objective changes the unit of robustness balancing. Rather than assigning one weight to each attack identity, the model assigns weight to discovered subgroups of adversarial examples, so hard regions that cut across attack families can receive more train-

ing emphasis. The renormalization over present clusters keeps the minibatch objective well-defined even when some latent groups are absent from a particular batch.

The cluster weights are updated using the same exponentiated-gradient principle as in AttackDRO, but now at the cluster level rather than the attack level. Let s_k denote the configured cluster reweighting score for cluster k. In the implementation, s_k can be based on the cluster loss $\mathcal{L}_k$, the configured difficulty score averaged over the cluster,

$$\frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} s_i, \tag{4.23}$$

or a hybrid of loss and difficulty score, depending on the configured q-update metric. The cluster update is

$$q_k \leftarrow \Pi_{q_{\min}} \left(q_k \exp(\eta_q s_k) \right), \qquad k = 1, \dots, K. \tag{4.24}$$

Here $\Pi_{q_{\min}}$ denotes the normalization and floor projection applied to the full cluster-weight vector after the exponentiated-gradient step. Thus, the update emphasizes clusters with higher configured reweighting scores rather than assuming that raw cluster loss is always the only update signal. This allows the cluster-DRO component to emphasize hard latent groups even when they do not align exactly with the original attack identities.

Since model representations evolve during training, cluster assignments must also be updated. The clustering procedure therefore maintains a feature bank $\mathcal{F}$, which stores detached augmented feature vectors ψ_i from recent minibatches. When the refresh condition is met, K-means is refitted on $\mathcal{F}$, and subsequent minibatches are assigned using the updated clusterer. Before the first K-means refresh, cluster assignments are bootstrapped randomly.

The refresh condition is treated as a policy rather than a fixed part of the method. It can be defined in terms of epochs or training steps. In the main experiments, we use an epoch-based refresh schedule with $R = 2$ epochs. After each refresh, the feature bank is reset or updated according to the memory policy, and the cluster weights are refreshed according to the chosen q-refresh policy.

4.5 Augmented Clustering with Gradient Fingerprints

Representation-space clustering alone may not fully capture adversarial difficulty. Two adversarial examples can be close in feature space while inducing different optimization behavior. To capture this additional signal, AttackDRO++ constructs an augmented clus-

tering feature that combines representation similarity, class information, and gradient-level training dynamics.

For an adversarial example $\tilde{\mathbf{x}}_i = \mathbf{x}_i^{\text{adv}}$, let

$$z_i = f_\theta(\tilde{\mathbf{x}}_i), \qquad h_i = \phi_\theta(\tilde{\mathbf{x}}_i).$$

The logits $z_i = f_\theta(\tilde{\mathbf{x}}_i)$ are used to compute prediction probabilities and gradient-fingerprint terms, while $h_i = \phi_\theta(\tilde{\mathbf{x}}_i)$ provides the representation component of the clustering feature. Let

$$p_i = \text{softmax}(z_i)$$

be the predicted class distribution.

The first component is the normalized representation feature,

$$\bar{h}_i = \text{norm}(h_i),$$

which captures semantic similarity in the model feature space.

The second component is a class-aware scalar label feature. When label information is enabled, the implementation appends the normalized scalar class-index feature $\tilde{y}_i \in [0, 1]$ with weight λ_{lbl}:

$$\lambda_{\text{lbl}} \tilde{y}_i,$$

which provides weak label structure during clustering without using a one-hot label vector in the clustering feature.

The third component is the gradient fingerprint. For a linear classifier head

$$f_\theta(\cdot) = W\phi_\theta(\cdot) + b, \qquad W \in \mathbb{R}^{C \times d_h},$$

the per-example gradient of the cross-entropy loss with respect to the classifier weight matrix is

$$\nabla_W \ell(f_\theta(\tilde{\mathbf{x}}_i), y_i) = (p_i - \text{onehot}(y_i)) \otimes h_i \in \mathbb{R}^{C \times d_h}.$$

We use this quantity as a gradient proxy and denote it by

$$G_i = (p_i - \text{onehot}(y_i)) \otimes h_i.$$

The one-hot vector appears here only in the cross-entropy gradient proxy; it is not appended as the label component of ψ_i.

Intuitively, G_i describes the training direction induced by sample i. Thus, two adversarial examples may be grouped together not only because they are close in representation space, but also because they induce similar optimization updates.

The raw gradient proxy has dimension Cd_h. For CIFAR-10 with a ResNet-18 backbone, this corresponds to $10 \times 512 = 5120$ dimensions. To make the fingerprint more compact and efficient for clustering, we use a fixed random projection matrix

$$P \in \mathbb{R}^{Cd_h \times d_{\text{proj}}},$$

and define the projected gradient fingerprint as

$$g_i = \text{vec}(G_i)P \in \mathbb{R}^{d_{\text{proj}}}.$$

In our default setting, $d_{\text{proj}} = 128$. The projection is fixed throughout training and introduces no additional learnable parameters.

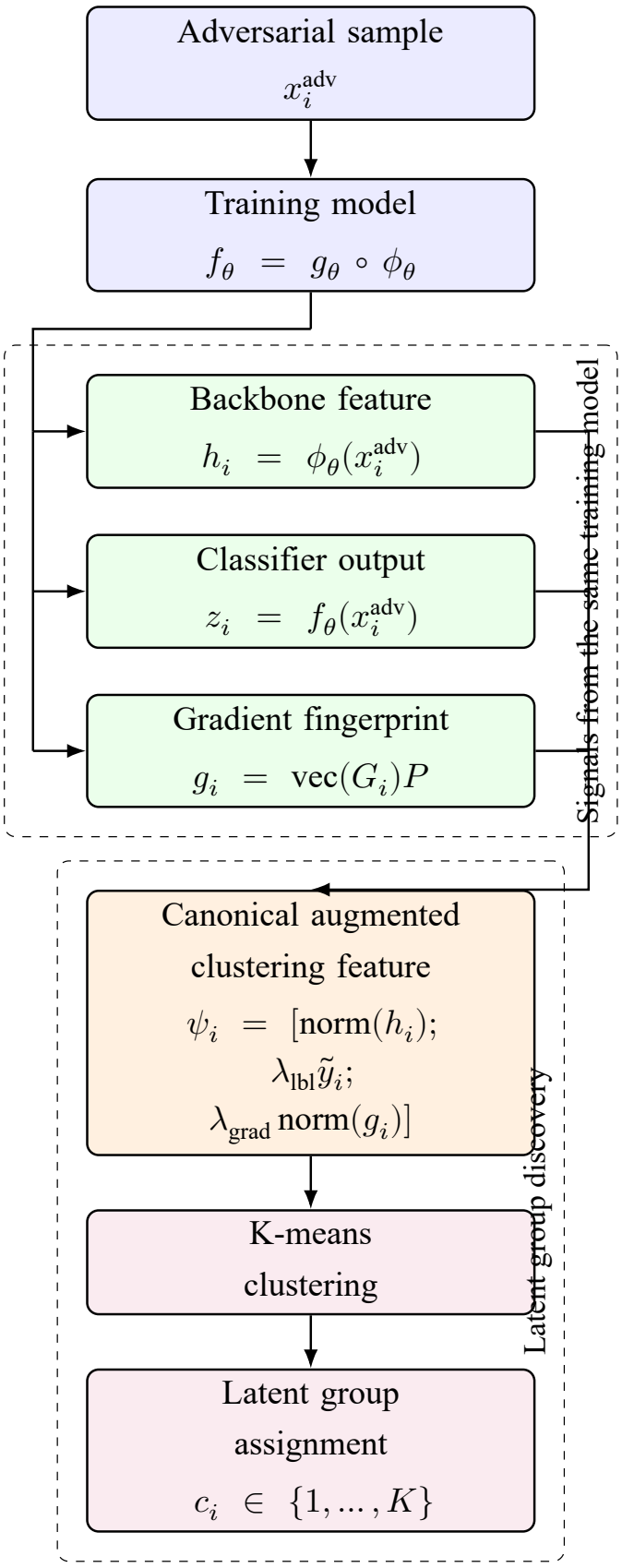


Figure 4.1: Augmented clustering feature construction in AttackDRO++. Each adversarial sample x_i^{adv} is passed through the same training model f_θ, producing both the backbone feature h_i and classifier output z_i. A gradient fingerprint g_i is computed from the classifier-head loss gradient and projected to a low-dimensional vector. The representation feature, scalar normalized label feature, and gradient fingerprint are combined as the canonical augmented clustering feature $\psi_i = [\text{norm}(h_i); \lambda_{\text{lbl}}\tilde{y}_i; \lambda_{\text{grad}}\,\text{norm}(g_i)]$, which is then used for K-means clustering and latent group assignment. Logits are used to compute probabilities and gradient fingerprints, but they are not concatenated directly into ψ_i.

The final augmented clustering feature is

$$\psi_i = \left[\text{norm}(h_i) \parallel \lambda_{\text{lbl}}\tilde{y}_i \parallel \lambda_{\text{grad}}\,\text{norm}(g_i)\right].$$

The three components play complementary roles. Representation features describe where an example lies in feature space, whereas gradient fingerprints describe how the classifier head would need to change to classify that example correctly. Combining representation, scalar label, and gradient information makes the clustering signal both representation-aware and optimization-aware. This allows AttackDRO++ to discover latent adversarial groups that reflect both structural similarity and optimization difficulty.

4.6 Uniform-Anchored Training Objective

A pure Cluster-DRO objective can become unstable when the cluster weights $\mathbf{q}$ concentrate too strongly on a small number of hard clusters. We refer to this failure mode as *q-collapse*. In the extreme case, the objective approaches a worst-cluster loss, where most of the training signal is dominated by one cluster. This behavior is undesirable in multi-attack adversarial training because the hardest cluster may change after each cluster refresh, and over-emphasizing one cluster can reduce robustness on the remaining attacks or groups.

To stabilize training, AttackDRO++ uses a uniform-anchored Cluster-DRO objective that interpolates between the uniform multi-attack ERM loss and the adaptive cluster-DRO loss:

$$\mathcal{L}_{\text{AttackDRO++}} = (1 - \lambda_{\text{DRO}})\mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\mathcal{L}_{\text{ClusterDRO}}. \tag{4.25}$$

Here, $\lambda_{\text{DRO}} \in [0, 1]$ controls the strength of the Cluster-DRO correction. When $\lambda_{\text{DRO}} = 0$, the method reduces to uniform multi-attack ERM. When $\lambda_{\text{DRO}} = 1$, the method becomes pure Cluster-DRO.

Equivalently, Equation (4.25) can be written as

$$\mathcal{L}_{\text{AttackDRO++}} = \mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\left(\mathcal{L}_{\text{ClusterDRO}} - \mathcal{L}_{\text{ERM}}\right). \tag{4.26}$$

This form shows that λ_{DRO} scales the difference between the cluster-reweighted loss and the uniform loss. If the cluster-DRO loss is close to the uniform loss, the correction is small. If some clusters are substantially harder, the correction becomes larger, but its influence remains controlled by λ_{DRO}.

In the main experiments, we set $\lambda_{\text{DRO}} = 0.35$ as the default anchor strength. This value keeps the uniform multi-attack objective as the dominant stabilizing term while still allowing the model to adapt toward harder clusters. Intuitively, this suggests an inverted-U behavior: a small amount of Cluster-DRO pressure can improve robustness by emphasizing hard groups, whereas excessive pressure may overfit unstable clusters and harm the balance across attacks.

A second stabilization mechanism is the floor projection applied to the cluster weights. After each exponentiated-gradient update, $\mathbf{q}$ is normalized and projected so that every cluster keeps at least q_{min} mass:

$$q_k \geq q_{\text{min}}, \qquad \sum_{k=1}^{K} q_k = 1. \tag{4.27}$$

The anchor controls how much the adaptive cluster loss can influence the total objective, while the floor projection controls the cluster weights themselves. Together, they prevent the adaptive component from collapsing onto a small number of clusters and keep all discovered groups represented during training. In our default setting, $q_{\text{min}} = 0.03$.

Finally, the q-update is delayed during the early training phase. For the first T_w epochs, $\mathbf{q}$ remains uniform while the model representation and initial cluster assignments become more reliable. After this warmup period, the cluster weights are updated using the exponentiated-gradient rule. In the main configuration, $T_w = 3$ epochs.

4.7 Complete Training Framework

AttackDRO++ is the final method studied in this report. It combines latent cluster discovery, augmented clustering features, gradient fingerprints, and a uniform-anchored Cluster-DRO objective into one training pipeline. The goal is to move beyond fixed attack identities and let the model identify harder adversarial subgroups during training. At the same time, the uniform anchor prevents the adaptive objective from drifting too far from the strong multi-attack baseline. Algorithm 1 gives the complete procedure, while Table 4.1 lists the default hyperparameters used in the main experiments.

From Diagnostic Motivation to the Final Method The method follows directly from the attacks-as-domains diagnosis introduced earlier. Single-attack adversarial training reveals the cross-attack robustness gap: a model trained against one source attack can remain weak against other threat models. Uniform multi-attack training addresses this

by exposing the model to multiple source attacks, but it treats all generated examples equally. AttackDRO then adds attack-level reweighting, but this still assumes that each attack family is internally homogeneous.

AttackDRO++ removes this assumption. Instead of using attack identity as the only group label, it discovers latent groups from the adversarial examples themselves. Gradient fingerprints add optimization-level information to the clustering features, helping the grouping capture not only representation similarity but also training difficulty. Finally, the uniform anchor keeps the objective close to the strong multi-attack baseline while still allowing Cluster-DRO to emphasize harder groups. The result is a method that moves from observing the cross-attack gap to actively training against the latent structure behind it.

The same evaluation discipline is preserved throughout the framework. Source attacks are fixed during training, held-out attacks are never used for optimization, and robustness is reported both per attack and through aggregate metrics.

The dominant cost remains adversarial example generation, which is shared by all multi-attack methods in the comparison. Therefore, AttackDRO++ mainly changes the training objective and grouping mechanism, while keeping the source attacks, threat models, and evaluation protocol fixed. The cluster refresh schedule is an implementation choice rather than a fixed part of the mathematical objective. In the main experiments, clusters are refreshed every $R = 2$ epochs, and $\mathbf{q}$ is reset to the uniform distribution after each refresh to avoid carrying weights across changed cluster identities.

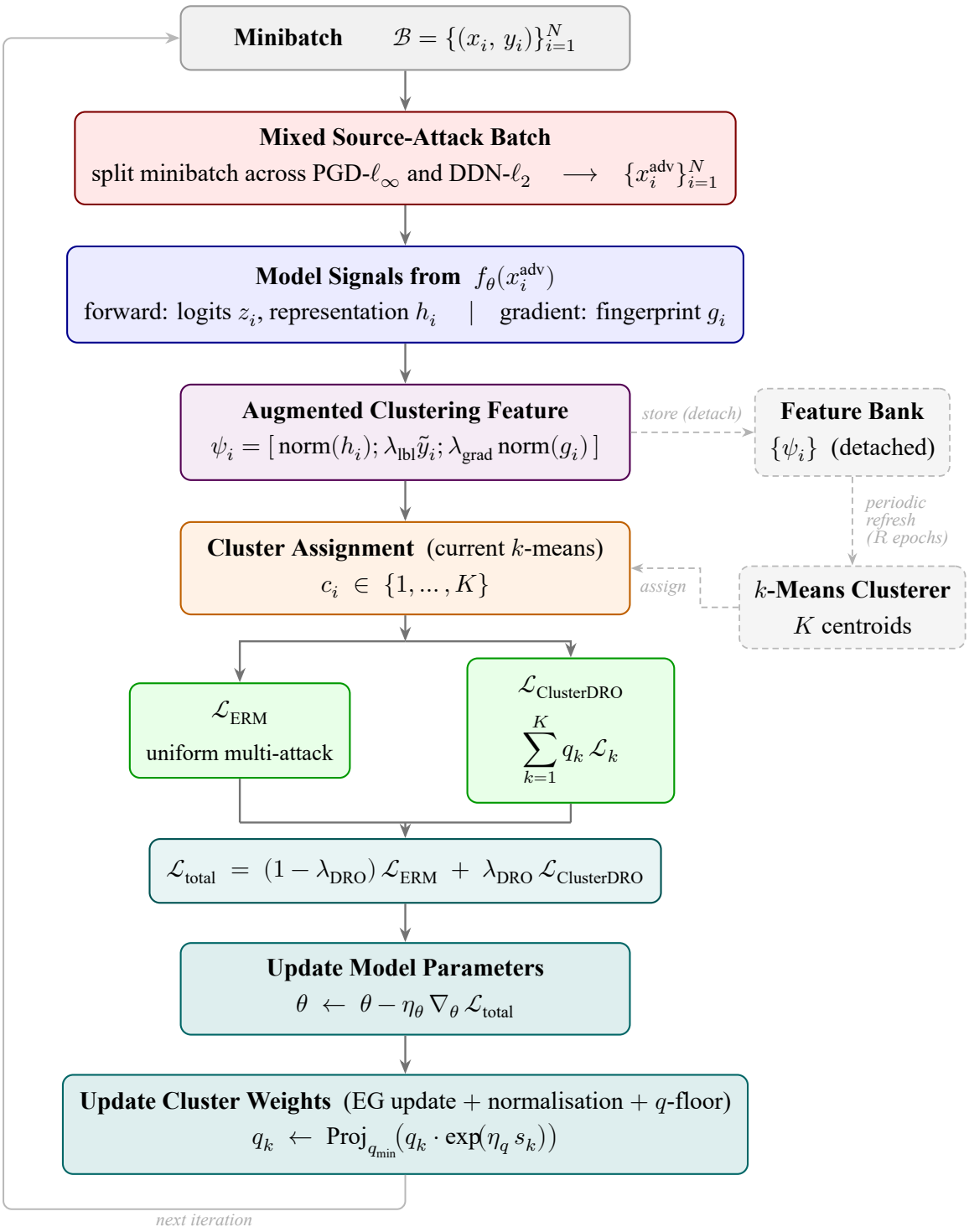


Figure 4.2: **Core training loop of AttackDRO++.** At each iteration, the minibatch is split across two source attacks (PGD-ℓ_∞ and DDN-ℓ_2), producing a mixed adversarial batch. The batch is passed through f_θ to obtain logits z_i, penultimate representations h_i, and gradient fingerprints g_i. Logits are used to construct prediction and gradient-fingerprint terms, while the canonical augmented clustering feature is $\psi_i = [\text{norm}(h_i); \lambda_{\text{lbl}}\tilde{y}_i; \lambda_{\text{grad}}\,\text{norm}(g_i)]$. These features are assigned to one of K latent clusters by the current k-means clusterer. The objective interpolates a uniform multi-attack ERM term $\mathcal{L}_{\text{ERM}}$ with a cluster-DRO term $\mathcal{L}_{\text{ClusterDRO}} = \sum_k q_k \mathcal{L}_k$ using anchor coefficient λ_{DRO}. Model parameters θ are updated every step, while cluster weights $\mathbf{q}$ are updated after the warmup period using exponentiated-gradient updates with the configured cluster score s_k and floor projection; the detached feature bank periodically refreshes the k-means centroids every R epochs.

Algorithm 1: AttackDRO++ with Gradient Fingerprints and Uniform Anchor

Input: Training set $\mathcal{D}_{\text{train}}$, model f_θ, source attacks $\mathcal{A}_{\text{src}} = \{A_1, \dots, A_M\}$, Cluster count K, anchor weight λ_{DRO}, learning rates η_θ, η_q, q-floor $q_{\min}$, Label weight λ_{lbl}, gradient weight λ_{grad}, projection dimension d_{proj}, Refresh interval R, warmup epochs T_w, q-update metric

Output: Trained model f_θ^*

Initialize $\mathbf{q} \leftarrow (1/K, \dots, 1/K)$
Initialize feature bank $\mathcal{F} \leftarrow \emptyset$
Initialize K-means clusterer as unavailable
Sample fixed projection matrix $P \in \mathbb{R}^{Cd_h \times d_{\text{proj}}}$
for *epoch* $e = 1, \dots, E$ **do**
    for *each minibatch* (x, y) *of size* N **do**
        Generate mixed adversarial batch x^{adv} using $\mathcal{A}_{\text{src}}$
        Compute logits $z \leftarrow f_\theta(x^{\text{adv}})$ and features $h \leftarrow \phi_\theta(x^{\text{adv}})$
        Build augmented clustering features $\{\psi_i\}_{i=1}^N \leftarrow \text{BuildFeature}(z_i, h_i, y_i, P)$
        Assign cluster ids $c_i \leftarrow \text{AssignCluster}(\psi_i, \text{Clusterer}, K)$ for all samples
        Compute uniform loss $\mathcal{L}_{\text{ERM}}$
        Compute cluster-DRO loss $\mathcal{L}_{\text{ClusterDRO}}$
        Compute final loss

$$\mathcal{L} = (1 - \lambda_{\text{DRO}})\mathcal{L}_{\text{ERM}} + \lambda_{\text{DRO}}\mathcal{L}_{\text{ClusterDRO}}.$$

        Update model parameters θ using SGD
        if $e > T_w$ **then**
            Compute cluster scores s_k from the configured q-update metric
            Update $\mathbf{q}$ using exponentiated gradient with s_k and floor projection
        end
        Add detached features $\{\psi_i\}_{i=1}^N$ to feature bank $\mathcal{F}$
    end
    if *refresh condition is met* **then**
        Refresh K-means clusterer using detached ψ_i in $\mathcal{F}$
        Apply q-refresh policy to $\mathbf{q}$ `// default: reset to uniform`
        Reset or update feature bank $\mathcal{F}$ according to the memory policy
    end
end
return f_θ^*

The cluster refresh schedule is treated as a configurable training choice rather than a

fixed mathematical requirement. It can be instantiated using either epoch-based or step-based schedules. Unless otherwise specified, the main experiments use an epoch-based schedule with $R = 2$ epochs and reset $\mathbf{q}$ to the uniform distribution after each cluster refresh. The cluster reweighting score s_k is also configurable: the implementation can update $\mathbf{q}$ from cluster loss, from the configured difficulty score, or from a hybrid of the two.

4.7.1 Progression from Stage 1 to the Final Method

This methodology directly extends the attacks-as-domains framework from Stage 1, which diagnosed the cross-attack robustness gap. Stage 2 moves from diagnosis to intervention. The progression builds sequentially: Single-AT diagnoses the cross-attack gap by training against one source attack, which leaves a large vulnerability to other threat models. Uniform multi-attack ERM covers multiple source attacks with equal weighting, providing a strong baseline but failing to adapt to weak groups. AttackDRO introduces attack-level reweighting but assumes each attack identity is homogeneous. To resolve this, the cluster-DRO component replaces fixed attack identities with discovered latent groups. Gradient fingerprints enrich this grouping with optimization-level information, and the uniform anchor stabilizes the adaptive objective by preserving the strong multi-attack training signal. Together, these form the final AttackDRO++ method.

The evaluation discipline from Stage 1 carries through the method design: source attacks are fixed during training, heldout attacks are never used for optimization, and results are reported both per attack and through aggregate robustness metrics.

4.7.2 Implementation and Computational Considerations

The final implementation evaluated in the main experiments uses gradient fingerprints, an online K-means refresh policy, $K = 4$ clusters, and uniform-anchor strength $\lambda_{\text{DRO}} = 0.35$. Chapter 5 documents the full training configuration, software stack, and evaluation protocol.

The main additional state introduced by AttackDRO++ is the feature bank used for cluster refresh. The bank stores detached augmented clustering features ψ_i, so clustering does not backpropagate through past minibatches. K-means is refitted only at the configured refresh interval, and the cluster weights are updated using the configured cluster score s_k before being reset or refreshed when cluster identities change. This keeps the adaptive grouping mechanism tied to recent model representations while avoiding a per-step clustering cost.

The dominant computational cost remains adversarial example generation, because all compared multi-attack methods use the same mixed source attacks. The AttackDRO++ additions therefore change the training objective and bookkeeping rather than the threat model or evaluation protocol. Chapter 5 specifies the concrete training configuration, software stack, and evaluation procedures used to run this implementation.

Table 4.1: Default hyperparameters for AttackDRO++ in the main experiments.

Hyperparameter	Symbol	Value	Role
DRO anchor strength	λ_{DRO}	0.35	Interpolates between uniform ERM and Cluster-DRO
Number of clusters	K	4	Controls latent group granularity
q learning rate	η_q	3×10^{-4}	Step size for exponentiated-gradient update using s_k
q-floor	$q_{\min}$	0.03	Minimum mass retained by each cluster
q-warmup	T_w	3 epochs	Delays adaptive reweighting until clusters are more reliable
Gradient projection dim.	d_{proj}	128	Dimension of projected gradient fingerprint
Gradient weight	λ_{grad}	0.1	Weight of GradFP component in ψ_i
Label weight	λ_{lbl}	0.1	Weight of scalar label component in ψ_i
Feature bank size	$\lvert\mathcal{F}\rvert$	4,096	Buffer used to refit K-means
Cluster refresh interval	R	2 epochs	Default K-means refit frequency
q-refresh policy	N/A	reset	Resets $\mathbf{q}$ to uniform after cluster refresh
Bootstrap assignment	N/A	random	Used before the first K-means refit

Up to this point, we have formulated cross-attack adversarial training as a domain-aware optimization problem and introduced the progression from Multi-AT to AttackDRO and AttackDRO++. The final method is not a collection of independent additions. Each component addresses a limitation of the previous step: Multi-AT provides source attack coverage, AttackDRO introduces group-aware reweighting, cluster-level DRO replaces coarse attack identities with latent groups, gradient fingerprints make grouping optimization-aware, and the uniform anchor keeps the adaptive objective tied to the stable multi-attack baseline.

The chapter has defined the method and its training pipeline, but it has not yet established how the method behaves empirically. Chapter 5 therefore specifies the dataset, attacks, metrics, and evaluation protocols used to evaluate the proposed method.

Chapter 5

Experimental Setup

This chapter defines the reproducibility contract for the empirical analysis: dataset splits, model architecture, attack suite, training configuration, hyperparameters, statistical protocol, evaluation metrics, graybox transfer design, and experiment tracking. Results and interpretation are reserved for Chapter 6.

5.1 Dataset and Preprocessing

The experiments are conducted on CIFAR-10 [39], a standard image-classification benchmark consisting of ten object categories.

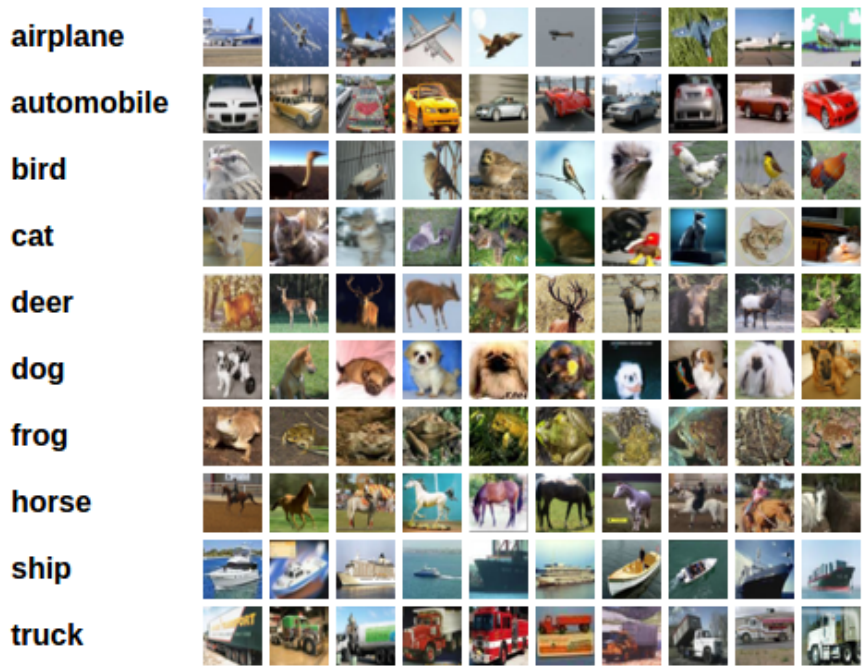


Figure 5.1: Example images from the CIFAR-10 dataset. CIFAR-10 contains ten object categories represented by low-resolution 32×32 color images.

Each image has spatial resolution 32×32 with three color channels. The official dataset contains 50,000 training images and 10,000 test images. In our experimental pipeline, the original training split is partitioned into 40,000 images for training and 10,000 images for validation and checkpoint selection, while the official 10,000-image test split is reserved for final test-set evaluation.

The validation set is used only for model selection and hyperparameter monitoring. Final reported numbers are computed on the test set after the checkpoint has been selected. This separation is important because adversarial robustness estimates can vary across random seeds and training checkpoints; using a held-out validation set reduces the risk of selecting a checkpoint based on test-set performance. All images are normalized using the same preprocessing pipeline across methods. During training, standard CIFAR-10 data augmentation is applied to the clean images before adversarial examples are generated. The augmentation includes random cropping with padding and random horizontal flipping. During validation and testing, no stochastic augmentation is used. This ensures that differences between methods are caused by the training objective rather than by differences in preprocessing or evaluation.

The same dataset split, preprocessing pipeline, and evaluation dataloaders are used for all compared methods. This includes the single-attack PGD-AT and DDN-AT baselines, Multi-AT, AttackDRO, Cluster-DRO, and AttackDRO++.

5.2 Model Architecture

The primary architecture used in this study is ResNet-18, based on the residual learning framework of He et al. [40], adapted for CIFAR-10 resolution. ResNet-18 provides a useful balance between expressiveness and computational cost: it is large enough to support nontrivial adversarial training experiments, but still efficient enough to run repeated-seed comparisons and ablation studies.

The classifier is denoted by f_θ. For an input image x, the model outputs logits

$$z = f_\theta(x) \in \mathbb{R}^C,$$

where $C = 10$ is the number of CIFAR-10 classes. In addition to logits, the proposed method requires access to the penultimate-layer representation

$$h = \phi_\theta(x) \in \mathbb{R}^{d_h}.$$

This feature vector is used for clustering adversarial examples in AttackDRO++ and for constructing the gradient-fingerprint augmented feature described in Chapter 4. For ResNet-18 on CIFAR-10, the penultimate representation dimension is $d_h = 512$. Therefore, the raw classifier-head gradient proxy has dimension $Cd_h = 10 \times 512 = 5120$, which is later reduced by random projection.

The instantiated model differs from the ImageNet ResNet-18 stem before the residual stages begin: the training code replaces the standard 7×7, stride-2 convolution and max-pooling with a CIFAR-adapted 3×3, stride-1 convolution followed by an identity max-pool; Table 5.1 records the resulting tensor shapes and trainable parameters used by the report.

Table 5.1: CIFAR-10 adapted ResNet-18 architecture used in the instantiated training model.

Layer / block	Input shape	Output shape	Params
Conv2d 3×3, 64, stride 1	[1, 3, 32, 32]	[1, 64, 32, 32]	1,728
BatchNorm2d + ReLU	[1, 64, 32, 32]	[1, 64, 32, 32]	128
Identity maxpool	[1, 64, 32, 32]	[1, 64, 32, 32]	--
layer1: 2 BasicBlocks, 64 channels	[1, 64, 32, 32]	[1, 64, 32, 32]	147,968
layer2: 2 BasicBlocks, 128 channels	[1, 64, 32, 32]	[1, 128, 16, 16]	525,568
layer3: 2 BasicBlocks, 256 channels	[1, 128, 16, 16]	[1, 256, 8, 8]	2,099,712
layer4: 2 BasicBlocks, 512 channels	[1, 256, 8, 8]	[1, 512, 4, 4]	8,390,656
AdaptiveAvgPool2d	[1, 512, 4, 4]	[1, 512, 1, 1]	--
Linear classifier	[1, 512]	[1, 10]	5,130
Total trainable parameters	--	--	**11,170,122**

Unless otherwise stated, all main experiments use the same ResNet-18 architecture and differ only in the training objective. This controlled setup ensures that performance differences can be attributed to the training method rather than to architectural changes. Higher-capacity architectures are treated as scaling checks or future extensions rather than the main basis for comparison.

5.3 Attack Suite and Configuration

The evaluation protocol tests whether robustness learned from a small set of source attacks generalizes to a broader attack suite. The attack suite is divided into two groups: source attacks used during training and evaluation attacks used only for validation or final testing.

5.3.1 Training Attacks

The main training setup uses two source attacks:

$$\mathcal{A}_{\text{src}} = \{\text{PGD-}\ell_\infty, \text{DDN-}\ell_2\}.$$

PGD-ℓ_∞ represents the standard first-order ℓ_∞ adversarial training setting. It uses cross-entropy loss and perturbation budget $\varepsilon_\infty = 8/255$. DDN-$\ell_2$ is based on the Decoupled Direction and Norm attack of Rony et al. [13]. DDN is an efficient gradient-based ℓ_2 attack that decouples the perturbation direction from its norm: the direction is updated using gradient information, while the perturbation norm is adapted depending on whether the current example is already adversarial.

Because all multi-attack methods share the same adversarial data generation process, Table 5.2 fixes the source attacks before the report compares weighting and grouping strategies.

Table 5.2: Source attack configuration used during training. Both attacks are used to generate mixed adversarial minibatches for Multi-AT, AttackDRO, Cluster-DRO, and AttackDRO++.

Attack	Norm	Budget	Steps	Loss / objective	Role
PGD-ℓ_∞	ℓ_∞	$8/255$	20	Cross-entropy	Source attack
DDN-ℓ_2	ℓ_2	0.5	20	DDN objective	Source attack

For Multi-AT training, each minibatch is split across the source attacks and adversarial examples are generated using the assigned attack. The same mixed-batch source attack construction underlies the AttackDRO and Cluster-DRO design steps and the final AttackDRO++ method. Thus, the main Chapter 6 comparison keeps adversarial data generation fixed and focuses on PGD-AT, DDN-AT, Multi-AT, and AttackDRO++.

5.3.2 Evaluation Attacks

The evaluation suite includes both source attacks and held-out attacks. The two source attacks used during training are also included in validation and test evaluation, while the remaining attacks are held out from training. This protocol directly tests the attacks-as-domains hypothesis: a robust model should not only perform well on the attacks used for optimization, but should also generalize to attacks with different perturbation geometries and optimization dynamics.

For validation and final testing, the attack suite in Table 5.3 defines the shared evaluation surface used to measure both seen-source robustness and held-out generalization. The main aggregate metric, denoted Mean(8), is computed over the eight attacks in the table. AutoAttack is reported separately and is therefore not included in Mean(8).

Table 5.3: Validation and test attacks used to compute Mean(8), the average robust accuracy over the eight non-AutoAttack evaluation attacks. Source attacks are also used during training; held-out attacks are used only for validation and test evaluation.

Attack	Norm	Budget	Steps	Group
FGSM-RS	ℓ_∞	$8/255$	1	Held-out ℓ_∞
PGD-ℓ_∞	ℓ_∞	$8/255$	20	Source
TPGD	ℓ_∞	$8/255$	10	Held-out ℓ_∞
MI-FGSM	ℓ_∞	$8/255$	10	Held-out ℓ_∞
PGD-ℓ_2	ℓ_2	0.5	10	Held-out ℓ_2
DDN-ℓ_2	ℓ_2	0.5	20	Source
DeepFool-ℓ_2	ℓ_2	0.5	50	Held-out ℓ_2
CW-ℓ_2	ℓ_2	0.5	200	Held-out ℓ_2

For aggregate analysis, we report four summary metrics derived from the same eight attacks: Seen, Held-out ℓ_∞, Held-out ℓ_2, and Mean(8). Their exact definitions are given in Section 5.7.

AutoAttack configuration AutoAttack is reported as a separate standardized ℓ_∞ stress test following Croce and Hein [5]. It is excluded from Mean(8) so that the main aggregate metric remains tied to the fixed eight-attack evaluation suite, while AutoAttack serves as an independent robustness check. This also reflects that AutoAttack is an ensemble protocol, not a single attack, so it is reported separately.

The AutoAttack configuration uses the standard ℓ_∞ setting with perturbation budget $\varepsilon_\infty = 8/255$. In early experiments and ablations, AutoAttack is sometimes evaluated on a subset of 512 test samples as a sanity check. These 512-sample results are useful for detecting severe robustness collapse, but they are not used as the primary ranking criterion because the sample size is smaller and the variance is higher. For the final stress test, AutoAttack is evaluated on the full test set for a fixed random-seed panel.

Table 5.4: AutoAttack-ℓ_∞ configuration used as a separate stress test. AutoAttack is reported separately from Mean(8).

Attack	Norm	Budget	Role
AutoAttack	ℓ_∞	$8/255$	Separate stress test

5.4 Training Configuration

All methods are trained under a controlled adversarial training pipeline. Unless otherwise stated, the optimizer, model architecture, dataset split, preprocessing pipeline, and evaluation protocol are kept fixed across methods. This ensures that performance differences mainly come from the training objective rather than from implementation or evaluation differences.

Table 5.5: General training configuration used for the main experiments.

Component	Setting	Description
Dataset	CIFAR-10	40k train / 10k validation / 10k test.
Backbone	ResNet-18	CIFAR-10 adapted residual network.
Training epochs	50	Main training budget.
Batch size	128	Used for all main methods.
Optimizer	SGD	Momentum-based stochastic gradient descent.
Momentum	0.9	Standard SGD momentum.
Weight decay	5×10^{-4}	Weight regularization.
Initial learning rate	0.1	Decayed during training.
Learning-rate schedule	Multi-step	Same schedule for all compared methods.
Learning-rate milestones	(30, 40)	Epochs at which the learning rate is decayed.
Mixed precision	Enabled	Reduces memory usage and training time.
Gradient clipping	Enabled	Stabilizes adversarial training.
Checkpoint selection	Validation robust average	Test set is used only after selection.

To make method differences attributable to the training objective, Table 5.5 fixes the shared training configuration for the main experiments. All main experiments use ResNet-18 on CIFAR-10 with stochastic gradient descent (SGD), momentum, weight decay, mixed-precision training, and a multi-step learning-rate schedule.

The main methods are trained for 50 epochs. During training, each minibatch contains a balanced mixture of source attacks. In the two-source setting, approximately half of the minibatch is assigned to PGD-ℓ_∞ and the other half to DDN-ℓ_2.

Checkpoint selection is based on validation robustness rather than test performance. After training, the selected checkpoint is evaluated once on the test set. This protocol is applied consistently to the reported method checkpoints.

The method overview in Table 5.6 separates the main methods and intermediate design objectives by their grouping or weighting strategy. All multi-attack objectives use the same source attacks and mixed-batch adversarial example generation; they differ only in how adversarial examples are weighted or grouped during optimization. The main empirical comparison in Chapter 6 focuses on PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ under the fixed protocol.

Table 5.6: Training methods and intermediate objectives defined for the CIFAR-10 and ResNet-18 experiments.

Method	Grouping / weighting strategy	Experimental role
PGD-AT	One source attack only	Specialist ℓ_∞ baseline.
DDN-AT	One source attack only	Specialist ℓ_2 baseline.
Multi-AT	Uniform sample weighting over mixed PGD-ℓ_∞ and DDN-ℓ_2 batches	Main strong baseline.
AttackDRO	Group DRO over attack identity	Coarse attack-level DRO baseline.
Cluster-DRO	Group DRO over discovered clusters	Intermediate cluster-level grouping baseline.
AttackDRO++	Uniform-anchored Cluster-DRO with gradient fingerprints	Proposed final method.

For the proposed method, cluster assignments are refreshed periodically using a feature bank. The default refresh interval is two epochs. The cluster weight vector $\mathbf{q}$ is updated only after a warmup period, so that early noisy representations do not dominate adaptive reweighting.

5.5 Hyperparameter Choices and Ablation Factors

Table 5.7: Default hyperparameters for AttackDRO++.

Hyperparameter	Symbol	Default value	Purpose
Source attacks	$\mathcal{A}_{\text{src}}$	PGD-ℓ_∞, DDN-ℓ_2	Training attack domains.
Number of clusters	K	4	Latent group granularity.
DRO anchor strength	λ_{DRO}	0.35	Balances ERM and Cluster-DRO.
q learning rate	η_q	3×10^{-4}	Step size for q update.
q floor	$q_{\min}$	0.03	Prevents ignored clusters.
q warmup	T_w	3 epochs	Delays early reweighting.
Cluster refresh interval	R	2 epochs	K-means refresh frequency.
Feature bank size	$\lvert\mathcal{F}\rvert$	4096	Stores detached features.
Gradient projection dim.	d_{proj}	128	Compresses GradFP features.
Gradient feature weight	λ_{grad}	0.1	Weights GradFP features.
Label feature weight	λ_{lbl}	0.1	Weights label features.
Clusterer	–	K-means	Assigns latent groups.
q refresh policy	–	Reset after refresh	Avoids stale cluster weights.

The number of clusters is set to $K = 4$ as a moderate grouping resolution. The default anchor strength $\lambda_{\text{DRO}} = 0.35$ is chosen to keep the Multi-AT loss as the dominant stabilizing signal while still allowing cluster-level adaptive correction.

The q-floor and warmup settings are included for stability. The floor projection prevents any cluster from being completely ignored, while the warmup period prevents the exponentiated-gradient update from acting on unreliable early cluster assignments. The feature bank and refresh interval control how quickly the clusterer adapts to evolving model representations. In the default setting, refreshing every two epochs provides a balance between stability and adaptivity.

The gradient fingerprint is projected to $d_{\text{proj}} = 128$ dimensions for computational efficiency. For CIFAR-10 with ResNet-18, the classifier-head gradient proxy has raw dimension $Cd_h = 10 \times 512 = 5120$. The fixed random projection reduces this dimension while retaining useful information about the optimization direction induced by each adversarial example.

The ablation factors in Table 5.8 are the configuration levers used in Chapter 6 to justify the default setting and to separate the effect of the proposed components from the strong uniform multi-attack baseline.

Table 5.8: Hyperparameters varied in ablation studies.

Ablation factor	Values / variants	Purpose
Anchor strength	$\lambda_{\text{DRO}} \in \{0.20, 0.35, 0.50\}$	Tests the uniform-anchor trade-off.
Cluster count	$K \in \{2, 4, 6, 10\}$	Tests sensitivity to group granularity.
q update	update q vs. uniform q	Tests adaptive reweighting.
Refresh schedule	1 epoch vs. 2 epochs	Tests cluster-refresh frequency.
Gradient fingerprint	With vs. without GradFP	Tests optimization-level features.
Random clusters	Learned vs. random assignment	Negative control for clustering.

5.6 Statistical Significance Protocol

To ensure that the reported improvements are not caused by seed-specific randomness, this study applies a repeated-seed statistical protocol. Each method is trained and evaluated under the same set of seeds, dataset split, architecture, attack suite, and evaluation configuration. The seed-level result is used as the basic unit of analysis.

For a given metric, let x_i be the performance of AttackDRO++ under seed i, and let y_i be the performance of a baseline method under the same seed. The paired difference is defined as:

$$d_i = x_i - y_i. \tag{5.1}$$

All statistical comparisons are performed on these paired differences. This paired design reduces the influence of random initialization, data shuffling, and optimization noise, allowing the analysis to focus on the relative performance difference between methods.

The primary significance test is the Wilcoxon signed-rank test. This test is selected because it is non-parametric and does not assume that the paired differences are normally distributed. This property is important for repeated-seed adversarial robustness experiments, where the number of seeds is limited and the distribution of performance differences may be irregular. For directional comparisons, the alternative hypothesis is that AttackDRO++ achieves higher performance than the baseline:

$$H_0 : \text{median}(d_i) \leq 0, \tag{5.2}$$

$$H_1 : \text{median}(d_i) > 0. \tag{5.3}$$

A paired t-test is also reported as a secondary parametric check. While the paired t-test assumes approximate normality of the paired differences, it provides a useful sanity check. When both the Wilcoxon signed-rank test and the paired t-test support the same conclusion, the result is considered more stable. If the two tests disagree, the Wilcoxon test is prioritized because it is less dependent on distributional assumptions.

In addition to p-values, this study reports the paired effect size using Cohen's d_z:

$$d_z = \frac{\bar{d}}{s_d}, \tag{5.4}$$

where $\bar{d}$ is the mean paired difference and s_d is the sample standard deviation of the paired differences. Cohen's d_z is used to quantify the magnitude of the improvement. Conventional thresholds are used as rough guidance, with 0.2, 0.5, and 0.8 corresponding to small, medium, and large effects, respectively. However, in adversarial robustness evaluation, even moderate effect sizes may be practically meaningful if the improvement is consistent across strong attacks or improves worst-case robustness.

To estimate uncertainty, 95% bootstrap confidence intervals are computed from seed-level results. For each method and metric, the seed-level values are resampled with replacement 10,000 times, and the mean is recomputed for each resample. The 2.5th and 97.5th percentiles of the bootstrap distribution are used as the confidence interval. Bootstrap confidence intervals are also computed for the paired difference $x_i - y_i$, as this directly estimates the uncertainty of the improvement over each baseline. In result tables, CI denotes confidence interval, and paired differences are reported in percentage points (pp).

Because multiple attacks, metrics, and baseline comparisons are evaluated, Holm–Bonferroni correction is applied within each metric family. Both raw and corrected p-values are reported, but corrected p-values are used for final significance interpretation. This correction reduces the risk of false positive findings while remaining less conservative than the standard Bonferroni correction.

The reporting format in Table 5.9 keeps the results chapter tied to the paired-seed protocol by defining the quantities reported for each method comparison.

Table 5.9: Reporting format for the statistical significance protocol used in Chapter 6.

Quantity	Reported value	Purpose
Mean performance	Mean across seeds	Reports average robustness.
Seed variation	Standard deviation	Measures run-to-run variability.
Uncertainty	95% bootstrap confidence interval	Estimates uncertainty of the mean.
Primary test	Wilcoxon signed-rank test	Non-parametric paired comparison.
Secondary test	Paired t-test	Parametric sanity check.
Effect size	Cohen's d_z	Measures magnitude of improvement.
Correction	Holm–Bonferroni correction	Controls multiple comparisons.

A comparison is considered statistically supported when AttackDRO++ improves the mean metric, obtains a corrected p-value below the significance threshold, and shows a non-trivial paired effect size. This protocol is applied consistently to whitebox robustness, graybox transfer robustness, aggregate robustness metrics, and ablation studies.

5.7 Evaluation Metrics and Robustness Definitions

This section defines the metrics used to compare all methods. Let

$$\mathcal{D}_{\text{test}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$$

denote the test set, and let A be an adversarial attack. The adversarial example generated by A is denoted by

$$\mathbf{x}_i^A = A(f_\theta, \mathbf{x}_i, y_i).$$

The robust accuracy under attack A is

$$\text{Acc}(A) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_\theta(\mathbf{x}_i^A)_c = y_i\right]. \tag{5.5}$$

Clean accuracy is computed without adversarial perturbation:

$$\text{Acc}_{\text{clean}} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_\theta(\mathbf{x}_i)_c = y_i\right]. \tag{5.6}$$

The primary aggregate metric is Mean(8), defined as the average robust accuracy over

the eight non-AutoAttack evaluation attacks:

$$\text{Mean}(8) = \frac{1}{|\mathcal{A}_8|} \sum_{A \in \mathcal{A}_8} \text{Acc}(A), \tag{5.7}$$

where

$$\mathcal{A}_8 = \{\text{FGSM-RS}, \text{PGD-}\ell_\infty, \text{TPGD}, \text{MI-FGSM}, \\ \text{PGD-}\ell_2, \text{DDN-}\ell_2, \text{DeepFool-}\ell_2, \text{CW-}\ell_2\}.$$

We also report the worst robust accuracy across the same eight attacks:

$$\text{Worst}(8) = \min_{A \in \mathcal{A}_8} \text{Acc}(A). \tag{5.8}$$

For finer analysis, we report seen-source, held-out ℓ_∞, and held-out ℓ_2 averages:

$$\text{Seen} = \frac{1}{2} \left[\text{Acc}(\text{PGD-}\ell_\infty) + \text{Acc}(\text{DDN-}\ell_2)\right], \tag{5.9}$$

$$\text{Heldout}_{\ell_\infty} = \frac{1}{3} \left[\text{Acc}(\text{FGSM-RS}) + \text{Acc}(\text{TPGD}) + \text{Acc}(\text{MI-FGSM})\right], \tag{5.10}$$

$$\text{Heldout}_{\ell_2} = \frac{1}{3} \left[\text{Acc}(\text{PGD-}\ell_2) + \text{Acc}(\text{DeepFool-}\ell_2) + \text{Acc}(\text{CW-}\ell_2)\right]. \tag{5.11}$$

AutoAttack-ℓ_∞ is reported separately from Mean(8). We use AutoAttack-ℓ_∞ on 512 test samples as a sanity check (we denote this subset evaluation as AutoAttack-512), while final AutoAttack comparisons are reported on the full test set for random seeds.

For graybox evaluation, adversarial examples are generated using a surrogate model f_{θ_s} and evaluated on a target model f_{θ_t}:

$$\text{Acc}_{s \to t}(A) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\left[\arg\max_c f_{\theta_t}\left(A(f_{\theta_s}, \mathbf{x}_i, y_i)\right)_c = y_i\right]. \tag{5.12}$$

Thus, the same table supports whitebox, same-method transfer, and cross-method graybox evaluation.

Finally, for fine-grained analysis, each attack-class pair is treated as an evaluation group. For attack A and class c, the attack-class group accuracy is

$$\text{Acc}(A, c) = \frac{1}{|\mathcal{D}_c|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_c} \mathbb{I}\left[\arg\max_j f_\theta(\mathbf{x}_i^A)_j = y_i\right], \tag{5.13}$$

where $\mathcal{D}_c$ is the subset of test examples whose label is c.

Table 5.10: Evaluation metrics used for whitebox, graybox, aggregate, and per-class analysis.

Metric	Definition	Purpose
Clean accuracy	Accuracy on unperturbed test images	Measures standard classification performance.
Per-attack robust accuracy	Accuracy under a single adversarial attack	Shows which attacks each method resists or fails against.
Mean(8)	Average accuracy over the eight non-AutoAttack attacks	Main aggregate robustness metric.
Worst(8)	Minimum accuracy over the eight non-AutoAttack attacks	Measures the weakest attack in the suite.
Seen-source mean	Average over PGD-ℓ_∞ and DDN-ℓ_2	Measures robustness on source attacks.
Held-out ℓ_∞ mean	Average over FGSM-RS, TPGD, and MI-FGSM	Measures transfer to unseen ℓ_∞ attacks.
Held-out ℓ_2 mean	Average over PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2	Measures transfer to unseen ℓ_2 attacks.
AutoAttack-ℓ_∞	Standard ℓ_∞ AutoAttack robust accuracy	Separate robustness stress test; not included in Mean(8).
Graybox transfer accuracy	Target accuracy on adversarial examples generated by a surrogate	Measures transfer robustness across models.
Attack-class group accuracy	Accuracy for each attack $\times$ class pair	Diagnoses fine-grained failure modes.

5.8 Graybox Transfer Protocol

To evaluate whether the proposed method changes the structure of learned robustness rather than only improving whitebox accuracy, we conduct a cross-model graybox transfer experiment. The experiment uses four method families: PGD-AT, DDN-AT, Multi-AT, and AttackDRO++. Each method is evaluated over five random seeds, giving 20 random-seed checkpoints in total.

Each checkpoint is used in two roles. As a *surrogate*, it generates adversarial examples. As a *target*, it is evaluated on adversarial examples generated by all surrogate checkpoints. Therefore, the full protocol evaluates

$$20 \times 20 = 400$$

surrogate–target pairs. This design produces whitebox, same-method transfer, and cross-

method graybox transfer results from a single unified evaluation table.

To keep the computation tractable while still supporting per-class and per-attack analysis, each surrogate checkpoint generates a balanced adversarial cache. The CIFAR-10 test set contains 1000 images per class. For each class, the test images are split evenly across the eight evaluation attacks. Thus, each attack-class cell contains 125 source images:

$$10 \text{ classes} \times 8 \text{ attacks} \times 125 \text{ examples} = 10{,}000$$

adversarial examples per surrogate checkpoint.

The class-wise split is generated deterministically using a fixed shuffle seed. Therefore, the same attack-class cell always refers to the same underlying CIFAR-10 source images across surrogate checkpoints. This makes paired comparisons between surrogate and target methods well aligned.

For each surrogate–target pair, the evaluation produces one row per attack-class group, indexed by surrogate method, surrogate seed, target method, target seed, attack, and class. Since there are 80 attack-class groups per pair, the full table contains $400 \times 80 = 32{,}000$ rows. Transfer matrices, attack-class drilldowns, and paired method comparisons are all computed by aggregating this same long-form table, so the regimes in Table 5.11 should be read as different slices of one aligned transfer design.

Table 5.11: Transfer regimes in the graybox evaluation protocol.

Regime	Condition	Rows	Purpose
Whitebox	Same method and same seed	$20 \times 80 = 1{,}600$	Recovers standard whitebox robustness as a sanity check.
Same-method transfer	Same method, different seed	$80 \times 80 = 6{,}400$	Measures seed-to-seed transfer within a method family.
Cross-method same-seed	Different methods, same seed	$60 \times 80 = 4{,}800$	Measures transfer between method families under matched seeds.
Cross-method cross-seed	Different methods, different seeds	$240 \times 80 = 19{,}200$	Measures the most general cross-method graybox transfer setting.

In Chapter 6, the cross-method same-seed and cross-method cross-seed regimes are combined to form the main graybox transfer analysis. The whitebox and same-method transfer regimes are reported separately as diagnostic checks.

5.9 Tools, Platforms, and Experiment Tracking

The experiments in this report were implemented in Python using a PyTorch-based training pipeline. The codebase is organized around a configurable experiment script that defines dataset construction, model initialization, adversarial attack generation, training objectives, checkpoint selection, and fixed evaluation scripts in one place.

Table 5.12: Tools and platforms used in the experimental workflow. The table lists only tools that are part of the implemented training, evaluation, logging, or reporting pipeline.

Tool / platform	Role	Use in this report
Google Colab	Interactive execution environment	Used to run training and evaluation notebooks with GPU support.
Google Drive	Persistent experiment storage	Stores checkpoints, cached adversarial examples, result tables, figures, and intermediate analysis files.
Python	Main implementation language	Used for the full training, evaluation, analysis, and plotting pipeline.
PyTorch	Deep-learning framework	Implements model training, automatic differentiation, mixed-precision execution, and checkpoint loading.
torchvision	Dataset and model utilities	Provides CIFAR-10 loading, image transforms, and ResNet backbones adapted for small images.
TorchAttacks	Adversarial attack library	Provides most gradient-based and optimization-based attacks used in training and evaluation.
AutoAttack package	Standardized robustness stress test	Used for AutoAttack-ℓ_∞ evaluation with perturbation budget $8/255$.
adv-lib	DDN attack implementation	Used for DDN-ℓ_2 adversarial example generation.
scikit-learn	Clustering utilities	Provides MiniBatch K-means for cluster discovery in AttackDRO++.
NumPy and pandas	Result processing	Used to aggregate per-seed, per-attack, per-class, and graybox transfer results.
matplotlib and SciencePlots	Visualization	Used to generate plots and publication-style figures for analysis.
Weights & Biases	Experiment tracking	Logs configurations, training curves, validation metrics, group weights, and diagnostic figures when enabled.

This organization is important because the compared methods differ mainly in their grouping and weighting strategies, while the dataset split, architecture, source attacks, evaluation attacks, optimizer, and checkpointing rules are kept fixed.

Because reproducibility depends on both the training code and the analysis stack, Table 5.12 records the tools and platforms that form the experimental workflow. Google Colab and Google Drive are used for interactive execution and persistent storage of checkpoints, cached adversarial examples, figures, and result files. PyTorch and torchvision provide the core deep-learning infrastructure, including model definitions, data loading, and optimization. The attack implementations are built primarily with TorchAttacks, with AutoAttack-ℓ_∞ and DDN-ℓ_2 handled through their dedicated packages when required. scikit-learn is used for K-means clustering in AttackDRO++, while NumPy and pandas are used to aggregate seed-level and attack-level results.

Experiment tracking is treated as part of the reproducibility protocol rather than as a separate result source. Each run records the method name, seed, dataset, backbone, source attacks, evaluation attacks, and key hyperparameters. During training, the pipeline logs loss, adversarial accuracy, validation robustness, learning rate, checkpoint state, and group-weight diagnostics when applicable. Representative Weights & Biases (W&B) tracking exports are provided in Appendix B.

All test-set claims in Chapter 6 are based on saved checkpoints that are loaded after training and evaluated using fixed evaluation scripts. The main result tables are produced from structured result files rather than from manually copied console logs. This separation reduces the risk of mixing validation, test, whitebox, graybox, and diagnostic metrics. It also makes it possible to reuse the same result files for aggregate metrics, per-attack analysis, per-class analysis, and paired statistical comparisons.

Summary. The setup summary in Table 5.13 serves as the reproducibility contract for the result claims in Chapter 6 by collecting the dataset, model, attack suite, repeated-seed protocol, and analysis stack in one place.

Table 5.13: Summary of the experimental setup used in this report. Unless otherwise stated, evaluations use CIFAR-10, ResNet-18, and the eight non-AutoAttack attacks for Mean(8), with AutoAttack-ℓ_∞ reported separately.

Component	Configuration
Dataset	CIFAR-10 with 40,000 training images, 10,000 validation images, and 10,000 test images.
Backbone	ResNet-18 adapted for CIFAR-10 resolution.
Main Chapter 6 compared methods	PGD-AT, DDN-AT, Multi-AT, and AttackDRO++; AttackDRO and Cluster-DRO are intermediate design steps described in Chapter 4.
Source attacks	PGD-ℓ_∞ and DDN-ℓ_2.
Evaluation attacks for Mean(8)	FGSM-RS, PGD-ℓ_∞, TPGD, MI-FGSM, PGD-ℓ_2, DDN-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2.
Separate stress test	AutoAttack-ℓ_∞, reported separately from Mean(8).
Primary aggregate metric	Mean(8): mean robust accuracy over the eight non-AutoAttack evaluation attacks.
Secondary metrics	Worst(8), seen-source robustness, held-out ℓ_∞ robustness, held-out ℓ_2 robustness, per-attack accuracy, per-class accuracy, and graybox transfer accuracy.
Repeated-seed protocol	Aggregate whitebox tables use paired random-seed comparisons; class-wise and graybox panels use five random seeds per method.
Graybox protocol	Four method families are evaluated across five random seeds each, giving 20 random-seed checkpoints and 400 surrogate–target pairs.
Statistical analysis	Paired random-seed comparisons with Wilcoxon signed-rank tests, paired t-tests, bootstrap confidence intervals, effect sizes, and Holm–Bonferroni correction.
Implementation stack	PyTorch, torchvision, TorchAttacks, AutoAttack package, adv-lib, scikit-learn, NumPy, pandas, matplotlib, SciencePlots, W&B, Google Colab, and Google Drive.

With the experimental protocol fixed, the report now moves from setup to evidence. The next chapter uses this controlled setting to examine whether changing only the training objective is enough to reduce the cross-attack robustness gap. In particular, Chapter 6 compares the baseline methods with AttackDRO++ across aggregate, per-attack, per-class, and graybox transfer results, showing where the proposed approach improves robustness and where its limitations remain.

Chapter 6

Results and Analysis

6.1 Whitebox Robustness Under Direct Attacks

6.1.1 Aggregate Comparison Across Methods

Table 6.1: Aggregate robust accuracy (%) on the CIFAR-10 evaluation suite, reported as mean $\pm$ standard deviation across 20 random-seed runs. Mean(8) is the average robust accuracy over the eight non-AutoAttack evaluation attacks. Worst(8) is the lowest robust accuracy among those same eight attacks. The held-out attack mean averages FGSM-RS, TPGD, MI-FGSM, PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2. The held-out ℓ_∞ mean averages FGSM-RS, TPGD, and MI-FGSM; the held-out ℓ_2 mean averages PGD-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2. The seen-source mean averages PGD-ℓ_∞ and DDN-ℓ_2. AutoAttack is reported separately and is not included in Mean(8). Bold values indicate the best method for each metric.

Metric	PGD-AT	DDN-AT	Multi-AT	AttackDRO++
Mean(8)	60.499 ± 0.327	56.226 ± 0.233	62.045 ± 0.447	**62.324 ± 0.147**
Held-out attack mean	62.268 ± 0.390	59.628 ± 0.254	64.083 ± 0.505	**64.401 ± 0.155**
Held-out ℓ_∞ mean	**62.515 ± 0.334**	51.594 ± 0.306	60.916 ± 0.427	61.149 ± 0.170
Held-out ℓ_2 mean	62.021 ± 0.474	**67.661 ± 0.278**	67.249 ± 0.603	67.654 ± 0.223
Seen-source mean	55.192 ± 0.182	46.020 ± 0.247	55.934 ± 0.303	**56.093 ± 0.166**
Worst(8)	**49.656 ± 0.161**	25.957 ± 0.378	45.082 ± 0.281	45.079 ± 0.237

We compare four methods on 20 paired random-seed runs: PGD-AT, DDN-AT, Multi-AT, and AttackDRO++. Multi-AT is the main strong baseline because it uses both source attacks. Relative to this baseline, AttackDRO++ improves Mean(8) by $+0.279$ pp ($p = 0.0076$, 14/20 seed-level wins), with the largest channel on the held-out ℓ_2 mean

(+0.405 pp, $p = 0.0079$). These differences are small in absolute size, so the result is best read as a modest average improvement rather than a broad shift in all robustness metrics.

Worst(8) and AutoAttack-512 are statistically indistinguishable from Multi-AT ($|\Delta| < 0.005$ pp, $p > 0.97$). Because AutoAttack-512 is a separate sanity check and is not included in Mean(8), these rows do not change the main aggregate ranking. They show that the average gains are not accompanied by a measurable improvement in the standardized hardest-case checks.

The single-attack baselines confirm the expected specialist pattern. PGD-AT wins the held-out ℓ_∞ mean and Worst(8); DDN-AT wins the held-out ℓ_2 mean. Each is therefore strongest on the family closest to its training objective but loses substantially on the other. AttackDRO++ has the highest Mean(8), held-out attack mean, and seen-source mean, which are the metrics that average across both families. The next question is whether this aggregate advantage is broad across attacks or concentrated in a few cells.

Effect-size summary. Effect sizes complement the paired p-values by scaling each AttackDRO++–baseline difference by its seed-level variability; Table 6.2 provides this magnitude view, while Table 6.3 handles the corresponding aggregate significance tests. Cohen's $d_z = \Delta/\sigma_\Delta$ is the standardized paired effect size; conventional thresholds are 0.2 (small), 0.5 (medium), 0.8 (large).

Table 6.2: Effect-size summary for AttackDRO++ against each baseline across 20 paired random-seed runs. Δ in percentage points; negative values mean the baseline is stronger.

	vs. Multi-AT		vs. PGD-AT		vs. DDN-AT	
Metric	Δ (pp)	d_z	Δ (pp)	d_z	Δ (pp)	d_z
Mean(8)	+0.279	0.67	+1.826	5.98	+6.098	23.09
Held-out attack mean	+0.319	0.67	+2.134	5.65	+4.774	16.91
Held-out ℓ_∞ mean	+0.233	0.57	−1.366	−4.27	+9.555	25.44
Held-out ℓ_2 mean	+0.405	0.66	+5.633	11.64	−0.008	−0.03
Seen-source mean	+0.159	0.55	+0.901	4.59	+10.073	31.72
Worst(8)	−0.003	−0.01	−4.576	−15.36	+19.122	36.91

Two regimes are visible. Against Multi-AT, every average-case metric is a medium-effect gain ($d_z \in [0.55, 0.67]$, $\Delta \in [+0.16, +0.41]$ pp) with Worst(8) at zero effect:

AttackDRO++ shifts the average-case distribution by a small but consistent amount while leaving hardest-case behavior broadly unchanged. Against the single-AT baselines, d_z exceeds 4 on most metrics but flips negative on the family each specialist trains against. The medium-effect Multi-AT comparison supports the interpretation of AttackDRO++ as a refinement of Multi-AT; the large-but- asymmetric single-AT comparisons quantify the specialist tradeoff that motivates the multi-attack setting in the first place.

Paired aggregate robustness comparisons. The paired aggregate comparison in Table 6.3 separates the main Multi-AT baseline from the two single-attack specialists, which makes the uniform-baseline refinement and the specialist trade-off readable as different questions. AutoAttack is excluded from this table and reported separately in Section 6.1.3.

Table 6.3: Paired aggregate robustness comparison between AttackDRO++ and each baseline across 20 paired random-seed runs. Δ is computed as AttackDRO++ minus the corresponding baseline in percentage points. The p-value is from a two-sided paired t-test.

Baseline	Metric	n	Δ (pp)	σ_Δ	Wins	d_z	p
AttackDRO++ vs. Multi-AT							
Multi-AT	Mean(8)	20	$+0.279$	0.417	$14/20$	0.668	0.0076
Multi-AT	Held-out attack mean	20	$+0.319$	0.478	$13/20$	0.667	0.0076
Multi-AT	Held-out ℓ_∞ mean	20	$+0.233$	0.412	$14/20$	0.566	0.0204
Multi-AT	Held-out ℓ_2 mean	20	$+0.405$	0.609	$16/20$	0.664	0.0079
Multi-AT	Seen-source mean	20	$+0.159$	0.291	$15/20$	0.546	0.0246
Multi-AT	Worst(8)	20	-0.003	0.369	$9/20$	-0.008	0.971
AttackDRO++ vs. PGD-AT							
PGD-AT	Mean(8)	20	$+1.826$	0.305	$20/20$	5.980	$< 10^{-10}$
PGD-AT	Held-out attack mean	20	$+2.134$	0.378	$20/20$	5.650	$< 10^{-10}$
PGD-AT	Held-out ℓ_∞ mean	20	-1.366	0.320	$0/20$	-4.273	$< 10^{-10}$
PGD-AT	Held-out ℓ_2 mean	20	$+5.633$	0.484	$20/20$	11.638	$< 10^{-20}$
PGD-AT	Seen-source mean	20	$+0.901$	0.197	$20/20$	4.585	$< 10^{-10}$
PGD-AT	Worst(8)	20	-4.576	0.298	$0/20$	-15.364	$< 10^{-20}$
AttackDRO++ vs. DDN-AT							
DDN-AT	Mean(8)	20	$+6.098$	0.264	$20/20$	23.087	$< 10^{-20}$
DDN-AT	Held-out attack mean	20	$+4.774$	0.282	$20/20$	16.910	$< 10^{-20}$
DDN-AT	Held-out ℓ_∞ mean	20	$+9.555$	0.376	$20/20$	25.440	$< 10^{-20}$
DDN-AT	Held-out ℓ_2 mean	20	-0.008	0.319	$10/20$	-0.025	0.914
DDN-AT	Seen-source mean	20	$+10.073$	0.318	$20/20$	31.723	$< 10^{-20}$
DDN-AT	Worst(8)	20	$+19.122$	0.518	$20/20$	36.909	$< 10^{-20}$

Against Multi-AT, AttackDRO++ gives small but statistically supported gains on all average-case metrics, while Worst(8) is unchanged. Against PGD-AT, AttackDRO++ improves Mean(8) and held-out ℓ_2 robustness, but PGD-AT remains stronger on held-out ℓ_∞ and Worst(8). Against DDN-AT, AttackDRO++ improves all ℓ_∞-related aggregates while matching DDN-AT on held-out ℓ_2. This again supports the main interpretation that single-attack baselines are specialists, while AttackDRO++ provides better average cross-attack balance under this protocol. The limitation is that the uncorrected paired tests alone are not the most conservative evidence, so the next table applies the multiple-comparison protocol from Chapter 5.

Table 6.4: Multiple-comparison-corrected paired significance tests for AttackDRO++ against each baseline across $n = 20$ paired seeds. Δ is computed as AttackDRO++ minus the corresponding baseline in percentage points. The table reports bootstrap 95% confidence intervals, seed-level win counts, Cohen's paired effect size d_z, paired t-test p-values, Wilcoxon signed-rank p-values, and Holm-corrected Wilcoxon p-values. Holm correction is applied within each baseline comparison across the seven reported metrics. AutoAttack is excluded from this table and reported separately in Section 6.1.3.

Baseline	Metric	Δ (pp)	95% CI (pp)	Wins	d_z	p_t	p_W	$p_{W,\text{Holm}}$
AttackDRO++ vs. Multi-AT								
Multi-AT	Clean	+0.539	[+0.195, +0.903]	13/20	0.642	0.0098	0.0441	0.1449
Multi-AT	Mean(8)	+0.279	[+0.108, +0.463]	14/20	0.668	0.0076	0.0136	0.0953
Multi-AT	Held-out attack mean	+0.319	[+0.123, +0.527]	13/20	0.667	0.0076	0.0192	0.0962
Multi-AT	Held-out ℓ_∞ mean	+0.233	[+0.060, +0.410]	14/20	0.566	0.0204	0.0362	0.1449
Multi-AT	Held-out ℓ_2 mean	+0.405	[+0.144, +0.661]	16/20	0.664	0.0079	0.0136	0.0953
Multi-AT	Seen-source mean	+0.159	[+0.038, +0.285]	15/20	0.546	0.0246	0.0419	0.1449
Multi-AT	Worst(8)	-0.003	[-0.168, +0.155]	9/20	-0.008	0.9714	0.9839	0.9839
AttackDRO++ vs. PGD-AT								
PGD-AT	Clean	+4.015	[+3.666, +4.386]	20/20	4.779	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Mean(8)	+1.826	[+1.702, +1.965]	20/20	5.980	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out attack mean	+2.134	[+1.980, +2.303]	20/20	5.650	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out ℓ_∞ mean	-1.366	[-1.496, -1.223]	0/20	-4.273	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Held-out ℓ_2 mean	+5.633	[+5.441, +5.849]	20/20	11.638	< 0.0001	< 0.0001	< 0.0001
PGD-AT	Seen-source mean	+0.901	[+0.815, +0.984]	20/20	4.585	< 0.0001	< 0.0001	0.0002
PGD-AT	Worst(8)	-4.576	[-4.708, -4.454]	0/20	-15.364	< 0.0001	< 0.0001	0.0002
AttackDRO++ vs. DDN-AT								
DDN-AT	Clean	-4.563	[-4.712, -4.414]	0/20	-13.134	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Mean(8)	+6.098	[+5.986, +6.209]	20/20	23.087	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out attack mean	+4.774	[+4.654, +4.893]	20/20	16.910	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out ℓ_∞ mean	+9.555	[+9.399, +9.718]	20/20	25.440	< 0.0001	< 0.0001	< 0.0001
DDN-AT	Held-out ℓ_2 mean	-0.008	[-0.143, +0.127]	10/20	-0.025	0.9137	0.9563	0.9563
DDN-AT	Seen-source mean	+10.072	[+9.938, +10.208]	20/20	31.723	< 0.0001	< 0.0001	0.0002
DDN-AT	Worst(8)	+19.122	[+18.904, +19.344]	20/20	36.909	< 0.0001	< 0.0001	< 0.0001

The paired t-test table establishes the main aggregate deltas, and Table 6.4 adds the more conservative protocol from Chapter 5: Wilcoxon signed-rank tests with Holm correction, bootstrap confidence intervals, paired effect sizes, and seed-level win counts. The multiple-comparison-corrected tests provide a conservative check on the paired t-test results. Against Multi-AT, the key question is whether the small positive gains on average-case metrics remain supported after Wilcoxon testing and Holm correction. Clean accuracy is included as a sanity metric, but the main robustness claims are based on Mean(8),

held-out metrics, seen-source mean, and Worst(8). Against the single-attack baselines, the same analysis mainly verifies the specialist-versus-balanced pattern: PGD-AT remains stronger on ℓ_∞-heavy metrics, DDN-AT remains competitive on held-out ℓ_2, and AttackDRO++ provides stronger average cross-attack balance.

After Holm correction, the Multi-AT comparison should be interpreted cautiously. AttackDRO++ shows small positive differences over Multi-AT on Clean, Mean(8), held-out attack mean, held-out ℓ_∞ mean, held-out ℓ_2 mean, and seen-source mean, and these differences are supported by the paired t-tests, raw Wilcoxon tests, confidence intervals, and moderate paired effect sizes. However, the Holm-corrected Wilcoxon p-values for these metrics do not remain below the 0.05 threshold. Therefore, relative to Multi-AT, the evidence is best described as a small positive trend rather than a uniformly correction-robust improvement. Worst(8) is essentially unchanged, indicating that AttackDRO++ does not improve the worst-case attack performance over Multi-AT, but also does not meaningfully degrade it in this panel.

For the single-attack baselines, the corrected tests support the expected specialist-versus-balanced interpretation. Compared with PGD-AT, AttackDRO++ achieves correction-robust gains on Mean(8), held-out attack mean, held-out ℓ_2 mean, and seen-source mean, but remains weaker on the held-out ℓ_∞ mean and Worst(8), consistent with PGD-AT's specialization toward ℓ_∞ robustness. Compared with DDN-AT, AttackDRO++ obtains correction-robust gains on Mean(8), held-out attack mean, held-out ℓ_∞ mean, seen-source mean, and Worst(8), while the held-out ℓ_2 mean is statistically indistinguishable. Overall, Table 6.4 suggests that AttackDRO++ mainly improves average cross-attack balance and held-out robustness, while its worst-case behavior depends on the baseline being compared.

6.1.2 Per-Attack Accuracy: Where Do Methods Diverge?

The aggregate metrics show the overall trend, but they do not reveal whether the gain is broad or concentrated in one threat family. Table 6.5 breaks the paired comparison down by attack, with the main Multi-AT comparison followed by the two single-attack specialist comparisons.

Table 6.5: Per-attack robust accuracy (%) for all four methods across 20 paired random-seed runs. Bold values indicate the best method for each attack. The right-hand Δ and p columns report the paired comparison of AttackDRO++ against Multi-AT. AttackDRO++ is highest on 3 of 8 attacks; the strongest method varies by attack family. Δ values use the bold-marker convention * for $p < 0.05$ and ** for $p < 0.01$.

Attack	Family	PGD-AT	DDN-AT	Multi-AT	AttackDRO++	Δ vs. Multi-AT	p
FGSM-RS	held-out ℓ_∞	67.896	64.834	68.483	**68.939**	$+0.456^{**}$	0.0034
PGD-ℓ_∞	seen ℓ_∞	**49.656**	25.957	45.082	45.079	-0.003	0.971
TPGD	held-out ℓ_∞	**68.388**	59.310	66.934	67.043	$+0.109$	0.513
MI-FGSM	held-out ℓ_∞	**51.260**	30.638	47.331	47.466	$+0.135$	0.116
PGD-ℓ_2	held-out ℓ_2	64.319	**69.581**	69.197	69.571	$+0.374^{*}$	0.0120
DDN-ℓ_2	seen ℓ_2	60.728	66.083	66.785	**67.107**	$+0.321^{*}$	0.0136
DeepFool-ℓ_2	held-out ℓ_2	62.533	**68.290**	67.301	67.805	$+0.504^{**}$	0.0082
CW-ℓ_2	held-out ℓ_2	59.211	65.113	65.249	**65.585**	$+0.336^{**}$	0.0097

The attack-level breakdown in Table 6.5 clarifies why the Mean(8) gain does not automatically imply a stronger Worst(8). Against Multi-AT, AttackDRO++ improves most per-attack metrics, especially on held-out ℓ_2 attacks. Across all compared methods, however, the best method still depends on the attack family: PGD-AT remains strongest on several ℓ_∞-oriented attacks, while DDN-AT remains strongest on some ℓ_2-oriented attacks. AttackDRO++ is therefore best understood as improving average robustness balance rather than being best on every individual attack. The per-attack pattern explains why the Mean(8) gain does not translate into a stronger Worst(8): the improvement is concentrated on held-out attacks, while the weakest evaluation direction remains broadly unchanged.

6.1.3 Robustness Under the Hardest Evaluation Cases

This subsection examines two complementary stress indicators: the lowest robust accuracy across the eight main evaluation attacks and the separate AutoAttack-ℓ_∞ evaluation.

Two metrics test standardized worst-case behavior. Worst(8) is the minimum robust accuracy across the eight evaluation attacks per checkpoint; AutoAttack-ℓ_∞ is an ensemble ℓ_∞ stress test following Croce and Hein [5]. Tables 6.6 and 6.7 report each metric separately because they have different evaluation panels: Worst(8) is computed across the full $n = 20$ seed panel, while AutoAttack is reported in two layers: a 512-sample sanity evaluation across all $n = 20$ seeds and a full-test evaluation on the selected $n = 5$ seed panel. AutoAttack is therefore a separate stress check, not part of Mean(8) or the main

aggregate ranking.

Table 6.6: Worst(8) and AutoAttack-512 sanity results across 20 paired random-seed runs. Worst(8) is computed over the eight non-AutoAttack evaluation attacks. AutoAttack-512 denotes AutoAttack-ℓ_∞ evaluated on 512 test samples as a sanity check and is reported separately from Mean(8).

	Worst(8)		AutoAttack-512	
Method	Mean (%)	σ (pp)	Mean (%)	σ (pp)
PGD-AT	**49.656**	0.161	**44.912**	1.136
DDN-AT	25.957	0.378	23.809	0.796
Multi-AT	45.082	0.281	41.318	1.167
AttackDRO++	45.079	0.237	41.317	1.050

Table 6.7: Full-test AutoAttack-ℓ_∞ results on the selected $n = 5$ paired seed panel. The paired difference is AttackDRO++ $-$ Multi-AT: -0.006 pp, with $p = 0.9584$.

Method	Mean (%)	σ (pp)
Multi-AT	42.47	0.30
AttackDRO++	42.46	0.20

AttackDRO++ vs. Multi-AT. AttackDRO++ is statistically indistinguishable from Multi-AT on both hardest-case metrics: the paired Worst(8) difference is -0.003 pp ($p = 0.971$) on the $n = 20$ panel, the AutoAttack-512 sanity check is -0.001 pp ($p = 0.998$), and the full-test AutoAttack-ℓ_∞ evaluation on the selected five-seed panel is -0.006 pp ($p = 0.9584$). This means the average-case gains observed above should not be reinterpreted as a hardest-case or AutoAttack improvement. The remaining question is whether the average-case behavior is more predictable across repeated seeds.

PGD-AT specialization. PGD-AT obtains the highest Worst(8) and AutoAttack because both metrics are largely determined by ℓ_∞ robustness, the threat family it trains against. This advantage is therefore structural rather than evidence of broader robustness. As Table 6.5 shows, PGD-AT loses 5 to 6 pp to AttackDRO++ on held-out ℓ_2 attacks, while DDN-AT shows the opposite failure mode by collapsing under ℓ_∞ stress. Single-attack training therefore produces specialists; AttackDRO++ does not match PGD-AT's ℓ_∞ depth, but provides the best average cross-attack balance among the evaluated methods.

6.1.4 Training Stability: How Predictable Is the Outcome?

AttackDRO++ runs in this panel exhibit observed lower seed-to-seed standard deviation than Multi-AT on average-case metrics. To describe whether the observed outcomes are more predictable across repeated runs, Table 6.8 compares the standard deviation of each seed-level metric between the two methods.

Table 6.8: Observed seed-to-seed standard deviation across 20 paired random-seed runs. σ is the standard deviation of the per-seed metric. AttackDRO++ has lower standard deviation than Multi-AT on Mean(8), held-out averages, and held-out norm-specific metrics, while Worst(8) and AutoAttack standard deviations remain similar across methods.

Metric	σ Multi-AT (pp)	σ AttackDRO++ (pp)	Ratio
Mean(8)	0.447	0.148	3.03×
Held-out attack mean	0.505	0.155	3.26×
Held-out ℓ_2 mean	0.603	0.223	2.70×
Held-out ℓ_∞ mean	0.427	0.170	2.51×
Seen-source mean	0.303	0.166	1.83×
Worst(8)	0.281	0.237	1.18×
AutoAttack-ℓ_∞	1.167	1.050	1.11×

The difference is strongest for held-out robustness. The standard deviation decreases by $3.26\times$ for the held-out attack mean, $2.70\times$ for the held-out ℓ_2 mean, and $2.51\times$ for the held-out ℓ_∞ mean. Mean(8) also shows observed lower seed-to-seed standard deviation, decreasing from 0.447 pp under Multi-AT to 0.148 pp under AttackDRO++, a $3.03\times$ ratio.

This suggests a more predictable observed outcome on average-case metrics, but the comparison does not isolate which component drives the lower variation. The difference is weaker for Worst(8) and AutoAttack, whose standard deviations remain similar across methods. This is consistent with the previous sections: the hardest ℓ_∞ stress metrics are broadly unchanged relative to Multi-AT, while the main benefit of AttackDRO++ appears on average and held-out robustness.

The whitebox results give the main aggregate evidence: AttackDRO++ improves average and held-out robustness relative to Multi-AT while leaving the hardest standardized checks broadly unchanged. However, aggregate and per-attack metrics can still hide which CIFAR-10 classes and attack-by-class cells drive the remaining failures. The next section therefore drills down into class-wise whitebox robustness to test whether the ob-

served gains are broad or concentrated in a small part of the evaluation grid.

6.2 Class-wise Whitebox Robustness Across Attacks

The aggregate results in Section 6.1 show a modest average gain for AttackDRO++ over Multi-AT while leaving the worst-case profile broadly similar. However, aggregate accuracy can hide localized failure modes: a method may improve the mean by increasing already-easy cells, while leaving hard class-by-attack combinations unchanged. To test whether the improvement is distributed across the evaluation grid, we perform a whitebox per-(attack $\times$ class) drilldown on the eight non-AutoAttack attacks. Each cell therefore corresponds to the robust accuracy of one method on one attack and one class, averaged over five seeds.

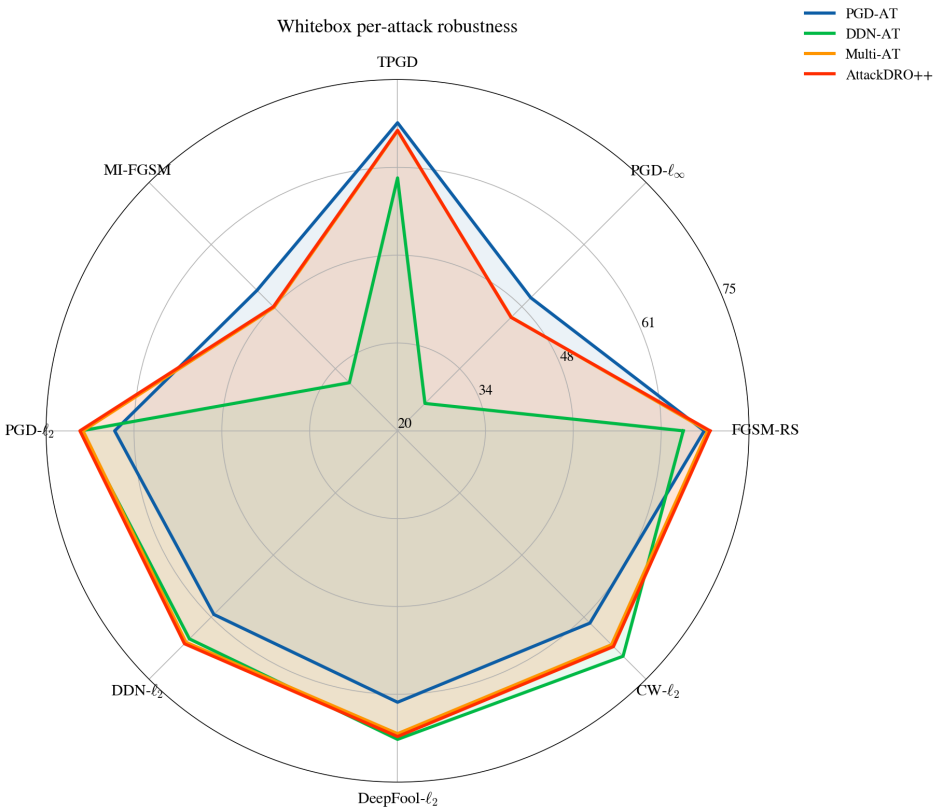


Figure 6.1: Whitebox per-attack robust accuracy (%) averaged over five seeds. The radar plot compares PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ across the eight non-AutoAttack evaluation attacks. The curve shapes summarize the specialist profile of the single-attack baselines and the more balanced profile of the multi-attack methods.

This analysis has two purposes. First, it identifies whether AttackDRO++ behaves as a balanced multi-attack method or merely shifts robustness from one threat family to another. Second, it exposes the tail cells that drive practical failure behavior, especially under hard attacks such as PGD-ℓ_∞ and MI-FGSM on visually ambiguous classes. We

compare AttackDRO++ against three baselines: the uniform multi-attack model (Multi-AT), the PGD-ℓ_∞ specialist (PGD-AT), and the DDN-ℓ_2 specialist (DDN-AT).

Absolute per-class heatmaps with a shared scale. Before analyzing deltas, we first report the raw per-(class $\times$ attack) robust accuracy of each method. All four heatmaps use the same global color scale from 0% to 100%. This shared scale is important because it makes the maps visually comparable across methods: a darker cell always represents the same absolute robust accuracy, regardless of which method is being shown.

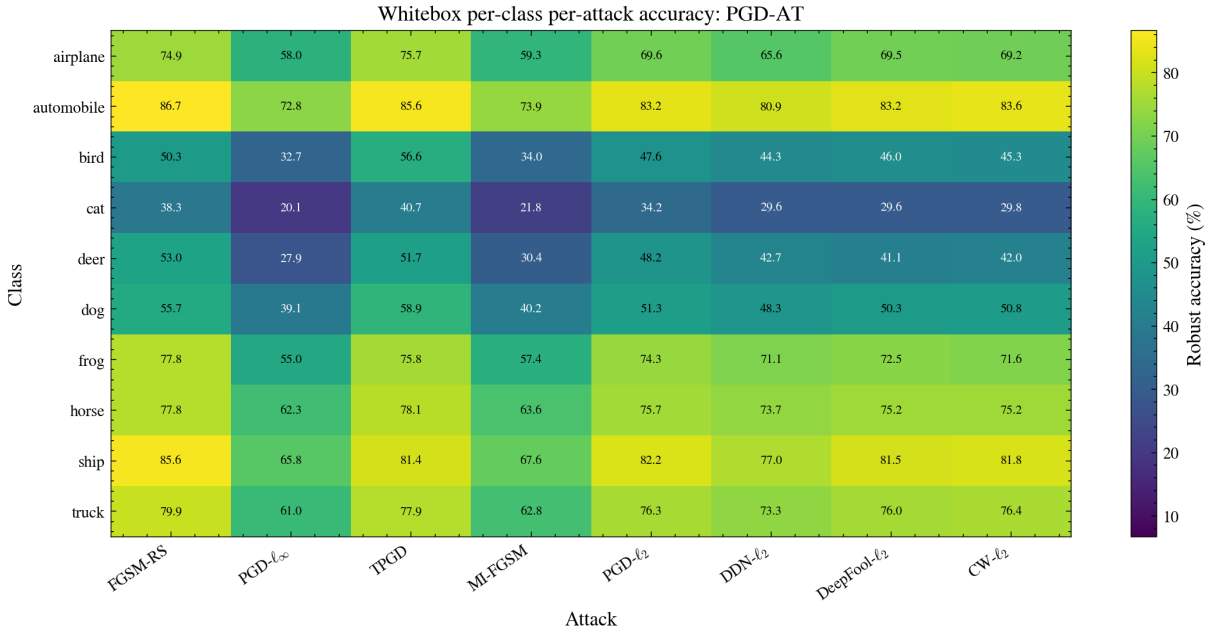


Figure 6.2: Absolute whitebox per-(class $\times$ attack) robust accuracy for PGD-AT, averaged over five seeds. Cells report robust accuracy (%).

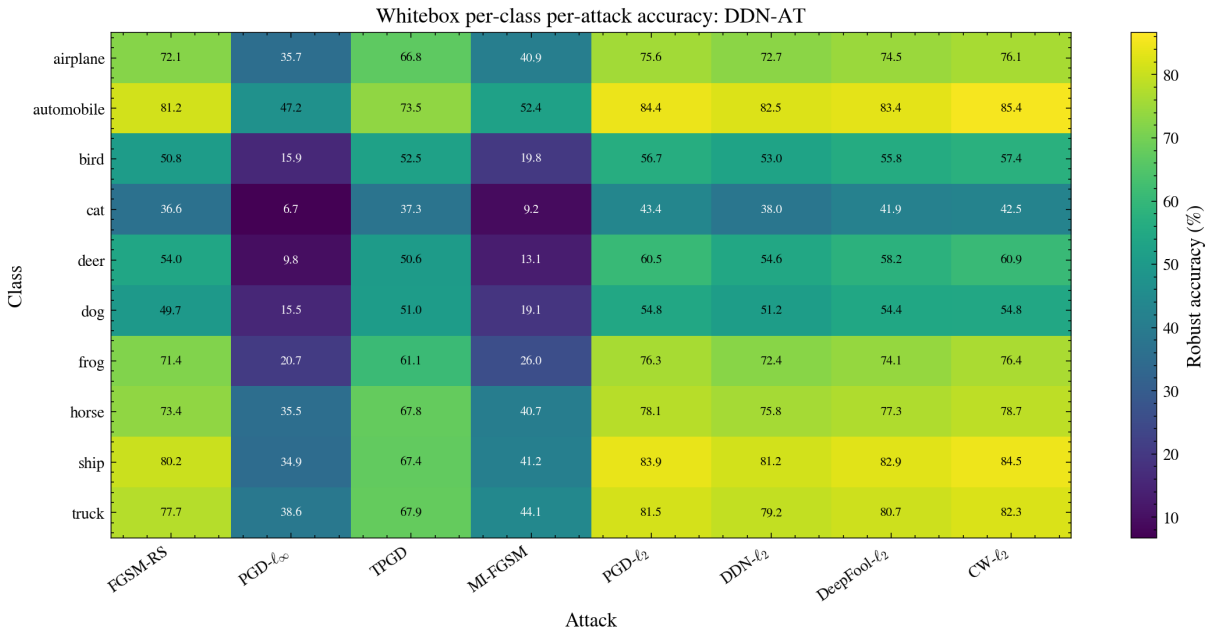


Figure 6.3: Absolute whitebox per-(class × attack) robust accuracy for DDN-AT, averaged over five seeds. Cells report robust accuracy (%).

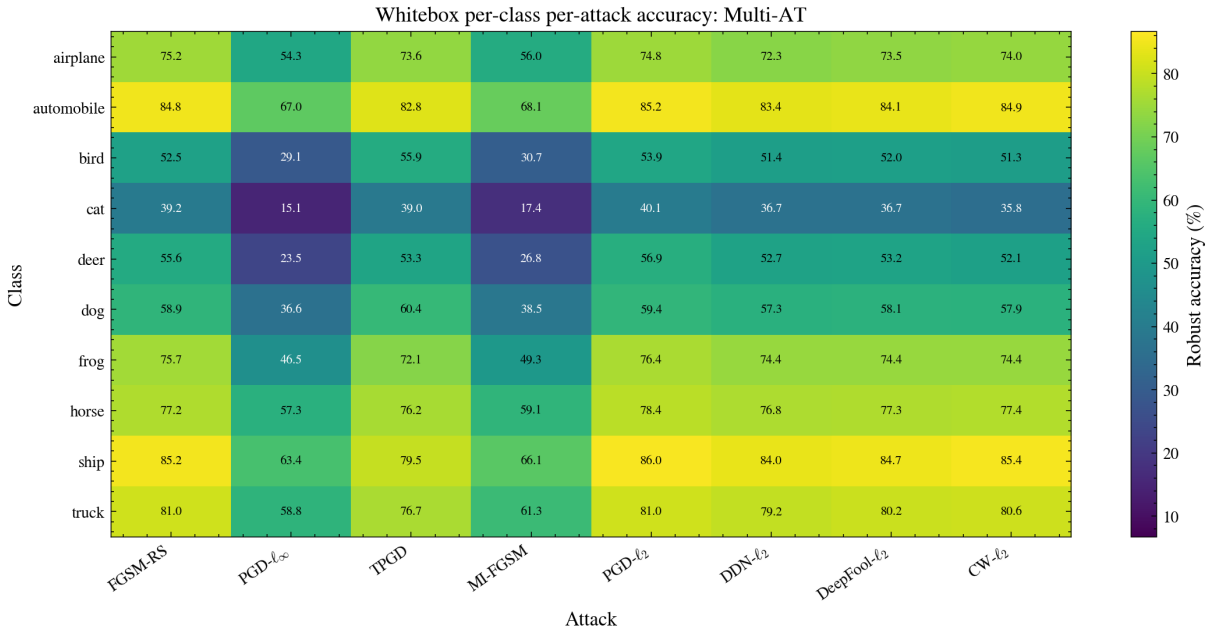


Figure 6.4: Absolute whitebox per-(class × attack) robust accuracy for Multi-AT, averaged over five seeds. Cells report robust accuracy (%).

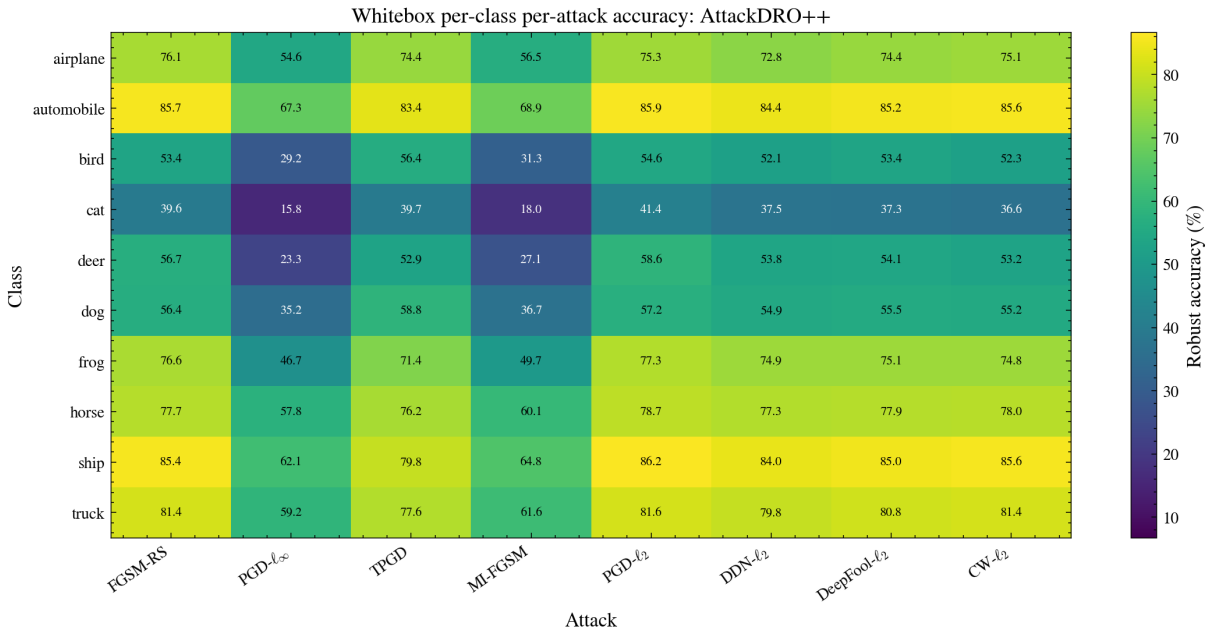


Figure 6.5: Absolute whitebox per-(class × attack) robust accuracy for AttackDRO++, averaged over five seeds. Cells report robust accuracy (%).

PGD-AT is stronger on several ℓ_∞ cells, while DDN-AT is more biased toward bounded ℓ_2 attacks and visibly weaker on PGD-ℓ_∞ and MI-FGSM. Compared with the specialists, Multi-AT and AttackDRO++ show a more balanced cross-attack profile. AttackDRO++ largely preserves the Multi-AT structure while slightly lifting many cells, especially outside the hardest ℓ_∞ animal-class tail. The next step is to quantify whether those visual differences are widespread or driven by a few large cells.

6.2.1 Class-Level Gains and Persistent Difficulties

Across all methods, the per-class robustness pattern is highly non-uniform. Classes such as *automobile*, *ship*, and *truck* tend to remain easier under most attacks, while *cat*, *deer*, *bird*, and *dog* form the recurring hard tail. This pattern is consistent with the visual similarity and intra-class variation of CIFAR-10: animal classes often share local textures and backgrounds, so small adversarial perturbations can more easily move samples across nearby decision regions.

The hardest whitebox cells are concentrated in the ℓ_∞ iterative attacks. For AttackDRO++, the lowest cells are PGD-ℓ_∞ on *cat* and *deer*, followed by MI-FGSM on the same classes. This indicates that, even though AttackDRO++ improves the average multi-attack profile, the most difficult failure modes are still driven by strong ℓ_∞ optimization on semantically similar animal classes. In contrast, bounded ℓ_2 attacks such as PGD-ℓ_2, DDN-ℓ_2, DeepFool-ℓ_2, and CW-ℓ_2 are generally less damaging for the proposed model than the strongest ℓ_∞ cells.

Attack-Class Delta Against Multi-AT Baseline Compared with Multi-AT, AttackDRO++ produces a broad but modest shift rather than a large change concentrated in a few cells. The mean per-cell gain is only $+0.281$ percentage points, but 67 out of 80 attack-by-class cells are positive.

Figure 6.6: Whitebox per-(class $\times$ attack) delta of AttackDRO++ relative to Multi-AT. Each cell reports AttackDRO++ minus Multi-AT in percentage points, averaged over five seeds.

Attack-Class Delta Against Single-AT Baselines The per-cell delta summary in Table 6.9 is most important for the Multi-AT comparison because both methods train on the same source attacks and differ mainly in the group structure used for optimization. AttackDRO++ improves 67/80 cells over Multi-AT, with a mean gain of $+0.281$ percentage points and a bottom-10 lift of $+0.282$ percentage points. The gains are small, but they are widespread, which supports the interpretation that cluster-based DRO acts as a modest refinement over uniform multi-attack training. Because the per-cell panel uses five seeds, the emphasis here is on distributional pattern rather than strong cell-level significance.

Table 6.9: Per-(class $\times$ attack) whitebox comparison of AttackDRO++ against each baseline. Δ is computed as AttackDRO++ minus the baseline. Positive cells count the number of attack-by-class cells where $\Delta > 0$ out of 80 total cells.

Baseline	Positive cells ($\Delta > 0$)	Mean Δ (pp)	Significant cells AttackDRO++ / Baseline	Bottom-10 lift (pp)	Main pattern
Multi-AT	67/80	$+0.281$	0 / 2	$+0.282$	Balanced refinement
PGD-AT	47/80	$+1.782$	40 / 19	$+0.726$	Gains mainly on ℓ_2 cells
DDN-AT	62/80	$+5.781$	44 / 5	$+15.710$	Gains mainly on ℓ_∞ cells

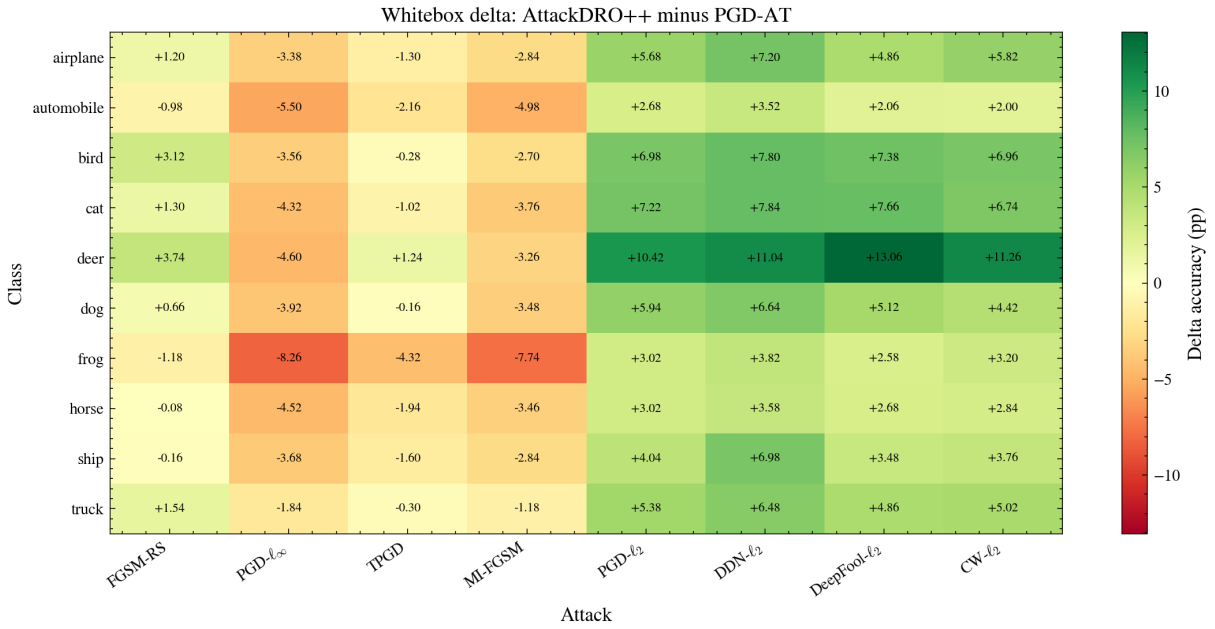


Figure 6.7: Whitebox per-(class $\times$ attack) delta of AttackDRO++ relative to PGD-AT, averaged over five seeds. Cells report AttackDRO++ minus PGD-AT in percentage points; positive values indicate where AttackDRO++ improves over the PGD-ℓ_∞ specialist.

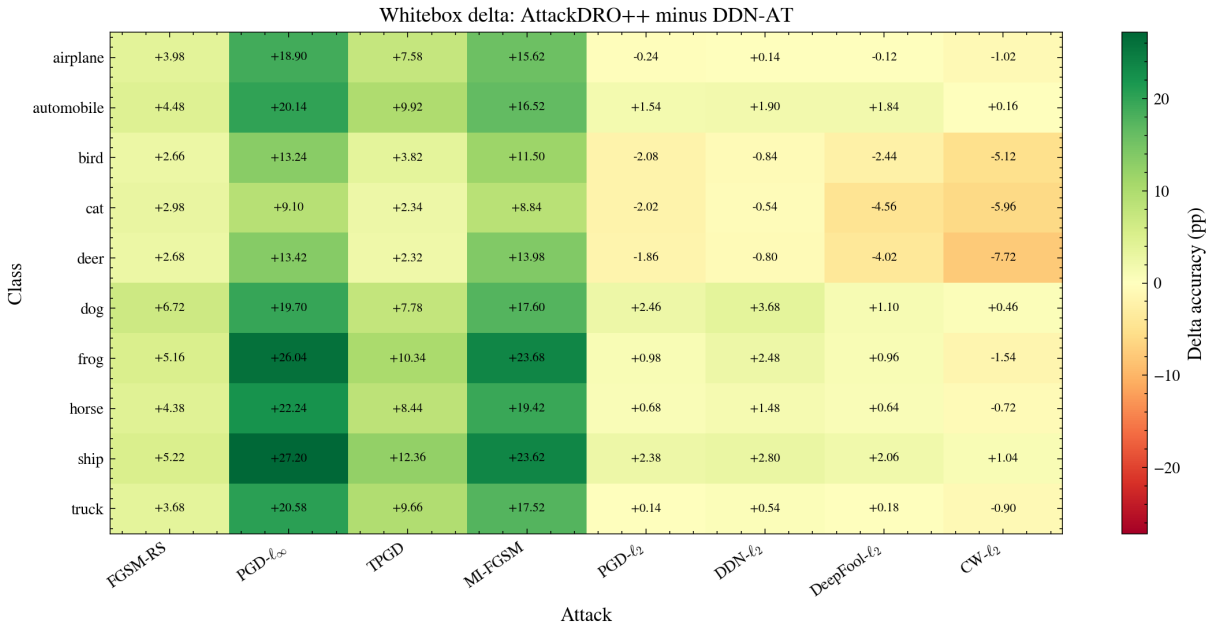


Figure 6.8: Whitebox per-(class $\times$ attack) delta of AttackDRO++ relative to DDN-AT, averaged over five seeds. Cells report AttackDRO++ minus DDN-AT in percentage points; positive values indicate where AttackDRO++ improves over the DDN-ℓ_2 specialist.

The comparison against PGD-AT shows the expected specialist trade-off. AttackDRO++ is positive in 47/80 cells and has a mean gain of $+1.782$ percentage points, but the gains and losses are not uniformly distributed. The largest improvements occur on bounded ℓ_2 attacks on hard animal classes such as *deer*, *cat*, and *bird*; for example, AttackDRO++ gains $+13.06$ pp over PGD-AT on DeepFool-ℓ_2 for *deer*. The heatmap in Figure 6.7 makes this distribution visible across all class-by-attack cells. Conversely, PGD-AT remains stronger on several ℓ_∞ cells, particularly PGD-ℓ_∞ and MI-FGSM for *frog*, *automobile*, *horse*, and *cat*. This confirms that PGD-AT is not weak in absolute terms; rather, it is specialized toward the ℓ_∞ geometry used during training.

The comparison against DDN-AT is even more asymmetric. AttackDRO++ improves 62/80 cells and gains $+5.781$ percentage points on average, with a large $+15.710$ percentage-point improvement on the bottom-10 cells. The largest improvements occur under ℓ_∞ attacks: PGD-ℓ_∞ on *ship* ($+27.20$ pp), PGD-ℓ_∞ on *frog* ($+26.04$ pp), MI-FGSM on *frog* ($+23.68$ pp), and MI-FGSM on *ship* ($+23.62$ pp). These large deltas show that DDN-AT develops a strong ℓ_2 bias and leaves many ℓ_∞ attack-by-class cells under-protected. At the same time, DDN-AT remains stronger on some ℓ_2 boundary/optimization cells, especially CW-ℓ_2 and DeepFool-ℓ_2 for *deer*, *cat*, and *bird*. Thus, AttackDRO++ improves cross-attack balance but is not uniformly best on every specialist-preferred cell.

Attack-family interpretation. The attack-wise deltas make the specialization effect clear. Relative to PGD-AT, AttackDRO++ loses on PGD-ℓ_∞ and MI-FGSM on average, but gains strongly on the bounded ℓ_2 attacks. Relative to DDN-AT, the pattern reverses: AttackDRO++ gains substantially on PGD-ℓ_∞, MI-FGSM, TPGD, and FGSM-RS, while being slightly weaker on CW-ℓ_2 and DeepFool-ℓ_2. The Multi-AT comparison is more balanced: most attack averages are positive, but the magnitudes are below one percentage point.

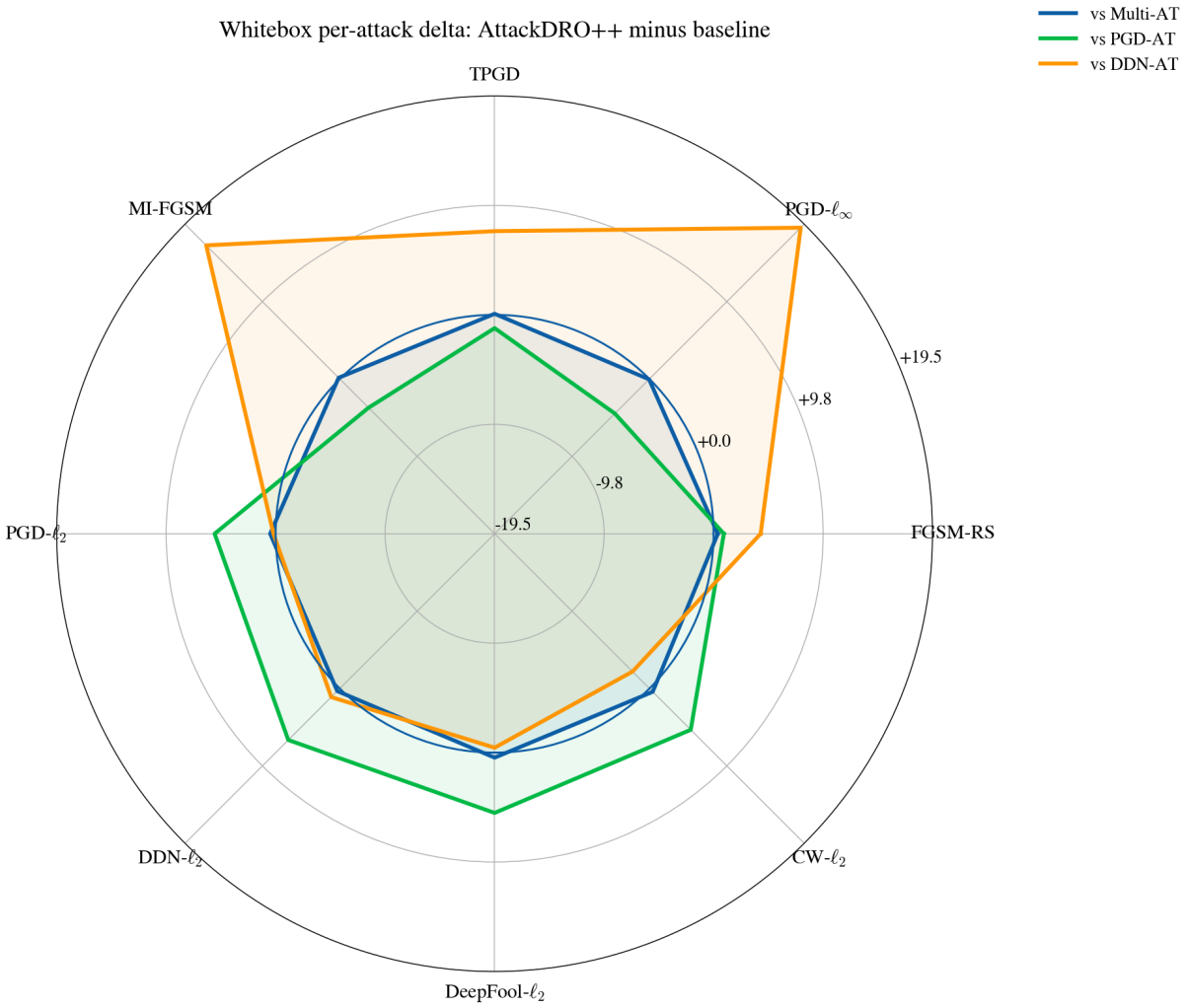


Figure 6.9: Attack-wise whitebox delta of AttackDRO++ relative to the three baselines, averaged over five seeds. Values are AttackDRO++ minus the baseline in percentage points.

This result supports the whitebox interpretation from Section 6.1. AttackDRO++ should not be interpreted as a replacement for a single specialist when the deployment threat model is known exactly. Instead, it is a cross-attack robustness method: it sacrifices some of the peak strength of a specialist in exchange for a more even robustness profile across attacks and classes.

6.2.2 Tail-Class Robustness: Do the Weakest Cells Improve?

We next examine the weakest attack-by-class cells for each baseline, using $K \in \{5, 10, 20\}$. These cells capture localized failure modes that can be hidden by mean accuracy. For Multi-AT, the weakest cells are mainly strong ℓ_∞ attacks, especially PGD-ℓ_∞ and MI-FGSM, on hard animal classes such as *cat*, *deer*, and *bird*. AttackDRO++ improves this tail only slightly: the bottom-10 lift is $+0.282$ pp. Although small, this shows that the average gain over Multi-AT is not obtained by sacrificing its weakest cells.

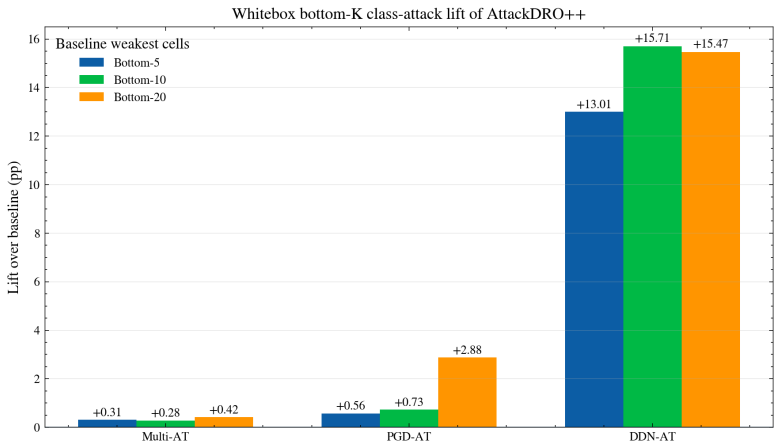


Figure 6.10: Weakest-K whitebox class-by-attack lift of AttackDRO++ over each baseline. For each baseline, the weakest-K cells are the lowest-accuracy class-by-attack cells of that baseline; bars report AttackDRO++ minus the baseline on those same cells in percentage points, averaged over five seeds.

The tail comparison is most pronounced against DDN-AT. Its weakest cells collapse under ℓ_∞-oriented attacks, with the bottom-10 cells averaging only 15.602%. AttackDRO++ raises these same cells by $+15.710$ pp, explaining why it achieves a much more balanced whitebox profile than the ℓ_2 specialist, even though DDN-AT remains competitive on several ℓ_2 cells.

Against PGD-AT, the lift is smaller but still positive: $+0.726$ pp on the bottom-10 cells. PGD-AT's weakest cells include several bounded ℓ_2 attacks on difficult classes such as *cat* and *deer*; AttackDRO++ reduces many of these failures, consistent with the idea that multi-attack training plus the Cluster-DRO component can mitigate part of the cross-norm weakness of PGD-only training.

Overall, the weakest-cell analysis supports the aggregate conclusion from Section 6.1: AttackDRO++ gives a broad but modest refinement over Multi-AT, substantially corrects the ℓ_∞ tail weakness of DDN-AT, and reduces the ℓ_2 tail weakness of PGD-AT. The

remaining limitation is that all attacks in this section are generated directly against the target checkpoint.

The class-wise whitebox analysis shows where the aggregate gains appear and which hard cells remain. It still evaluates attacks generated directly against the target checkpoint, so it does not answer whether the same robustness pattern holds when adversarial examples are crafted on a different model. The next section therefore moves to the graybox transfer protocol from Section 5.8.

6.3 Graybox Robustness Across Surrogate Models

The whitebox results in Section 6.1 show that AttackDRO++ improves average and held-out robustness over Multi-AT on $n = 20$ paired seeds. We next test whether this gain also holds under cross-model transfer, using the graybox protocol of Section 5.8. In this setting, adversarial examples are generated on one checkpoint and evaluated on a separately trained target checkpoint.

Random-seed checkpoint panel and whitebox sanity check. The graybox analysis uses 20 random-seed checkpoints to define the surrogate-to-target evaluation panel. These checkpoints come from four method families evaluated across five random seeds each and are fixed before graybox transfer is analyzed. The diagonal sanity check in Table 6.10 compares this random-seed checkpoint panel against the full whitebox panel to verify that the main method ordering is preserved.

The diagonal entries of the graybox table correspond to the case where the surrogate and target are the same checkpoint. We use these entries as a whitebox sanity check. They need not match the main whitebox evaluation exactly, because the graybox cache uses a balanced 125-image-per-class-per-attack split instead of the full test set for each attack. However, they should preserve the same method ordering and remain close in absolute value.

Table 6.10: Whitebox sanity check from the graybox pipeline. The five-random-seed columns use diagonal graybox entries, where surrogate and target are the same checkpoint. The 20-random-seed columns report the main whitebox results from Table 6.1. Bold values indicate the best method for each metric.

Target method	Mean(8) $n = 5$	Worst(8) $n = 5$	Mean(8) $n = 20$	Worst(8) $n = 20$
PGD-AT	61.524	**50.384**	60.499	**49.656**
DDN-AT	56.922	25.568	56.226	25.957
Multi-AT	62.458	45.584	62.045	45.082
AttackDRO++	**62.642**	45.088	**62.324**	45.079

The Mean(8) ordering is identical on both panels (AttackDRO++ $>$ Multi-AT $>$ PGD-AT $>$ DDN-AT), and absolute values agree to within 1.0 pp. More importantly, the paired Mean(8) delta of AttackDRO++ minus Multi-AT is $+0.184$ pp on the $n = 5$ graybox panel and $+0.279$ pp on the full $n = 20$ panel: both are positive and of the same order of magnitude. The graybox cache is therefore consistent with the main evaluation protocol. Because this is a smaller five-seed panel, the next subsections use it to test whether the whitebox trend has a corresponding transfer pattern rather than to replace the main aggregate evidence.

6.3.1 Cross-Method Transfer Structure

We aggregate the per-attack per-class transfer table into a method-level 4×4 transfer matrix. Rows index the target method being evaluated, and columns index the surrogate method that generates adversarial examples. Each cell reports the target robust accuracy averaged over all matching target-surrogate seed pairs and the eight evaluation attacks. Higher values indicate that the row target is more robust to adversarial examples crafted on the column surrogate.

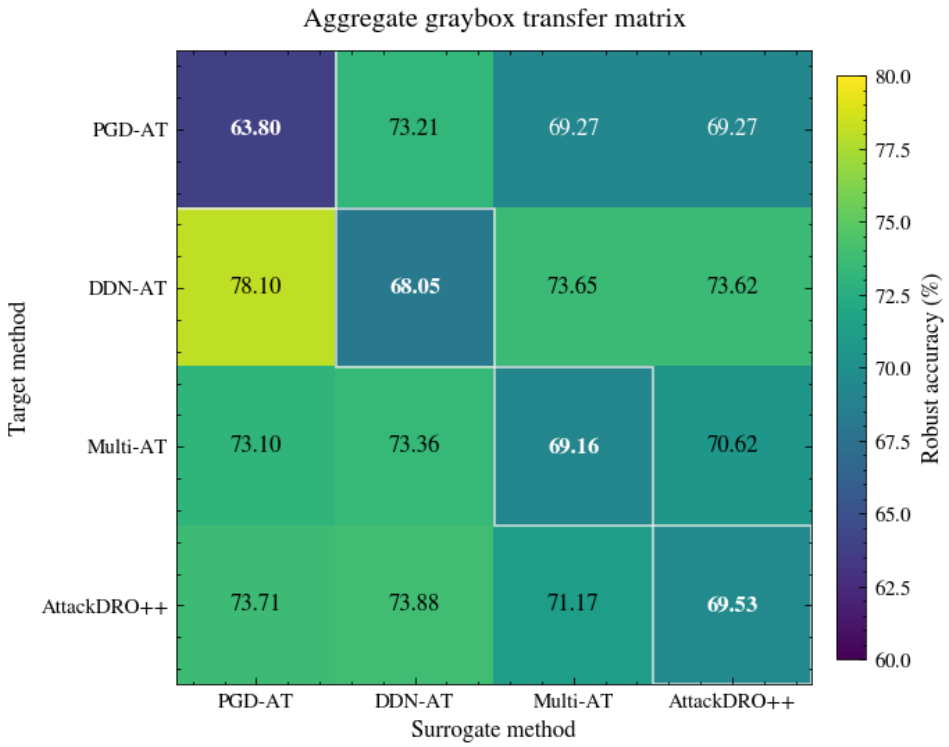
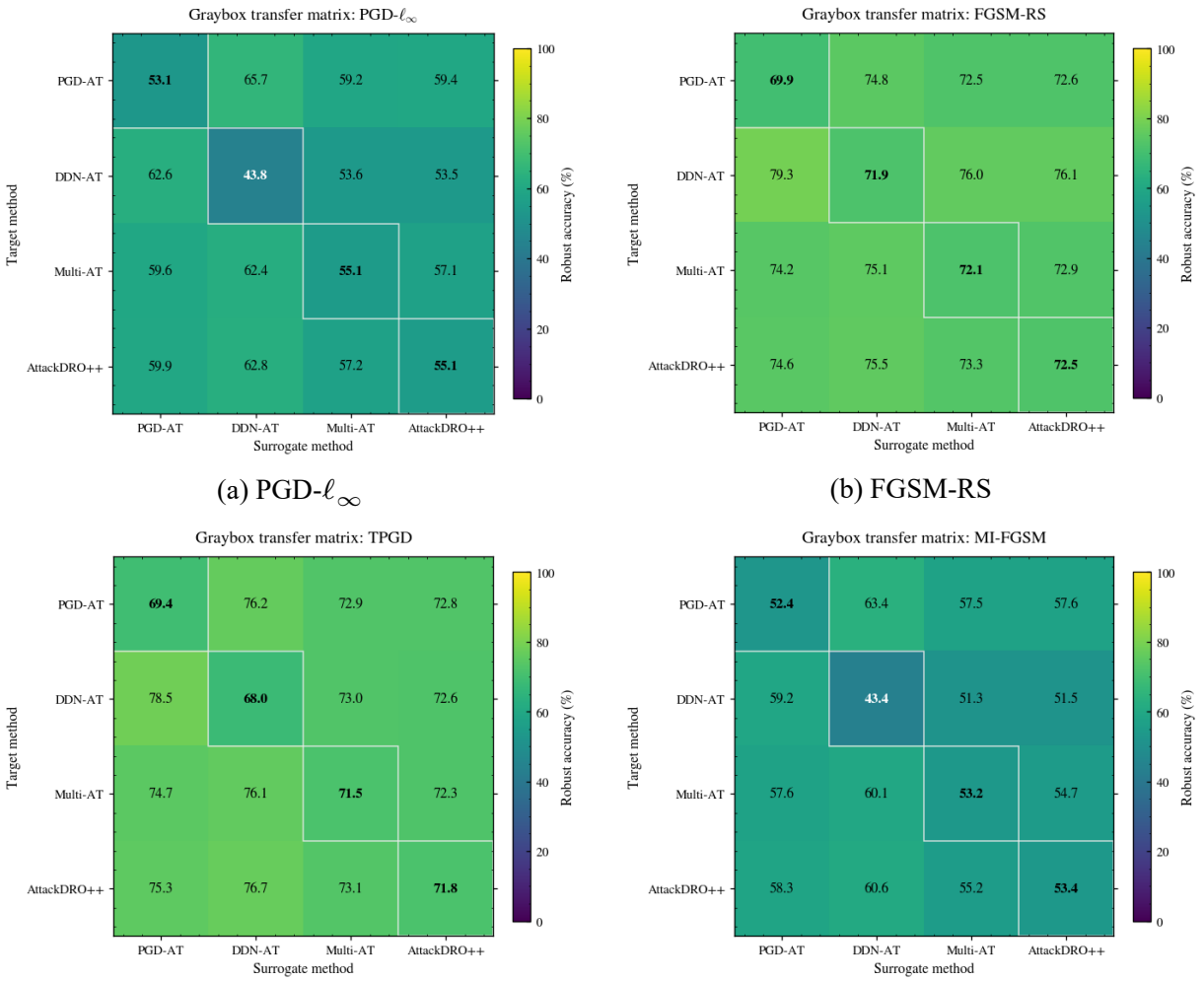


Figure 6.11: Method-level graybox transfer matrix on the $n = 5$ panel. Rows are target methods and columns are surrogate methods. Cell values are robust accuracy (%) averaged over seeds and the eight evaluation attacks.

Two structural patterns are visible. First, rows ranked by mean off-diagonal accuracy give DDN-AT (75.12%) $>$ AttackDRO++ (72.92%) $>$ Multi-AT (72.36%) $>$ PGD-AT (70.59%). The DDN-AT row is highest not because DDN-AT is the strongest defense (its whitebox Mean(8) is the lowest of the four), but because ℓ_∞ adversarial examples generated by other surrogates transfer poorly to a DDN-AT target whose decision boundary is shaped by ℓ_2 training. The gap analysis below quantifies this structural inflation directly. Second, the AttackDRO++ row is consistently above the Multi-AT row for every surrogate, with cross-method gaps ranging from $+0.17$ pp (DDN-AT surrogate) to $+0.61$ pp (PGD-AT surrogate). The whitebox advantage of AttackDRO++ over Multi-AT therefore appears in the same direction when adversarial examples are crafted on separately trained surrogates, but the transfer setting still needs the paired-dependence caution below.

The aggregate transfer matrix hides important attack-family structure. Figures 6.12 and 6.13 therefore disaggregate the transfer behavior into the four ℓ_∞-oriented attacks and the four ℓ_2-oriented attacks. This grouping makes the specialist behavior more explicit.

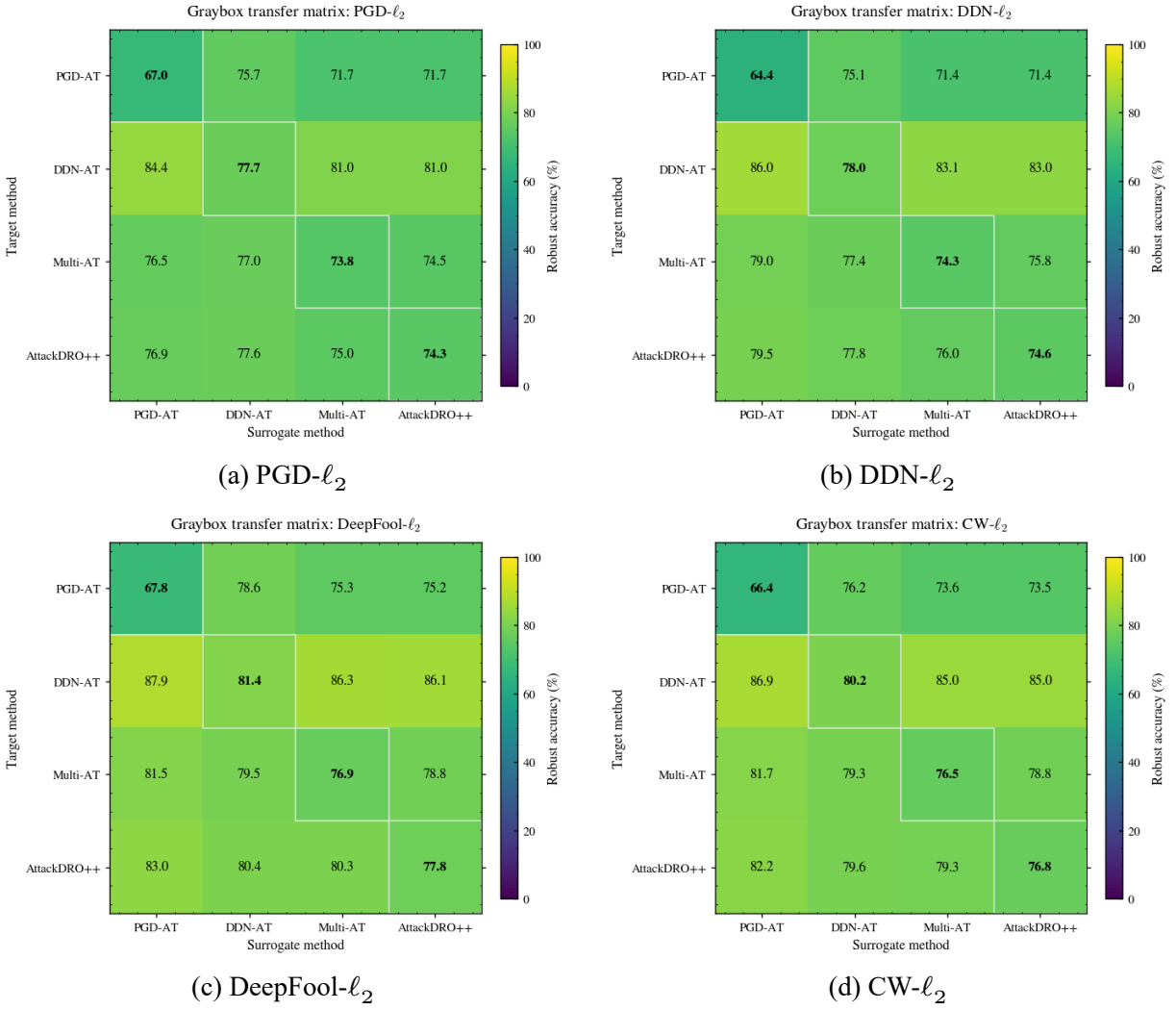


(a) PGD-ℓ_∞ (b) FGSM-RS

(c) TPGD (d) MI-FGSM

Figure 6.12: Per-attack graybox transfer matrices for the ℓ_∞-oriented attacks. Rows are target methods and columns are surrogate methods; cells report robust accuracy (%) averaged over target-surrogate seed pairs.

On the ℓ_∞ attacks, this advantage disappears: PGD-AT and AttackDRO++ are more competitive, and AttackDRO++ remains closer to the balanced Multi-AT profile. Thus, the per-attack matrices show that DDN-AT's strong graybox row average should be interpreted as family-specific transfer specialization rather than uniform robustness.



(a) PGD-ℓ_2 (b) DDN-ℓ_2

(c) DeepFool-ℓ_2 (d) CW-ℓ_2

Figure 6.13: Per-attack graybox transfer matrices for the ℓ_2-oriented attacks. Rows are target methods and columns are surrogate methods; cells report robust accuracy (%) averaged over target-surrogate seed pairs.

On the ℓ_2 attacks, the DDN-AT target row is consistently strong, especially when attacks are transferred from PGD-AT, Multi-AT, or AttackDRO++ surrogates. This indicates that DDN-AT's high aggregate graybox score is driven mainly by ℓ_2-family transfer behavior, rather than by uniformly strong robustness across both attack geometries. AttackDRO++ remains closer to the balanced Multi-AT profile, while still improving over Multi-AT on most ℓ_2-oriented transfer cells.

6.3.2 Paired Graybox Comparisons: Does the Whitebox Advantage Carry Over?

The main graybox comparison is between AttackDRO++ and Multi-AT. Both methods train on the same source attacks $\{\text{PGD-}\ell_\infty,\ \text{DDN-}\ell_2\}$ and use the same backbone and schedule; they differ only in the use of latent clustering, gradient fingerprints, and the anchored Cluster-DRO objective.

We use the symmetric paired test that has already been described in Section 5.8: for every triple $(\text{target seed},\ \text{surrogate seed},\ \text{attack})$, we pair the configuration "AttackDRO++ as target, Multi-AT as surrogate" against its mirror "Multi-AT as target, AttackDRO++ as surrogate" and compute the per-triple difference. This yields $n = 5 \times 5 \times 8 = 200$ paired observations and makes the comparison invariant to which method is chosen as the surrogate. Because these paired observations share target checkpoints, surrogate checkpoints, random seeds, and attack families, the per-cell tests should be interpreted as evidence of a consistent transfer trend rather than as standalone evidence from mutually independent training-run panels.

Table 6.11: Paired graybox comparisons. Each row uses $n = 200$ paired target-surrogate-attack triples. Positive Δ favors the reference method named in the panel heading.

Compared method	Accuracy		Δ	d_z	t	p
	Reference	Baseline	(pp)			
AttackDRO++ vs. baselines						
Multi-AT	71.17	70.62	+0.549	0.42	5.96	1.1×10^{-8}
PGD-AT	73.71	69.27	+4.435	1.33	18.78	6.1×10^{-46}
DDN-AT	73.88	73.62	+0.258	0.04	0.60	0.549
Multi-AT vs. single-attack baselines						
PGD-AT	73.10	69.27	+3.829	1.19	16.89	2.7×10^{-40}
DDN-AT	73.36	73.65	-0.292	-0.05	-0.67	0.504

The AttackDRO++ vs. Multi-AT contrast is the primary result of this section: under symmetric cross-method graybox transfer, AttackDRO++ improves on Multi-AT by $+0.549$ pp at $p \approx 1.1 \times 10^{-8}$, with a small-to-medium paired effect size of $d_z \approx 0.42$. This effect is roughly twice the corresponding whitebox delta of $+0.279$ pp on the same seed panel and has a very small p-value under the shared-checkpoint paired test. Read with the dependence caution above, this supports a consistent transfer trend rather than

a claim from a mutually independent training-run panel.

The single-attack contrasts confirm the expected specialist-versus-balanced pattern. AttackDRO++ improves on PGD-AT by $+4.44$ pp at $p < 10^{-45}$ ($d_z = 1.33$, a large effect), tracking the whitebox trend that PGD-AT loses heavily on the ℓ_2 family it does not train against. The AttackDRO++ vs. DDN-AT contrast is statistically indistinguishable from zero ($+0.26$ pp, $p = 0.55$); this reflects the DDN-AT graybox inflation discussed above, not actual parity. The graybox-minus-whitebox gap analysis below quantifies this asymmetry.

Per-attack transfer behavior. To check whether the $+0.549$ pp Mean(8) gain over Multi-AT is broad or concentrated, we report paired graybox deltas attack by attack on the same $n = 200$ pair set. Each attack contributes 25 paired triples (one per surrogate-target seed combination).

Table 6.12: Per-attack symmetric graybox comparison between AttackDRO++ and Multi-AT. Each attack uses $n = 25$ paired seed combinations. Δ is computed as AttackDRO++ minus Multi-AT in percentage points. Significance is reported in the p-value column.

Attack	Family	Ours mean (%)	Multi-AT mean (%)	Δ (pp)	p
FGSM-RS	held-out ℓ_∞	73.32	72.86	$+0.454$	0.0114
PGD-ℓ_∞	seen ℓ_∞	57.17	57.07	$+0.102$	0.307
TPGD	held-out ℓ_∞	73.09	72.30	$+0.790$	0.0240
MI-FGSM	held-out ℓ_∞	55.20	54.75	$+0.458$	0.0022
PGD-ℓ_2	held-out ℓ_2	74.97	74.53	$+0.432$	0.0373
DDN-ℓ_2	seen ℓ_2	76.04	75.79	$+0.256$	0.341
DeepFool-ℓ_2	held-out ℓ_2	80.29	78.82	$+1.469$	0.0002
CW-ℓ_2	held-out ℓ_2	79.26	78.83	$+0.429$	0.249

AttackDRO++ produces a positive paired delta on all eight attacks. Five of these deltas have $p < 0.05$ under the per-attack paired test, and the gain is largest on DeepFool-ℓ_2 ($+1.47$ pp, $p = 1.6 \times 10^{-4}$), the most local-gradient ℓ_2 attack and the held-out attack on which the whitebox gain over Multi-AT was also largest. The two seen attacks (PGD-ℓ_∞ and DDN-ℓ_2) contribute small, non-significant gains, consistent with the fact that both methods already train against these attacks. The graybox Mean(8) gain is therefore distributed across attack families rather than concentrated on one family. Because the per-attack observations reuse the same target and surrogate checkpoints across attack

families, these p-values should be read as consistency checks over the graybox panel rather than as standalone training-run proof.

We aggregate the per-attack deltas into the four attack groups defined in Section 5.7 to compare the cross-method graybox profile across all four methods.

Table 6.13: Cross-method graybox accuracy by attack family. Values are off-diagonal target accuracy (%) averaged over the $n = 5$ panel. Bold values indicate the best method for each metric.

Target method	Seen ℓ_∞	Seen ℓ_2	Held-out ℓ_∞	Held-out ℓ_2	Mean(8)
PGD-AT	**61.48**	72.61	68.93	74.60	70.59
DDN-AT	56.58	**84.03**	68.60	**84.85**	**75.12**
Multi-AT	59.71	77.40	68.64	78.62	72.36
AttackDRO++	59.96	77.78	**69.18**	79.35	72.92

The family decomposition makes three points concrete. The held-out ℓ_∞ column has AttackDRO++ at the top (69.18%) by a small but consistent margin over the other three methods. The held-out ℓ_2 column is led by DDN-AT (84.85%), with AttackDRO++ (79.35%) ranked second and improving on Multi-AT by $+0.73$ pp. The seen ℓ_2 column shows the same pattern. The seen ℓ_∞ column favors PGD-AT, again by structure: surrogates generate strong PGD-ℓ_∞ adversarial examples, and only PGD-AT is robust to them. In this focused five-seed panel, AttackDRO++ is the only method that is at or above Multi-AT on every attack family, while DDN-AT and PGD-AT each remain narrow specialists under transfer. This supports the balanced-profile interpretation, but it remains a focused five-seed graybox panel rather than a replacement for the larger whitebox comparison.

Graybox-minus-whitebox gap and specialist inflation. The DDN-AT target row in Figure 6.11 and the DDN-AT row in Table 6.13 are the highest under graybox transfer, even though DDN-AT has the lowest whitebox Mean(8) on both the $n = 5$ and the full $n = 20$ panels. We resolve this apparent contradiction by quantifying cross-method graybox accuracy minus diagonal whitebox accuracy for each method. A large positive gap indicates that a defense looks much stronger under graybox transfer than under direct whitebox attack, which is a signature of a structural transfer mismatch rather than a broader robustness profile.

Table 6.14: Graybox-minus-whitebox gap by target method on the $n = 5$ panel. The gap is cross-method graybox Mean(8) minus diagonal whitebox Mean(8).

Target method	Whitebox Mean(8)	Graybox Mean(8)	Gap (pp)
PGD-AT	61.52	70.59	+ 9.06
DDN-AT	56.92	75.12	+18.20
Multi-AT	62.46	72.36	+ 9.90
AttackDRO++	62.64	72.92	+10.27

PGD-AT, Multi-AT, and AttackDRO++ all show a comparable graybox-minus-whitebox gap of approximately 9 to 10 pp, indicating that adversarial examples crafted on alternate surrogates are weaker than those crafted directly on the target by a roughly constant amount. DDN-AT, by contrast, gains $+18.20$ pp under graybox transfer, almost twice the gap of any other method. Its row-leading graybox accuracy is therefore not evidence of a stronger defense; it is an artifact of how poorly ℓ_∞ adversarial examples generated by PGD-AT, Multi-AT, and AttackDRO++ surrogates transfer to a purely-ℓ_2-trained target. AttackDRO++ retains the highest whitebox Mean(8) of the four methods while exhibiting a graybox gap that is only $+0.37$ pp larger than that of Multi-AT, which is consistent with a balanced cross-attack defense rather than a transfer-favored specialist.

The per-(class $\times$ attack) graybox-minus-whitebox gap heatmaps in Figures 6.14 through 6.17 make the specialist-inflation effect concrete. Cell values are computed as graybox accuracy minus whitebox accuracy in percentage points, so positive values indicate that transferred attacks are weaker than direct whitebox attacks on the same target.

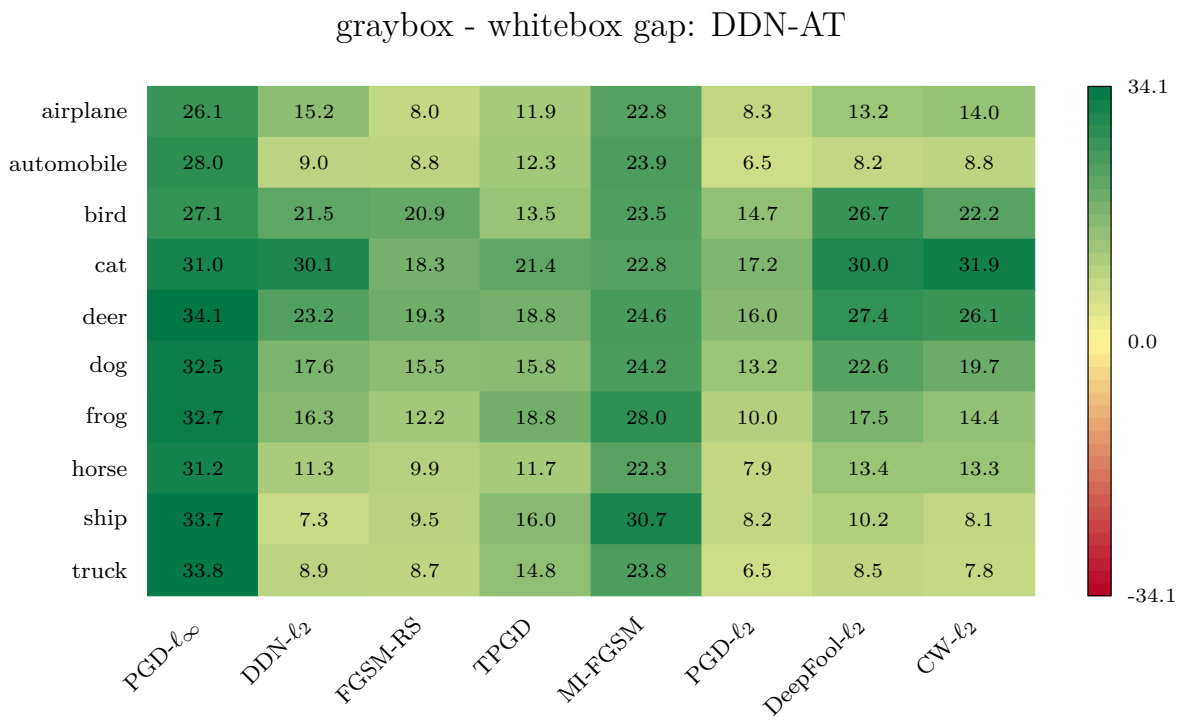


Figure 6.14: Graybox-minus-whitebox gap heatmap for DDN-AT. Cells report graybox accuracy minus whitebox accuracy in percentage points; positive values indicate that transferred attacks are weaker than direct whitebox attacks.

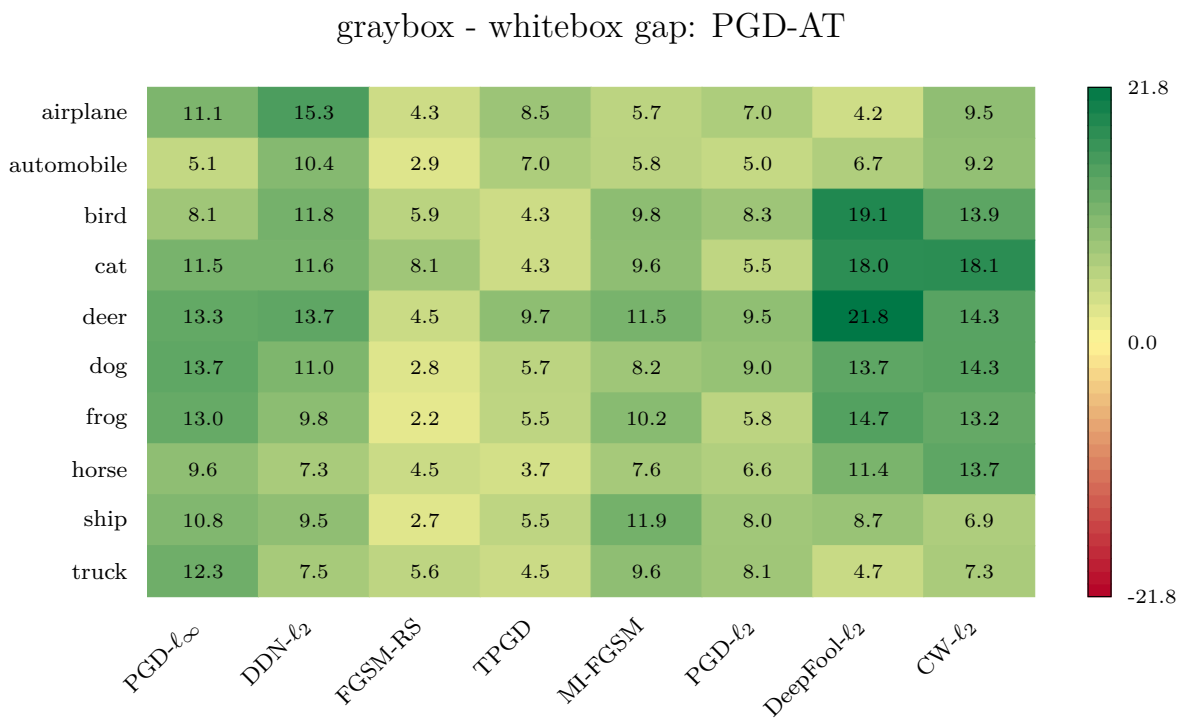


Figure 6.15: Graybox-minus-whitebox gap heatmap for PGD-AT. Cells report graybox accuracy minus whitebox accuracy in percentage points; positive values indicate that transferred attacks are weaker than direct whitebox attacks.

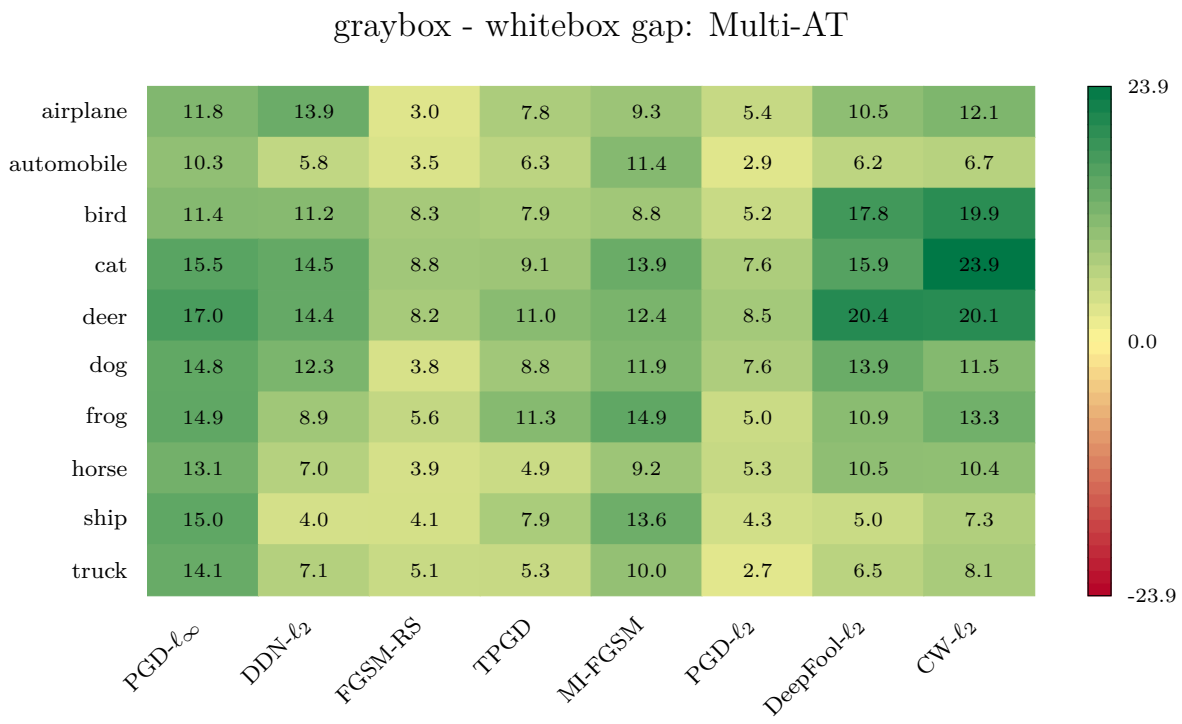


Figure 6.16: Graybox-minus-whitebox gap heatmap for Multi-AT. Cells report graybox accuracy minus whitebox accuracy in percentage points; positive values indicate that transferred attacks are weaker than direct whitebox attacks.

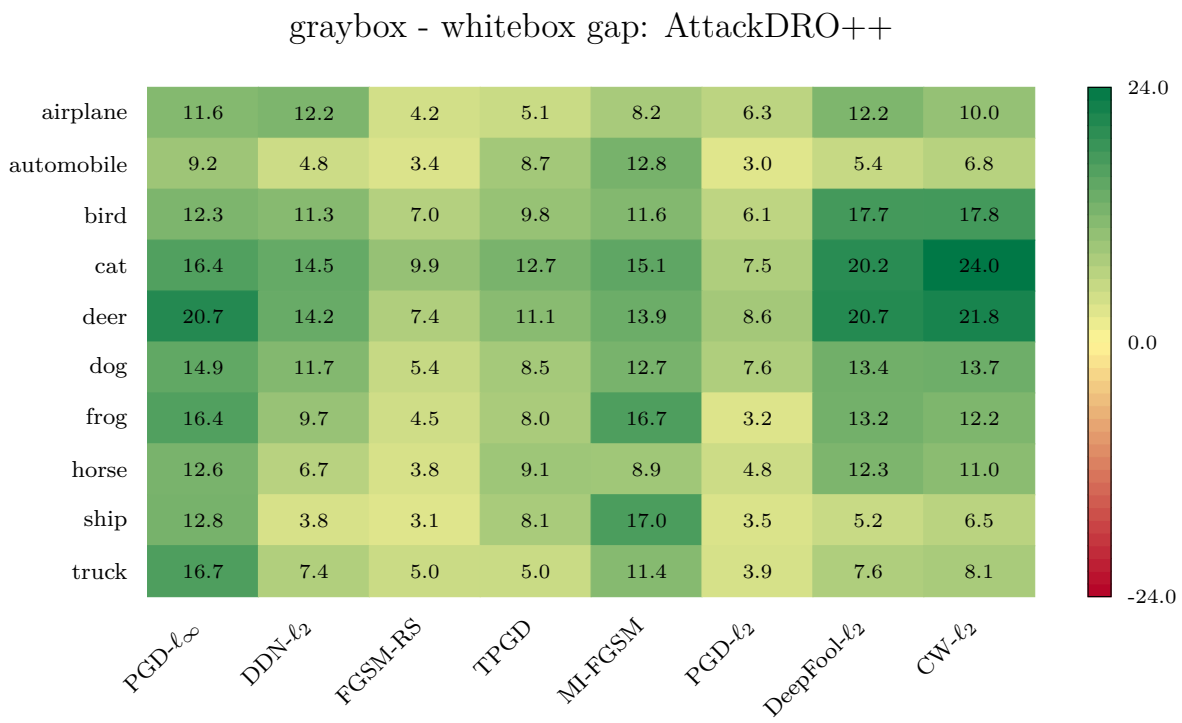


Figure 6.17: Graybox-minus-whitebox gap heatmap for AttackDRO++. Cells report graybox accuracy minus whitebox accuracy in percentage points; positive values indicate that transferred attacks are weaker than direct whitebox attacks.

DDN-AT exhibits broad positive gaps on the ℓ_∞-oriented attack columns, indicating that its high graybox score is partly driven by weak transfer from ℓ_∞ surrogates rather than uniformly strong robustness. PGD-AT, Multi-AT, and AttackDRO++ show smaller and more balanced gap patterns, with AttackDRO++ avoiding the severe specialist inflation observed for DDN-AT.

6.3.3 Class-Level Transfer Patterns

We complement the aggregate graybox analysis with a paired per-cell test over the full attack-by-class evaluation grid. For each of the 80 attack-by-class cells, we compare the graybox accuracy of the AttackDRO++ target against each baseline target across the five training seeds, so each cell contributes $n = 5$ paired seed-level observations.

This design keeps the comparison aligned at the cell level: the same attack, class, and seed index are compared across methods rather than being averaged away before testing. For each cell, we compute the paired difference between AttackDRO++ and the baseline and apply a two-sided paired t-test to assess whether the mean cell-level difference is distinguishable from zero. Because the per-cell sample size is small, these tests are used as localized diagnostic evidence rather than as the main basis for the chapter's claims.

Table 6.15: Per-(class $\times$ attack) graybox tests comparing AttackDRO++ against each baseline. Δ is computed as AttackDRO++ minus the baseline in percentage points.

Baseline	Positive cells ($\Delta > 0$)	Sign-test p	Mean Δ (pp)	Significant cells AttackDRO++ / Baseline	Bottom-10 lift (pp)
Multi-AT	59/80	1.3×10^{-5}	$+0.557$	2 / 3	$+0.779$
PGD-AT	52/80	4.8×10^{-3}	$+2.332$	34 / 12	$+1.329$
DDN-AT	27/80	2.4×10^{-3}	-2.206	16 / 32	$+0.709$

Three observations follow. First, AttackDRO++ exceeds Multi-AT on 59 of 80 cells with a mean per-cell delta of $+0.557$ pp, in close agreement with the $+0.549$ pp aggregate from Table 6.11. Cell-level $p < 0.05$ counts are sparse (2 vs. 3 cells reach that threshold in either direction), as expected under the $n = 5$ per-cell sample size and the small absolute deltas; the bulk of the evidence for AttackDRO++ is therefore the *distributional* shift across many cells rather than a few cell-level landslides.

Second, the bottom-10 lift is positive in all three contrasts. The cells on which Multi-AT, PGD-AT, and DDN-AT are weakest concentrate on the cat, deer, dog, and bird classes under MI-FGSM, PGD-ℓ_∞, and PGD-ℓ_2. AttackDRO++ improves on Multi-AT

by $+0.779$ pp on Multi-AT's worst 10 cells, and improves on PGD-AT by $+1.329$ pp on PGD-AT's worst 10 cells.

Third, the AttackDRO++ vs. DDN-AT cell-level comparison (-2.21 pp on average, with 32 cells favoring DDN-AT at $p < 0.05$) is the per-cell counterpart of the DDN-AT graybox inflation: most of DDN-AT's wins are on ℓ_2 cells, while AttackDRO++ retains the wins it needs on the ℓ_∞ cells that DDN-AT collapses on under whitebox attack (Section 6.1.2).

The per-cell deltas in Figures 6.18 through 6.20 make these patterns explicit: green cells favor AttackDRO++, red cells favor the baseline. The AttackDRO++ vs. Multi-AT heatmap is broadly green and small in magnitude; the AttackDRO++ vs. PGD-AT heatmap is broadly green with a few large red cells on ℓ_∞ attacks where PGD-AT specializes; and the AttackDRO++ vs. DDN-AT heatmap is mixed.

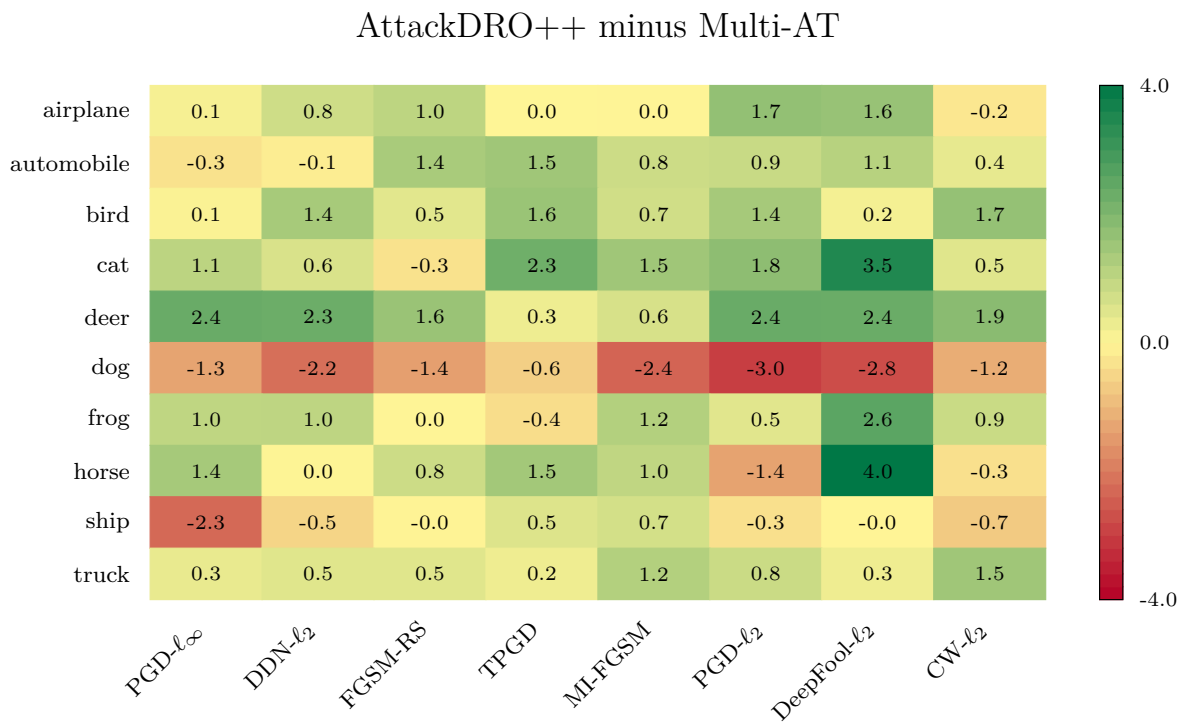


Figure 6.18: Per-(class $\times$ attack) graybox delta heatmap for AttackDRO++ minus Multi-AT. Values are percentage-point differences averaged over the five-seed graybox panel; green favors AttackDRO++ and red favors Multi-AT.

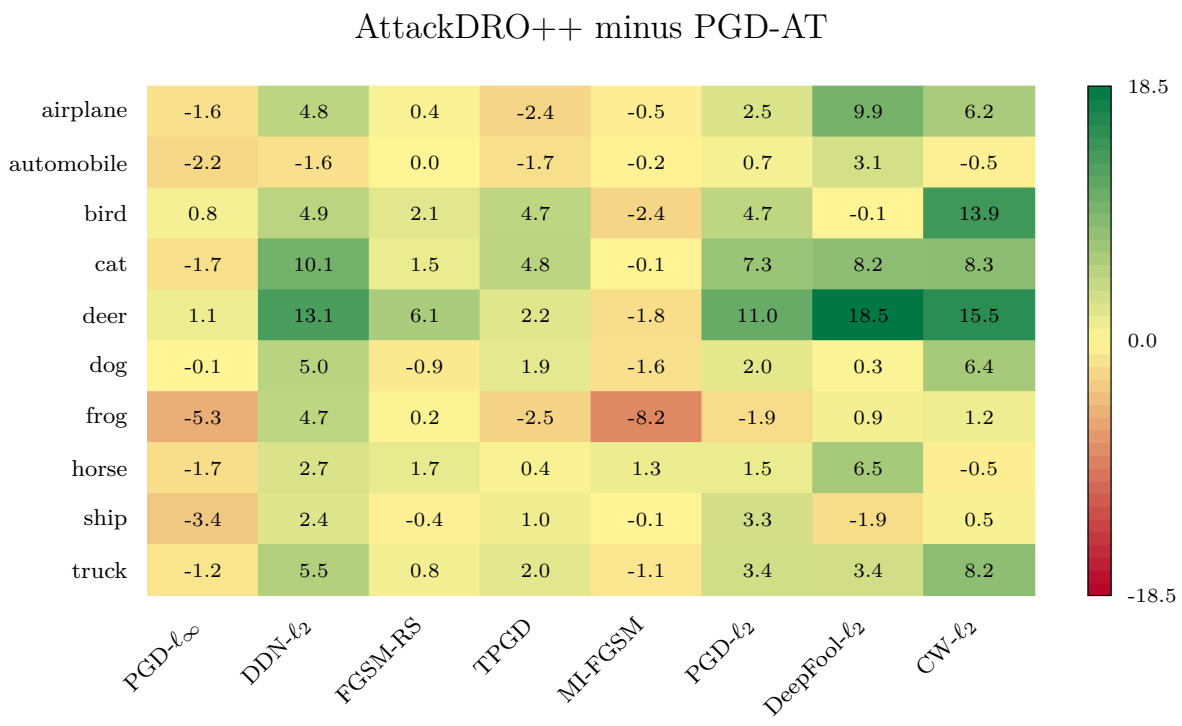


Figure 6.19: Per-(class × attack) graybox delta heatmap for AttackDRO++ minus PGD-AT. Values are percentage-point differences averaged over the five-seed graybox panel; green favors AttackDRO++ and red favors PGD-AT.

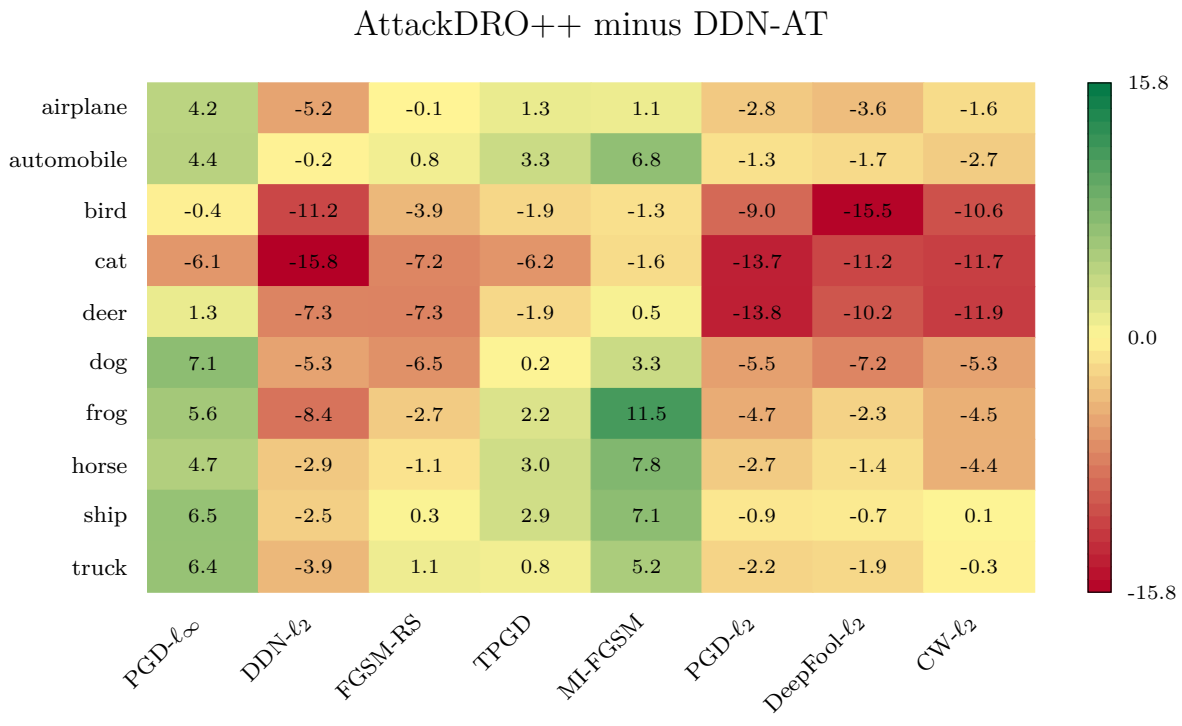


Figure 6.20: Per-(class × attack) graybox delta heatmap for AttackDRO++ minus DDN-AT. Values are percentage-point differences averaged over the five-seed graybox panel; green favors AttackDRO++ and red favors DDN-AT.

Method contribution decomposition. The whitebox results of Section 6.1 compare AttackDRO++ against Multi-AT and against the two single-attack baselines as three separate contrasts. Under graybox transfer, we can additionally ask how much of the AttackDRO++ improvement over each single-attack baseline is contributed by multi-attack training (the Multi-AT component) versus the Cluster-DRO objective (the AttackDRO++ component on top of Multi-AT). We decompose the total AttackDRO++ vs. single-attack delta as

$$\Delta_{\text{total}} = \underbrace{(\text{Multi-AT}) - (\text{single-AT})}_{\Delta_{\text{multi-attack}}} + \underbrace{(\text{AttackDRO++}) - (\text{Multi-AT})}_{\Delta_{\text{Cluster-DRO}}},$$

and report this decomposition for whitebox Mean(8), cross-method graybox Mean(8), and the bottom-10-cell cross-method graybox average (the average graybox accuracy on the ten attack-by-class cells in which the target is weakest). The bottom-10 metric is included because it isolates worst-cell behavior in this panel, where the design intent of AttackDRO++ is most visible.

Table 6.16: AttackDRO++ gain decomposition. Values are percentage-point changes; positive values indicate higher robust accuracy for the method named by the corresponding component.

Metric	Base	Multi-AT	AttackDRO++	Total
Whitebox Mean(8)	PGD-AT	$+0.934$	$+0.184$	$+1.118$
Whitebox Mean(8)	DDN-AT	$+5.536$	$+0.184$	$+5.720$
Cross-graybox Mean(8)	PGD-AT	$+1.775$	$+0.557$	$+2.332$
Cross-graybox Mean(8)	DDN-AT	-2.763	$+0.557$	-2.206
Bottom-10 graybox	PGD-AT	$+0.530$	$+0.799$	$+1.329$
Bottom-10 graybox	DDN-AT	-0.090	$+0.799$	$+0.709$

The AttackDRO++ component (column 4) is positive on every metric and decomposition path, ranging from $+0.184$ pp on whitebox Mean(8) to $+0.799$ pp on the bottom-10 cross-graybox metric. Two implications follow.

First, the AttackDRO++ component is larger under graybox transfer than under direct whitebox attack in this decomposition, consistent with the $+0.279$ pp $\rightarrow$ $+0.549$ pp scaling observed in Section 6.3.2.

Second, the AttackDRO++ component is roughly four times larger on the bottom-10 cells ($+0.799$ pp) than on whitebox Mean(8), consistent with the design intent that At-

tackDRO++ should help difficult cells rather than only raising the average. This decomposition does not identify which internal component drives the pattern; it only separates the observed Multi-AT and AttackDRO++ increments. The multi-attack column tells a complementary specialist story.

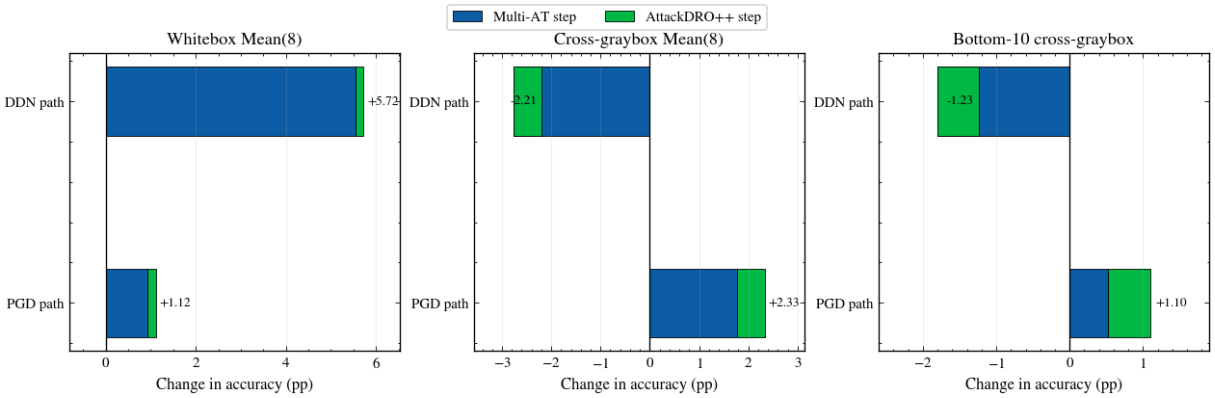


Figure 6.21: Visual decomposition of AttackDRO++ improvement over the single-attack baselines into a multi-attack effect (Multi-AT minus single-AT) and a Cluster-DRO effect (AttackDRO++ minus Multi-AT) across whitebox, cross-method graybox, and bottom-10 cross-method graybox metrics, for the two single-AT decomposition paths. Values are percentage-point differences.

Going from single-AT to Multi-AT adds $+5.54$ pp of whitebox Mean(8) when the baseline is DDN-AT, but removes -2.76 pp of cross-graybox Mean(8) for the same baseline, because DDN-AT's graybox accuracy is inflated by the very ℓ_∞ transfer mismatch that Multi-AT corrects. The AttackDRO++ component remains positive in both cases, suggesting an additional positive component beyond either multi-attack training alone or DDN-AT's structural graybox advantage in this decomposition.

The graybox analysis tests whether the whitebox pattern survives transfer from separately trained surrogate checkpoints. It supports the same broad interpretation as the direct whitebox evaluation, but transfer matrices and paired comparisons do not identify which design choices drive the pattern. The next section therefore turns to hyperparameter sensitivity, focusing on cluster granularity, anchor strength, and refresh frequency.

6.4 Sensitivity to Key Hyperparameters

The preceding sections evaluated the final AttackDRO++ configuration under whitebox, class-wise, and graybox settings. Those results show how the method behaves once the design choices are fixed, but they do not explain how sensitive the method is to those choices. This section therefore studies three practical hyperparameters: the number of discovered clusters, the strength of the uniform anchor, and the frequency with which

clusters are refreshed.

These ablations are intended as configuration guidance rather than as a second main comparison or theoretical proof. Unless otherwise stated, each setting uses three available runs under the same CIFAR-10, ResNet-18, source-attack, and evaluation-attack protocol defined in Chapter 5. Mean(8) and Worst(8) follow the definitions in Section 5.7: both are computed over the eight non-AutoAttack evaluation attacks, while AutoAttack-ℓ_∞ is reported separately.

6.4.1 Number of Clusters: How Many Groups Are Needed?

The number of clusters controls the granularity of the latent groups used by AttackDRO++. If K is too small, distinct failure modes may be merged into the same group, making the Cluster-DRO correction too coarse. If K is too large, the minibatch may be split into many small groups, making group-loss estimates noisier and the cluster weights less stable.

Because K controls the trade-off between coarse grouping and noisy small groups, Table 6.17 tests whether AttackDRO++ is sensitive to cluster granularity. Across the available runs, all settings remain close in Mean(8), with differences well below one percentage point. The default $K = 4$ setting gives the highest Worst(8) among the tested settings and remains competitive in Mean(8), while $K = 2$ is slightly more coarse and $K = 6$ or $K = 10$ do not provide a clear gain.

Table 6.17: Number-of-clusters ablation for AttackDRO++ with anchor strength $\lambda_{\text{DRO}} = 0.35$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across available runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Setting	K	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
Coarse grouping	2	3	62.33 ± 0.06	45.13 ± 0.27	40.95 ± 0.90
Default grouping	4	3	$\mathbf{62.35 \pm 0.20}$	$\mathbf{45.33 \pm 0.25}$	$\mathbf{41.14 \pm 1.15}$
Finer grouping	6	3	62.28 ± 0.14	45.17 ± 0.47	40.95 ± 1.30
Very fine grouping	10	3	62.35 ± 0.09	45.17 ± 0.08	40.62 ± 1.11

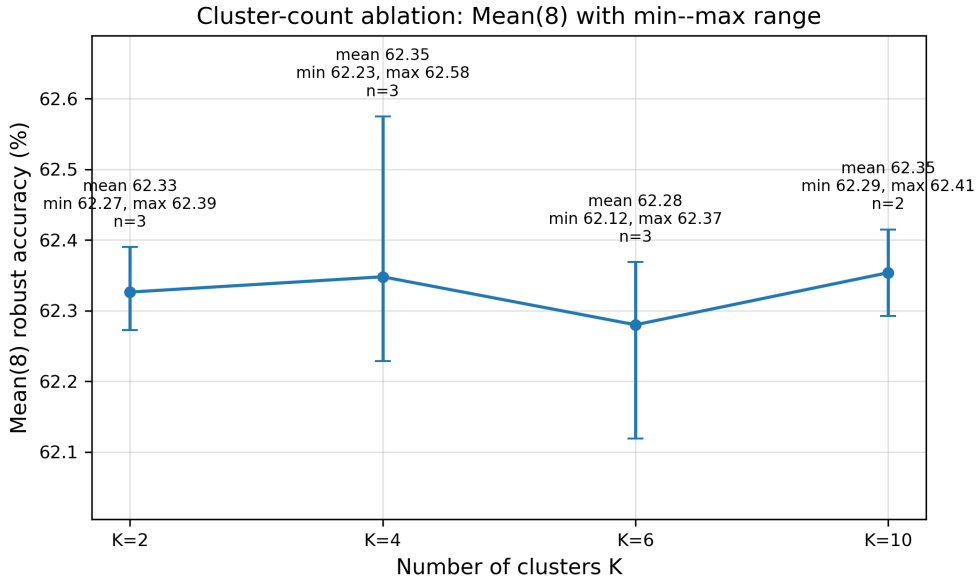


Figure 6.22: Sensitivity of AttackDRO++ to the number of clusters. Points show Mean(8) robust accuracy with the min-to-max range across available runs; Mean(8) is computed over the eight non-AutoAttack evaluation attacks.

The ablation supports using a moderate number of clusters rather than treating the cluster count as a highly tuned parameter. The default $K = 4$ is expressive enough to go beyond the two source attack identities, but it does not fragment the training batch as aggressively as larger values. Therefore, the evidence supports $K = 4$ as a practical default in this protocol, not as a universally optimal cluster count. The next ablation checks whether the anchor strength is similarly tolerant to moderate changes.

6.4.2 Anchor Strength: Finding a Stable Trade-off

The anchor strength λ_{DRO} controls the interpolation between the uniform multi-attack ERM objective and the adaptive Cluster-DRO objective. A smaller value keeps training closer to the uniform baseline, while a larger value gives the cluster-reweighted loss more influence. In this ablation, the previous $\lambda_{\text{DRO}} = 1.0$ setting is removed because it corresponds to an overly aggressive Cluster-DRO setting and is not part of the final candidate set.

Table 6.18: Anchor-strength ablation for AttackDRO++ with $K = 4$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across three runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Anchor strength λ_{DRO}	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
0.20	3	62.28 ± 0.09	45.01 ± 0.40	41.08 ± 1.66
0.35 (default)	3	$\mathbf{62.35 \pm 0.20}$	$\mathbf{45.33 \pm 0.25}$	$\mathbf{41.14 \pm 1.15}$
0.50	3	62.24 ± 0.18	45.24 ± 0.40	40.24 ± 1.09

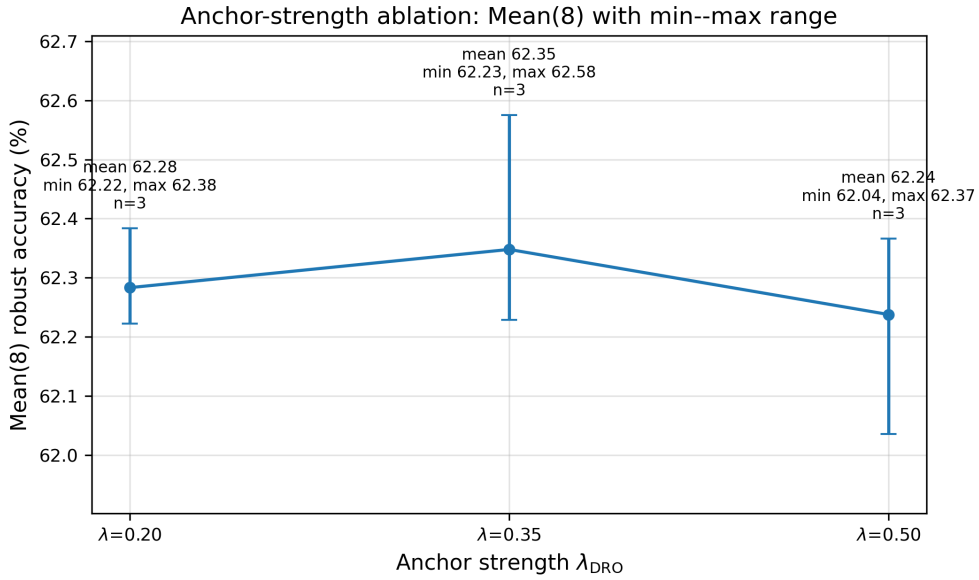


Figure 6.23: Sensitivity of AttackDRO++ to the anchor strength λ_{DRO}. Points show Mean(8) robust accuracy with the min-to-max range across available runs; Mean(8) is computed over the eight non-AutoAttack evaluation attacks. The previous $\lambda_{\text{DRO}} = 1.0$ setting is excluded from the final ablation panel.

Because the anchor controls how far the objective can move away from uniform Multi-AT, Table 6.18 tests three practical anchor strengths. The differences are again small, but the moderate setting $\lambda_{\text{DRO}} = 0.35$ gives the strongest Mean(8), Worst(8), and AutoAttack values among the tested settings.

The lower value $\lambda_{\text{DRO}} = 0.20$ remains close, suggesting that the method does not require a sharply tuned anchor. The higher value $\lambda_{\text{DRO}} = 0.50$ slightly reduces Mean(8) and AutoAttack, which is consistent with the idea that too much emphasis on adaptive cluster weighting can weaken the contribution of the uniform multi-attack baseline.

Overall, the anchor-strength ablation supports the design motivation from Chapter 4. AttackDRO++ benefits from a nonzero cluster-level correction, but the correction should

remain anchored to the uniform multi-attack objective. The default value $\lambda_{\text{DRO}} = 0.35$ is retained because it gives the best balance among the tested settings without relying entirely on either uniform ERM or aggressive Cluster-DRO weighting. Because this panel uses three runs per setting, the conclusion is configuration guidance rather than proof that this anchor is optimal in other protocols.

6.4.3 Recluster Frequency: How Often Should Groups Update?

The cluster-refresh interval R controls how often the K-means clusterer is refitted from the feature bank. Refreshing every epoch may adapt faster to changes in the representation, but it also changes the group structure more frequently. Refreshing less often may give a more stable grouping signal, but it may lag behind the evolving model.

Since the refresh interval trades adaptation speed against group stability, Table 6.19 compares refreshing every one epoch with refreshing every two epochs. The two schedules produce nearly identical Mean(8), Worst(8), and family-level behavior. The default two-epoch schedule gives a slightly higher Mean(8) and ℓ_∞-family average, while the one-epoch schedule gives a slightly higher Worst(8), AutoAttack, and ℓ_2-family average. These differences are small relative to the three-run setting, so the ablation does not show a strong advantage for either schedule.

Table 6.19: Cluster-refresh schedule ablation for AttackDRO++ with $\lambda_{\text{DRO}} = 0.35$, gradient fingerprints, and $K = 4$. Values are robust accuracy (%) reported as mean $\pm$ standard deviation across three runs. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best setting for each metric.

Refresh interval	Runs	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑	ℓ_2-family avg. ↑	ℓ_∞-family avg. ↑
1 epoch	3	62.38 ± 0.14	**45.17 ± 0.34**	**40.30 ± 1.37**	**67.57 ± 0.16**	57.18 ± 0.27
2 epochs (default)	3	**62.44 ± 0.22**	45.14 ± 0.23	39.45 ± 0.68	67.56 ± 0.08	**57.31 ± 0.35**

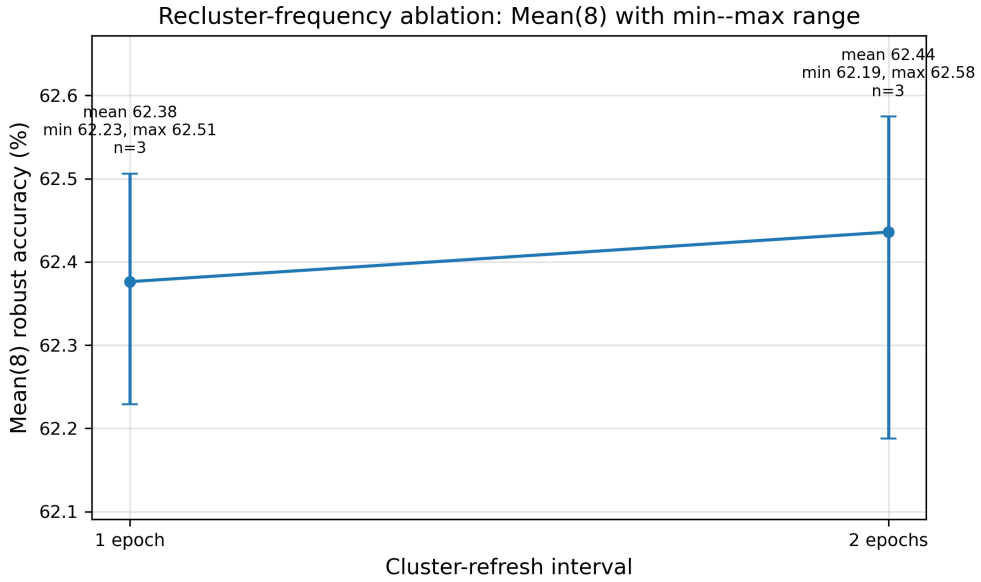


Figure 6.24: Sensitivity of AttackDRO++ to the cluster-refresh interval. Points show Mean(8) robust accuracy with the min-to-max range across available runs; Mean(8) is computed over the eight non-AutoAttack evaluation attacks. The two refresh schedules produce similar results, so the default two-epoch schedule is retained as a practical stability choice.

We retain $R = 2$ epochs as the default because it gives comparable robustness while reducing the frequency of cluster refitting. This is a practical choice: the ablation suggests that the method is not highly sensitive to the exact refresh interval, so the less frequent schedule is preferred for simplicity.

Supplementary architecture check. A single-run WRN-28-10 check is reported in Appendix C as supplementary evidence. This check is not included in the main paired claims because it does not use the repeated-random-seed protocol of the main ResNet-18 experiments. In that setting, Multi-AT slightly outperforms AttackDRO++ on both Mean(8) and AutoAttack-ℓ_∞, suggesting that the relative benefit of the proposed grouping strategy may depend on model capacity and should be evaluated more systematically in future work.

Together, these ablations suggest that the final AttackDRO++ configuration is not the result of a fragile hyperparameter choice. Moderate clustering granularity, moderate anchor strength, and a two-epoch refresh schedule provide a practical configuration for the evaluated setting. The ablations also define the limits of the current evidence: they use fewer runs than the main comparison and support the selected ResNet-18 configuration, but broader architecture-level conclusions require additional repeated-seed experiments. Chapter 6 evaluated AttackDRO++ through four connected evidence panels. The ag-

gregate whitebox results first showed that the proposed method gives modest average and held-out gains relative to Multi-AT while leaving Worst(8) and the separate AutoAttack checks broadly unchanged. The per-attack and class-wise drilldowns then clarified where this behavior comes from: the method acts less like a single-norm specialist and more like a balanced multi-attack model, although hard animal-class cells under strong ℓ_∞ attacks remain persistent failure cases.

The graybox analysis tested whether the same pattern survives transferred attacks from separately trained surrogate checkpoints. It supported the whitebox interpretation while also showing that transfer accuracy must be read carefully because specialist models can appear strong when attacks transfer poorly across threat geometries. Finally, the ablation section examined whether the observed behavior depends on key design choices, treating cluster count, anchor strength, and refresh frequency as sensitivity checks rather than standalone proof.

Overall, the supported conclusion is conservative: within the CIFAR-10 and ResNet-18 setting studied here, AttackDRO++ improves cross-attack balance over the uniform multi-attack baseline without establishing superiority on every attack or extending robustness claims beyond the evaluated protocol. Chapter 7 summarizes these findings, states the limitations, and outlines future extensions.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This report studied cross-attack adversarial robustness: the problem that a model trained against one perturbation family may not remain equally robust against attacks with different norms, objectives, or optimization dynamics. The main comparison treats attacks as domain-like sources of adversarial examples and separates source attacks used during training from held-out attacks used during evaluation. Under this framing, robustness is not summarized by a single attack score alone; it is also evaluated by how evenly a method behaves across attack families, classes, and transfer settings.

The first contribution is this controlled cross-attack framing. The report uses PGD-ℓ_∞ and DDN-ℓ_2 as the source attacks for the multi-attack methods, evaluates a bounded suite of non-AutoAttack attacks with Mean(8) and Worst(8), and reports AutoAttack-ℓ_∞ separately from the main aggregate ranking. The main empirical comparison focuses on PGD-AT, DDN-AT, Multi-AT, and AttackDRO++ under the same CIFAR-10/ResNet-18 protocol. AttackDRO and Cluster-DRO are part of the method development path, but the central Chapter 6 comparison is against the two single-source specialists and the stronger Multi-AT baseline.

The methodological contribution is AttackDRO++ as a stability-oriented refinement of Multi-AT. Multi-AT already exposes the model to both PGD-ℓ_∞ and DDN-ℓ_2 source attacks, making it the relevant strong baseline for this report. AttackDRO++ keeps that uniform multi-attack signal but replaces coarse attack identities with latent clusters, enriches the clustering feature with gradient fingerprints, and anchors the adaptive Cluster-DRO correction to the uniform Multi-AT objective. The method is therefore not presented as a universal defense; it is a practical attempt to make multi-attack training more sensitive

to latent difficulty groups.

The empirical contribution is also deliberately bounded. Chapter 6 supports modest average robustness gains over Multi-AT, lower observed seed-to-seed variation on average-case metrics, useful graybox trends, and configuration guidance from ablations. The same evidence also shows that AttackDRO++ is not highest on every individual attack, does not improve the hardest-case metric over Multi-AT, and is statistically indistinguishable from Multi-AT under the separate AutoAttack checks. These results support a conservative conclusion: under the evaluated CIFAR-10/ResNet-18 setting, AttackDRO++ improves average cross-attack balance without establishing unrestricted, certified, or architecture-general robustness.

7.2 Answers to Research Questions

The research questions posed in Chapter 1 can now be answered from the evidence in Chapter 6. The answers remain conditional on the CIFAR-10 and ResNet-18 setting used for the main experiments, and the strongest claims are restricted to the evaluation protocol defined in Chapter 5.

RQ1. Single-source adversarial training produces attack-dependent robustness. PGD-AT remains strongest on the most ℓ_∞-dominated behavior, including Worst(8), while DDN-AT is strongest on the held-out ℓ_2 mean. These specialists are therefore useful baselines rather than uniformly weak methods. However, each loses robustness outside its preferred attack family, and the per-attack breakdown in Table 6.5 confirms that the source attack leaves a clear footprint on the robustness profile.

RQ2. Multi-AT improves robustness balance compared with single-source training and is the main strong baseline for the proposed method. Its advantage comes from training on both PGD-ℓ_∞ and DDN-ℓ_2 source attacks, which reduces cross-norm specialization. The result should not be read as a new worst-case guarantee: the per-attack and class-wise analyses still identify difficult cells, especially for hard CIFAR-10 classes under strong ℓ_∞ attacks. Multi-AT therefore sets a meaningful practical baseline, while also motivating a finer grouping mechanism than fixed source-attack identity.

RQ3. AttackDRO++ gives a modest average gain over Multi-AT under the evaluated setting, but it does not dominate every individual attack. In the main whitebox panel, AttackDRO++ improves Mean(8) over Multi-AT by $+0.279$ pp and held-out attack mean

by $+0.319$ pp in Table 6.1. The corrected tests in Table 6.4 require conservative interpretation, so this should be described as a small positive average-case effect rather than as a broad dominance claim. Per-attack results further show that AttackDRO++ is highest on only part of the attack suite.

The same answer applies to stability. Table 6.8 shows lower observed seed-to-seed variation for AttackDRO++ than for Multi-AT on Mean(8), held-out averages, and seen-source mean; for Mean(8), the standard deviation decreases from 0.447 pp to 0.148 pp. This suggests a more predictable observed outcome under the paired protocol, but it does not identify the mechanism behind the variation reduction. The hardest-case and AutoAttack checks also remain bounded: Worst(8), the 512-sample AutoAttack sanity check, and the full-test AutoAttack-ℓ_∞ panel are statistically indistinguishable from Multi-AT, with the full-test AutoAttack difference reported as about -0.006 pp with $p = 0.9584$. AutoAttack is therefore separate from Mean(8) and does not support a stronger aggregate ranking claim.

The graybox results are useful but also bounded. The paired graybox comparison shows a $+0.549$ pp trend over Multi-AT in Table 6.11, and the per-attack graybox panel is broadly consistent with the whitebox average-case story. However, the paired graybox cells share checkpoints, seeds, surrogates, and attack families; they are not fully independent training runs. The graybox evidence therefore supports a consistent transfer trend, not a standalone proof that AttackDRO++ generalizes under all transfer conditions.

RQ4. The ablations provide configuration guidance, not an exhaustive optimization claim. Section 6.4 supports the selected $\lambda_{\text{DRO}} = 0.35$, $K = 4$, and two-epoch cluster-refresh setting as a practical operating point among the tested choices. The evidence is consistent with the design motivation: the adaptive cluster correction should be strong enough to emphasize harder groups, but still anchored to the uniform Multi-AT signal. Because these ablation panels are smaller than the main aggregate comparison, they should be read as practical guidance for this protocol rather than as evidence of universal hyperparameter optimality.

7.3 Limitations of the Current Study

The main limitation is scope. CIFAR-10 is the only main dataset, and ResNet-18 is the only main architecture. These choices make the study controlled and reproducible, but they also bound the conclusion. Appendix C includes a WRN-28-10 check, but that result is supplementary and does not support a broad architecture-general claim. Larger

datasets, higher-capacity models, and different data domains may change both the attack difficulty profile and the usefulness of latent cluster discovery.

The attack suite is also fixed. The report evaluates a defined set of whitebox and graybox attacks, with AutoAttack-ℓ_∞ reported separately from Mean(8). The AutoAttack protocol is useful as a standardized stress test, but the current implementation uses a 512-sample sanity layer and a selected full-test seed panel rather than a full RobustBench-style evaluation across all possible settings. The study therefore does not claim certified robustness, universal robustness, or robustness against adaptive attacks designed with full knowledge of the grouping and reweighting mechanism.

The graybox design has a dependency limitation. Although the graybox section uses independently trained checkpoints, the paired cells share target checkpoints, surrogate checkpoints, seed indices, and attack families. The resulting tests are useful for detecting transfer trends within the protocol, but they are not equivalent to tests over fully independent training runs. This matters because graybox accuracy can also be inflated by geometry mismatch, as seen in the DDN-AT transfer behavior.

Statistical power and compute budget vary across the evidence panels. The main aggregate comparison uses 20 paired random-seed runs, while the class-wise, graybox, ablation, and supplementary architecture analyses use smaller panels. These smaller analyses are valuable for diagnostics and configuration guidance, but they should not carry the same evidential weight as the main paired aggregate comparison. AttackDRO++ also adds practical overhead through feature banks, gradient fingerprints, clustering, cluster refreshes, and anchor tuning, which may become more expensive on larger datasets and models.

7.4 Directions for Future Work

The first priority is broader evaluation. The CIFAR-10/ResNet-18 result should be tested on CIFAR-100, Tiny ImageNet, ImageNet-scale data where feasible, and additional architectures such as ResNet variants, WRN backbones, and transformer-style classifiers. This would show whether the lower observed variation and modest average gains persist when the data distribution and model capacity change.

Future comparisons should also include stronger external baselines. Multi-AT is the right baseline for this report, but larger studies should compare against methods such as MSD-style training, max-over-source-attacks training, and other strong multi-norm or multi-source adversarial training procedures. Those comparisons would clarify whether the latent-group refinement remains useful when the base multi-attack objective is strength-

ened.

Attack evaluation should be expanded toward stronger AutoAttack and RobustBench-style protocols, larger held-out suites, and adaptive tests that account for the grouping mechanism. Graybox and blackbox evaluation should also use cleaner independence where possible, including cross-architecture transfer, larger surrogate ensembles, and method families not used during training or selection.

The method itself needs cleaner diagnostics and ablations. Future work should separate the value of gradient fingerprints from the value of clustering through no-GradFP and random-cluster controls if those panels are not already finalized in a stronger experimental setting. It should also study cluster formation, cluster purity, feature-bank drift, and the dynamics of the cluster weights q_k during training. Better theory and diagnostics for when latent groups help would make the method easier to tune and less dependent on trial-and-error configuration.

Finally, the computational cost should be reduced. Cheaper gradient-fingerprint approximations, smaller or adaptive feature banks, and refresh schedules that react to cluster drift could make AttackDRO++ more practical for larger datasets, larger models, and longer training schedules.

7.5 Closing Remarks

The main conclusion is that AttackDRO++ is a conservative refinement of Multi-AT, not a general solution to robustness. Under the evaluated CIFAR-10/ResNet-18 protocol, latent clusters, gradient fingerprints, and a uniform anchor provide modest average gains and lower observed seed-to-seed variation while leaving hardest-case and AutoAttack behavior broadly similar to Multi-AT.

This supports the report's central claim: adversarial robustness should be studied across attacks, not only through a single headline metric. Treating attacks as domain-like sources makes specialization visible, and latent group-aware training is a practical way to address part of that gap. Broader datasets, stronger baselines, and stricter attack protocols are still required before making claims beyond this setting.

Bibliography

[1] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," 2014. [Online]. Available: https://arxiv.org/abs/1312.6199

[2] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015. [Online]. Available: https://arxiv.org/abs/1412.6572

[3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2019. [Online]. Available: https://arxiv.org/abs/1706.06083

[4] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, "Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization," *arXiv preprint arXiv:1911.08731*, 2019. [Online]. Available: https://arxiv.org/abs/1911.08731

[5] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *International Conference on Machine Learning*, 2020, pp. 2206–2216.

[6] F. Tramer and D. Boneh, "Adversarial training and robustness for multiple perturbations," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/5d4ae76f053f8f2516ad12961ef7fe97-Abstract.html

[7] P. Maini, E. Wong, and Z. Kolter, "Adversarial robustness against the union of multiple perturbation models," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 6640–6650. [Online]. Available: https://proceedings.mlr.press/v119/maini20a.html

[8] A. Athalye, N. Carlini, and D. A. Wagner, "Obfuscated gradients give a false

sense of security: Circumventing defenses to adversarial examples," *CoRR*, vol. abs/1802.00420, 2018. [Online]. Available: http://arxiv.org/abs/1802.00420

[9] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, "Square attack: a query-efficient black-box adversarial attack via random search," *CoRR*, 2019. [Online]. Available: https://arxiv.org/abs/1912.00049

[10] E. Wong, L. Rice, and J. Z. Kolter, "Fast is better than free: Revisiting adversarial training," *CoRR*, vol. abs/2001.03994, 2020. [Online]. Available: https://arxiv.org/abs/2001.03994

[11] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," 2017. [Online]. Available: https://arxiv.org/abs/1608.04644

[12] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," 2016. [Online]. Available: https://arxiv.org/abs/1511.04599

[13] J. Rony, L. G. Hafemann, L. S. Oliveira, I. B. Ayed, R. Sabourin, and E. Granger, "Decoupling direction and norm for efficient gradient-based l2 adversarial attacks and defenses," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4322–4330.

[14] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, "Boosting adversarial attacks with momentum," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9185–9193. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Dong_Boosting_Adversarial_Attacks_CVPR_2018_paper.html

[15] H. Zhang, Y. Yu, J. Jiao, E. P. Xing, L. E. Ghaoui, and M. I. Jordan, "Theoretically principled trade-off between robustness and accuracy," *CoRR*, vol. abs/1901.08573, 2019. [Online]. Available: http://arxiv.org/abs/1901.08573

[16] L. Rice, E. Wong, and J. Z. Kolter, "Overfitting in adversarially robust deep learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 8093–8104. [Online]. Available: https://proceedings.mlr.press/v119/rice20a.html

[17] M. Zhao, L. Zhang, J. Ye, H. Lu, B. Yin, and X. Wang, "Adversarial training: A survey," *CoRR*, vol. abs/2410.15042, 2024. [Online]. Available: https://arxiv.org/abs/2410.15042

[18] T. Hashimoto, M. Srivastava, H. Namkoong, and P. Liang, "Fairness without demographics in repeated loss minimization," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1929–1938. [Online]. Available: https://proceedings.mlr.press/v80/hashimoto18a.html

[19] J. C. Duchi and H. Namkoong, "Learning models with uniform performance via distributionally robust optimization," *The Annals of Statistics*, vol. 49, no. 3, pp. 1378–1406, 2021.

[20] J. Kivinen and M. K. Warmuth, "Exponentiated gradient versus gradient descent for linear predictors," *Information and Computation*, vol. 132, no. 1, pp. 1–63, 1997.

[21] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, "Invariant risk minimization," *arXiv preprint arXiv:1907.02893*, 2019. [Online]. Available: https://arxiv.org/abs/1907.02893

[22] I. Gulrajani and D. Lopez-Paz, "In search of lost domain generalization," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=lQdXeXDoWtI

[23] N. Sohoni, J. Dunnmon, G. Angus, A. Gu, and C. Re, "No subclass left behind: Fine-grained robustness in coarse-grained classification problems," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/e0688d13958a19e087e123148555e4b4-Abstract.html

[24] E. Creager, J.-H. Jacobsen, and R. Zemel, "Environment inference for invariant learning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 2189–2200. [Online]. Available: https://proceedings.mlr.press/v139/creager21a.html

[25] E. Z. Liu, B. Haghgoo, A. S. Chen, A. Raghunathan, P. W. Koh, S. Sagawa, P. Liang, and C. Finn, "Just train twice: Improving group robustness without training group information," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6781–6792. [Online]. Available: https://proceedings.mlr.press/v139/liu21f.html

[26] G. Charpiat, N. Girard, L. Felardos, and Y. Tarabalka, "Input similarity from the neural network perspective," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://openreview.net/forum?id=B1lCtNreLr

[27] M. Paul, S. Ganguli, and G. K. Dziugaite, "Deep learning on a data diet: Finding important examples early in training," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://proceedings.neurips.cc/paper/2021/hash/ac56f8fe9eea3e4a365f29f0f1957c55-Abstract.html

[28] Student, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908.

[29] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[30] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[31] G. Cumming, "The new statistics: Why and how," *Psychological Science*, vol. 25, no. 1, pp. 7–29, 2014.

[32] J. Uesato, B. O'Donoghue, P. Kohli, and A. van den Oord, "Adversarial risk and the dangers of evaluating against weak attacks," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 5025–5034. [Online]. Available: https://proceedings.mlr.press/v80/uesato18a.html

[33] S. Dai, S. Mahloujifar, C. Xiang, V. Sehwag, P.-Y. Chen, and P. Mittal, "Multirobustbench: Benchmarking robustness against multiple attacks," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 6760–6785. [Online]. Available: https://proceedings.mlr.press/v202/dai23c.html

[34] P. Maini, X. Chen, B. Li, and D. Song, "Perturbation type categorization for multiple adversarial perturbation robustness," in *Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence*, ser. Proceedings of Machine Learning Research, vol. 180. PMLR, 2022, pp. 1317–1327. [Online]. Available: https://proceedings.mlr.press/v180/maini22a.html

[35] D. Krueger, E. Caballero, J.-H. Jacobsen, A. Zhang, J. Binas, D. Zhang, R. Le Priol, and A. Courville, "Out-of-distribution generalization via risk extrapolation (rex)," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 5815–5826. [Online]. Available: https://proceedings.mlr.press/v139/krueger21a.html

[36] D. Stutz, M. Hein, and B. Schiele, “Disentangling adversarial robustness and generalization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 6976–6987. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Stutz_Disentangling_Adversarial_Robustness_and_Generalization_CVPR_2019_paper.html

[37] K. Alhamoud, H. Abed Al Kader Hammoud, M. Alfarra, and B. Ghanem, “Generalizability of adversarial robustness under distribution shifts,” *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://openreview.net/forum?id=XNFo3dQiCJ

[38] X. Zou and W. Liu, “On the adversarial robustness of out-of-distribution generalization models,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://openreview.net/forum?id=IiwTFcGGTq

[39] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[40] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

Appendix A

Algorithms and Pseudocode

This appendix collects compact pseudocode for the attacks and training procedures used in the report. The algorithms use symbolic budgets, step counts, and learning rates; concrete configuration values are defined in Chapter 5.

Algorithm A.1: FGSM-RS

Input: Model f_θ, input x, label y, loss ℓ, budget ε, step size α

Output: Adversarial input x^{adv}

Sample $\delta_0 \in \mathcal{S}_\infty(\varepsilon)$

Set $x_0 \leftarrow \Pi_{[0,1]^d}(x + \delta_0)$

Compute $g \leftarrow \nabla_x \ell(f_\theta(x_0), y)$

Set $x^{\text{adv}} \leftarrow \Pi_{(x+\mathcal{S}_\infty(\varepsilon))\cap[0,1]^d}(x_0 + \alpha\,\text{sign}(g))$

return x^{adv}

Algorithm A.2: PGD Attack

Input: Model f_θ, input x, label y, loss ℓ, set $\mathcal{S}_p(\varepsilon)$, step size α, steps T

Output: Adversarial input x_T^{adv}

Initialize $x_0^{\text{adv}} \in (x + \mathcal{S}_p(\varepsilon)) \cap [0,1]^d$

for $t = 0, \ldots, T-1$ **do**

  Compute $g_t \leftarrow \nabla_x \ell(f_\theta(x_t^{\text{adv}}), y)$

  Take a normalized ascent step using the ℓ_p geometry

  Project x_{t+1}^{adv} onto $(x + \mathcal{S}_p(\varepsilon)) \cap [0,1]^d$

return x_T^{adv}

Algorithm A.3: CW-ℓ_2 Attack

Input: Model f_θ, input x, label y, margin loss g, trade-off c, optimizer steps T

Output: Adversarial input x^{adv}

Initialize perturbation variable δ

for $t = 1, \dots, T$ **do**

  Minimize $\|\delta\|_2^2 + c\, g(x + \delta)$ by gradient-based optimization

  Clip or transform $x + \delta$ so the candidate remains in $[0, 1]^d$

  Keep the best successful adversarial candidate found so far

return *best successful candidate, or the final candidate if none succeeds*

Algorithm A.4: DeepFool-ℓ_2 Attack

Input: Model f_θ, input x, current class y, steps T

Output: Adversarial input x^{adv}

Set $x_0^{\text{adv}} \leftarrow x$

for $t = 0, \dots, T - 1$ **do**

  if $\arg\max_c f_\theta(x_t^{\text{adv}})_c \neq y$ **then**

    return x_t^{adv}

  Linearize the class scores around x_t^{adv}

  Estimate the closest class boundary under the local linear model

  Move x_t^{adv} by the corresponding minimal ℓ_2 perturbation

return x_T^{adv}

Algorithm A.5: DDN-ℓ_2 Attack

Input: Model f_θ, input x, label y, loss ℓ, initial radius ρ_0, step size α, steps T

Output: Adversarial input x^{adv}

Initialize δ_0 and radius ρ_0

for $t = 0, \dots, T - 1$ **do**

  Compute $g_t \leftarrow \nabla_x \ell(f_\theta(x + \delta_t), y)$

  Update perturbation direction using the normalized gradient

  if $x + \delta_t$ *is adversarial* **then**

    Decrease the radius for the next candidate

  else

    Increase the radius for the next candidate

  Normalize the direction to the updated radius and clip to $[0, 1]^d$

return *best successful candidate found during the search*

Algorithm A.6: MI-FGSM

Input: Model f_θ, input x, label y, loss ℓ, budget ε, step size α, momentum μ, steps T

Output: Adversarial input x_T^{adv}

Initialize $x_0^{\text{adv}} \leftarrow x$ and $g_0 \leftarrow 0$

for $t = 0, \dots, T-1$ **do**

  Compute $r_t \leftarrow \nabla_x \ell(f_\theta(x_t^{\text{adv}}), y)$

  Set $g_{t+1} \leftarrow \mu g_t + r_t / \|r_t\|_1$

  Set $x_{t+1}^{\text{adv}} \leftarrow \Pi_{(x+\mathcal{S}_\infty(\varepsilon))\cap[0,1]^d}(x_t^{\text{adv}} + \alpha \operatorname{sign}(g_{t+1}))$

return x_T^{adv}

Algorithm A.7: TRADES-style Projected Gradient Descent (TPGD)

Input: Model f_θ, input x, budget ε, step size α, steps T

Output: Adversarial input x_T^{adv}

Initialize $x_0^{\text{adv}} \in (x + \mathcal{S}_\infty(\varepsilon)) \cap [0,1]^d$

Compute clean prediction $p_\theta(\cdot \mid x)$

for $t = 0, \dots, T-1$ **do**

  Compute $L_t \leftarrow \mathrm{KL}(p_\theta(\cdot \mid x) \,\|\, p_\theta(\cdot \mid x_t^{\text{adv}}))$

  Ascend $\nabla_x L_t$ and project onto $(x + \mathcal{S}_\infty(\varepsilon)) \cap [0,1]^d$

return x_T^{adv}

Algorithm A.8: AutoAttack Evaluation Wrapper

Input: Model f_θ, evaluation set $\mathcal{D}_{\text{test}}$, AutoAttack component suite $\mathcal{A}_{\text{AA}}$

Output: AutoAttack robust accuracy

Initialize success indicator $s_i \leftarrow 1$ for each test example

for *each component attack* $A \in \mathcal{A}_{\text{AA}}$ **do**

  Generate adversarial candidates $A(x_i, y_i, f_\theta)$

  Set $s_i \leftarrow 0$ for examples misclassified by the component attack

Compute robust accuracy $\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_i s_i$

return *robust accuracy*

Algorithm A.9: Multi-AT Training

Input: Training set $\mathcal{D}_{\text{train}}$, model f_θ, source attack set $\mathcal{E}_{\text{src}}$, loss ℓ, learning rate η_θ

Output: Trained model f_θ^*

for *each training epoch* **do**
    for *each minibatch* $\mathcal{B}$ **do**
        Assign examples in $\mathcal{B}$ uniformly across attacks in $\mathcal{E}_{\text{src}}$
        Generate adversarial examples using the assigned source attack
        Compute the uniform mean adversarial loss over the minibatch
        Update θ using the model optimizer and learning rate η_θ
return f_θ^*

Algorithm A.10: AttackDRO Training

Input: Training set $\mathcal{D}_{\text{train}}$, model f_θ, source attack set $\mathcal{E}_{\text{src}}$, attack weights q, learning rates η_θ, η_q

Output: Trained model f_θ^*

Initialize q uniformly over source attack identities
for *each training epoch* **do**
    for *each minibatch* **do**
        Generate adversarial examples using the source attacks
        Estimate loss or difficulty for each source attack group
        Compute the attack-weighted Group DRO loss using q
        Update θ with the weighted loss
        Update q using exponentiated-gradient reweighting over attack groups
return f_θ^*

Algorithm A.11: Cluster-DRO and AttackDRO++ Training Overview

Input: Training set $\mathcal{D}_{\text{train}}$, model f_θ, source attacks $\mathcal{E}_{\text{src}}$, cluster count K, anchor weight λ_{DRO}, group weights q

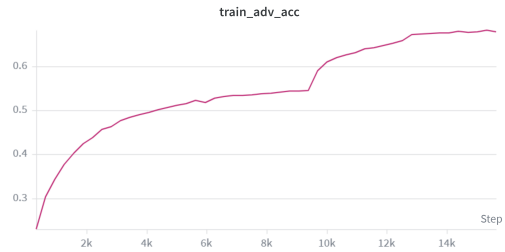
Output: Trained model f_θ^*

Initialize feature bank, clusterer, and uniform group weights q
for *each training epoch* **do**
    Refresh latent clusters from the feature bank when the refresh condition is met
    for *each minibatch* **do**
        Generate source adversarial examples as in Multi-AT
        Extract representations and, for AttackDRO++, augmented clustering features with gradient fingerprints
        Assign each adversarial example to a latent cluster
        Compute the uniform adversarial loss over the minibatch
        Compute the Cluster-DRO loss using cluster weights q
        Combine the uniform anchor and Cluster-DRO terms using λ_{DRO}
        Update model parameters and then update q from cluster-level difficulty estimates
        Add detached augmented features to the feature bank
return f_θ^*

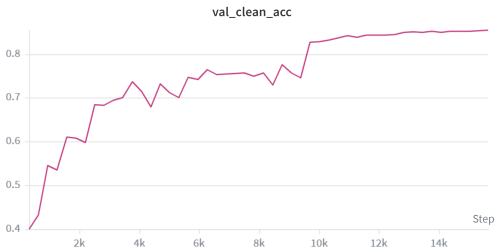
Appendix B

Experimental Logs and Configurations

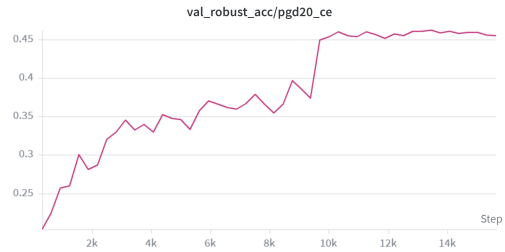
(a) Training loss.



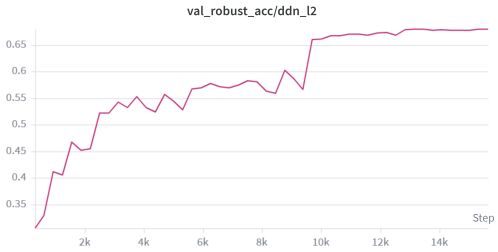
(b) Training accuracy.



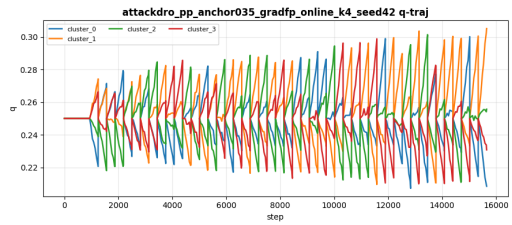
(c) Clean validation accuracy.



(d) PGD-ℓ_∞ validation accuracy.



(e) DDN-ℓ_2 validation accuracy.



(f) Group-weight trajectory.

Figure B.1: Representative W&B dashboard exports for experiment tracking. Panels (a)–(e) track training loss, training accuracy, clean validation accuracy, and validation robustness under PGD-ℓ_∞ and DDN-ℓ_2 during a training run. Panel (f) records the AttackDRO++ group-weight trajectory diagnostic. These exports document training telemetry and are not used as result evidence.

Appendix C

Additional Tables and Visualizations

C.1 Adversarial Perturbation Visualizations

C.1.1 Grouped Adversarial Perturbation Visualizations

Figure C.1 and Figure C.2 summarize adversarial examples and normalized perturbation heatmaps across the main ℓ_∞ and ℓ_2 evaluation attacks. These figures are intended as qualitative support for the attack descriptions in Chapter 2; all heatmaps are normalized for visibility.

Table C.1: Supplementary WRN-28-10 architecture check on CIFAR-10. Values are robust accuracy (%) from a single run and are therefore not included in the main paired claims. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately. Bold values indicate the best method for each metric.

Method	Mean(8) ↑	Worst(8) ↑	AutoAttack ↑
Multi-AT	**64.87**	**47.99**	**44.92**
AttackDRO++	64.68	47.72	43.16

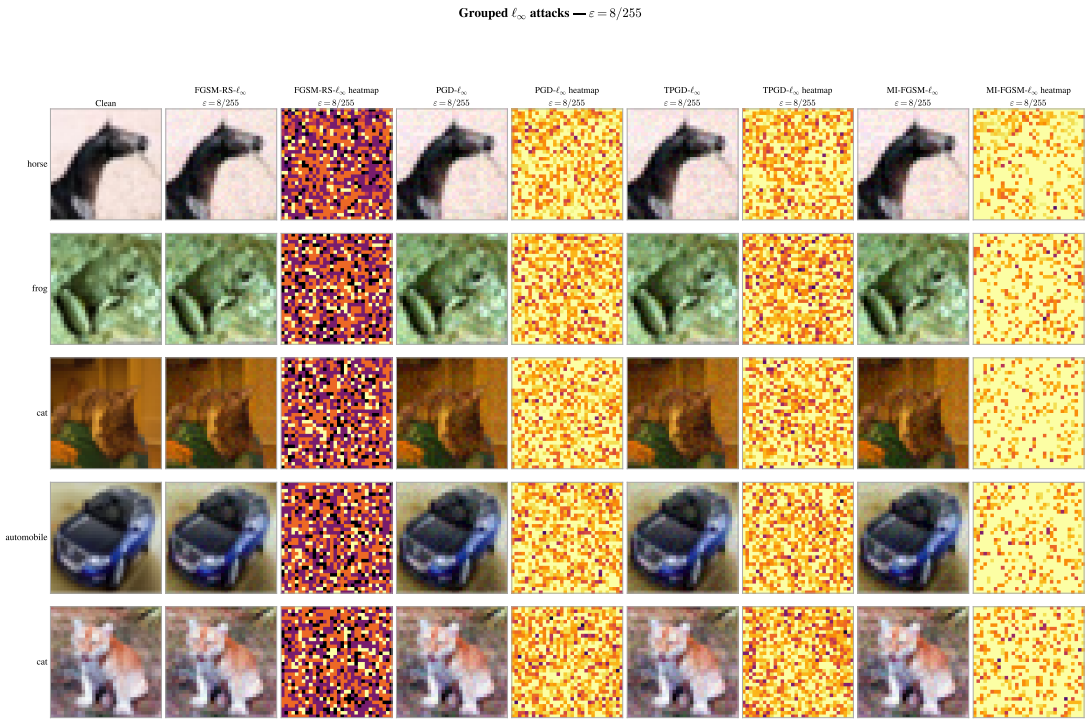


Figure C.1: Grouped ℓ_∞ adversarial examples on CIFAR-10, with adversarial images and normalized perturbation heatmaps.

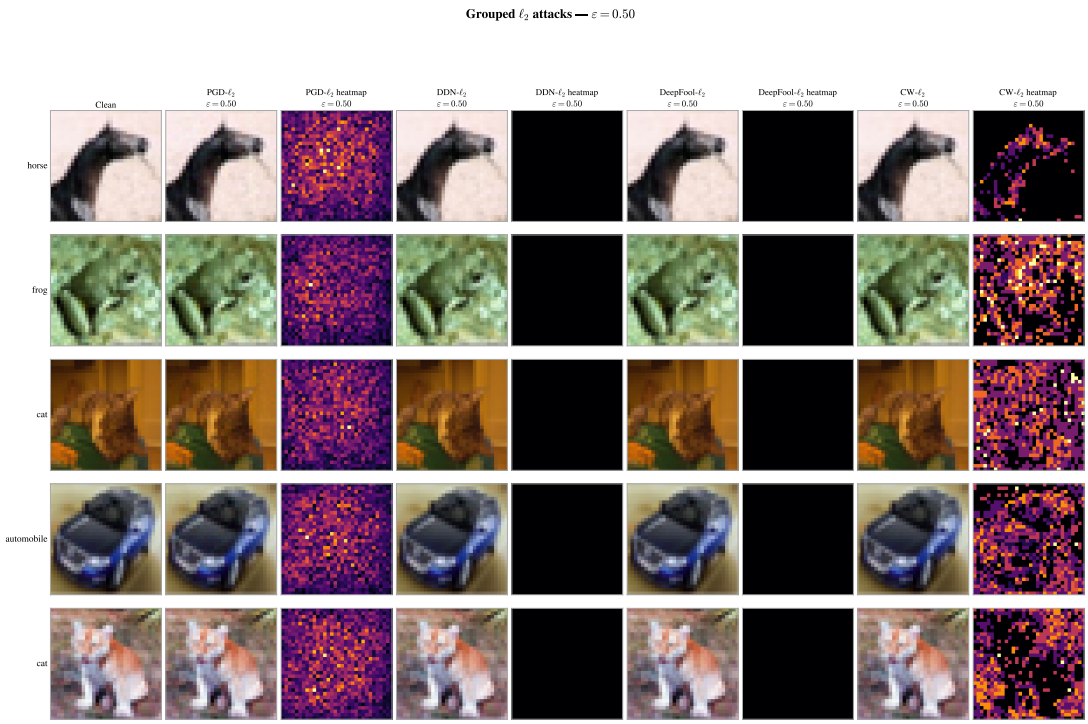


Figure C.2: Grouped ℓ_2 adversarial examples on CIFAR-10, with adversarial images and normalized perturbation heatmaps.

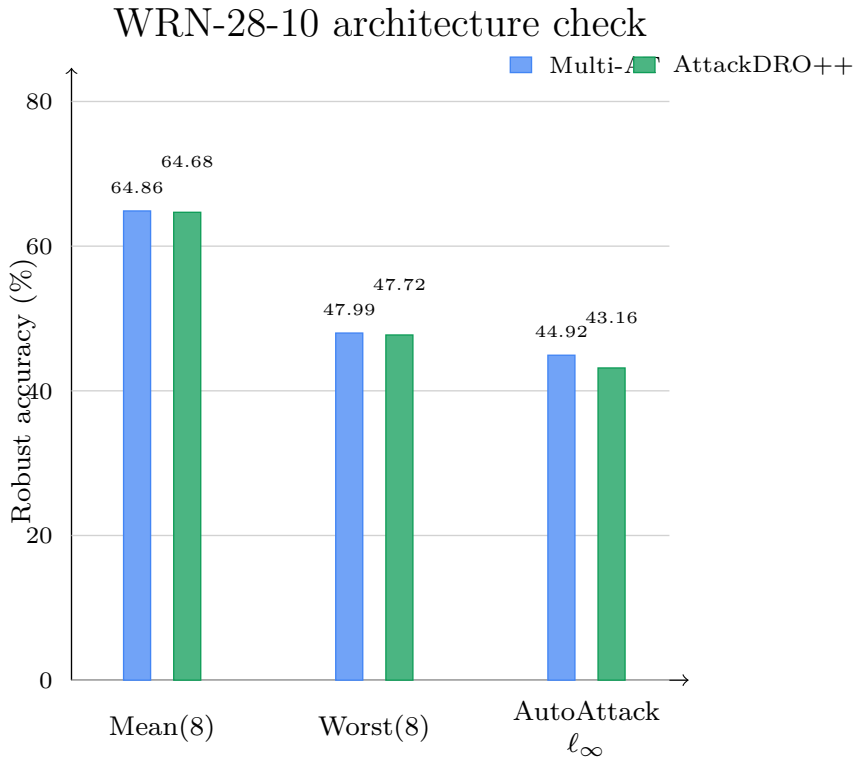


Figure C.3: Supplementary WRN-28-10 architecture check on CIFAR-10. Values are from a single run and are therefore not included in the main paired claims. Mean(8) and Worst(8) are computed over the eight non-AutoAttack attacks; AutoAttack-ℓ_∞ is reported separately.

Appendix D

Compute Resources and Environment Notes

The main experiments were executed through the Python and PyTorch pipeline described in Chapter 5. Google Colab provided the interactive GPU runtime, while Google Drive stored checkpoints, cached adversarial examples, result CSV files, and exported figures. The exact GPU model available to a run was treated as execution metadata rather than an experimental variable; model selection and statistical comparisons are based on saved checkpoints and fixed evaluation scripts, not on wall-clock runtime.

The implementation stack used PyTorch and torchvision for training, TorchAttacks and dedicated AutoAttack/DDN implementations for adversarial evaluation, scikit-learn for MiniBatch K-means, and NumPy, pandas, matplotlib, and SciencePlots for aggregation and figure generation. Weights & Biases was used for telemetry and configuration tracking. The source attack definitions, evaluation attack suite, seed protocol, and reporting metrics are fixed in Chapter 5; this appendix records the compute context only to avoid leaving the environment notes implicit.